
Apple System Design

Interview Questions & Staff-Level Answers

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

Apple System Design — Interview Questions & Staff-Level Answers

Every major design decision offers multiple options with trade-offs and a recommended choice. ASCII diagrams show architecture. Keywords are explained inline with depth. Format: clarify → estimate → diagram → decision matrix → deep dive → failure modes → privacy

Table of Contents

1. [Design iCloud Photo Library](#1-design-icloud-photo-library)
2. [Design Apple Push Notification Service (APNs)](#2-design-apple-push-notification-service-apns)
3. [Design iMessage End-to-End Encrypted Messaging](#3-design-imessage-end-to-end-encrypted-messaging)
4. [Design Apple Maps Routing Service](#4-design-apple-maps-routing-service)
5. [Design the App Store Search & Discovery](#5-design-the-app-store-search--discovery)
6. [Design a Real-Time Health Data Platform (HealthKit Backend)](#6-design-a-real-time-health-data-platform-healthkit-backend)
7. [Design Siri — Voice Assistant Backend](#7-design-siri--voice-assistant-backend)
8. [Design a Privacy-Preserving Analytics System](#8-design-a-privacy-preserving-analytics-system)
9. [Design an Offline-First Sync System (iCloud Drive)](#9-design-an-offline-first-sync-system-icloud-drive)
10. [Design a Distributed Rate Limiter](#10-design-a-distributed-rate-limiter)
11. [Design a Global CDN for App Binary Delivery](#11-design-a-global-cdn-for-app-binary-delivery)
12. [Design a Distributed Cache (like Redis)](#12-design-a-distributed-cache-like-redis)
13. [Design Search Autocomplete (Spotlight / App Store)](#13-design-search-autocomplete-spotlight--app-store)
14. [Design a Key-Value Store](#14-design-a-key-value-store)
15. [Design a Notification Preference & Delivery System](#15-design-a-notification-preference--delivery-system)
16. [Design FaceTime / Real-Time Video Conferencing](#16-design-facetime--real-time-video-conferencing)
17. [Design Apple Pay Backend](#17-design-apple-pay-backend)
18. [Design Real-Time Collaborative Editing (iWork / Notes)](#18-design-real-time-collaborative-editing-iwork--notes)
19. [Design an ML Feature Store & Model Serving Platform](#19-design-an-ml-feature-store--model-serving-platform)
20. [Design a Large-Scale Telemetry & Crash Reporting Pipeline](#20-design-a-large-scale-telemetry--crash-reporting-pipeline)
21. [Design Apple TV+ Video Streaming Platform](#21-design-apple-tv-video-streaming-platform)

- 22. [Design Find My Network](#22-design-find-my-network)
 - 23. [Design iCloud Keychain](#23-design-icloud-keychain)
 - 24. [Design Apple Music](#24-design-apple-music)
 - 25. [Design Game Center — Leaderboards & Matchmaking](#25-design-game-center--leaderboards--matchmaking)
 - 26. [Design HomeKit / Matter Smart Home Hub](#26-design-homekit--matter-smart-home-hub)
 - 27. [Design Apple ID & Authentication Platform](#27-design-apple-id--authentication-platform)
 - 28. [Design iCloud Calendar & Contacts Sync](#28-design-icloud-calendar--contacts-sync)
-

1. Design iCloud Photo Library

Clarifying Questions

- Scale: 500M users or full 1B iCloud base?
- Scope: upload + storage + multi-device sync, or just one of those?
- Multi-device sync in scope? (phone + Mac + iPad)
- Shared albums?
- Maximum file size? (iPhone 15 ProRes can produce multi-GB video clips)
- Consistency model: how long can it take for a photo uploaded on iPhone to appear on Mac?

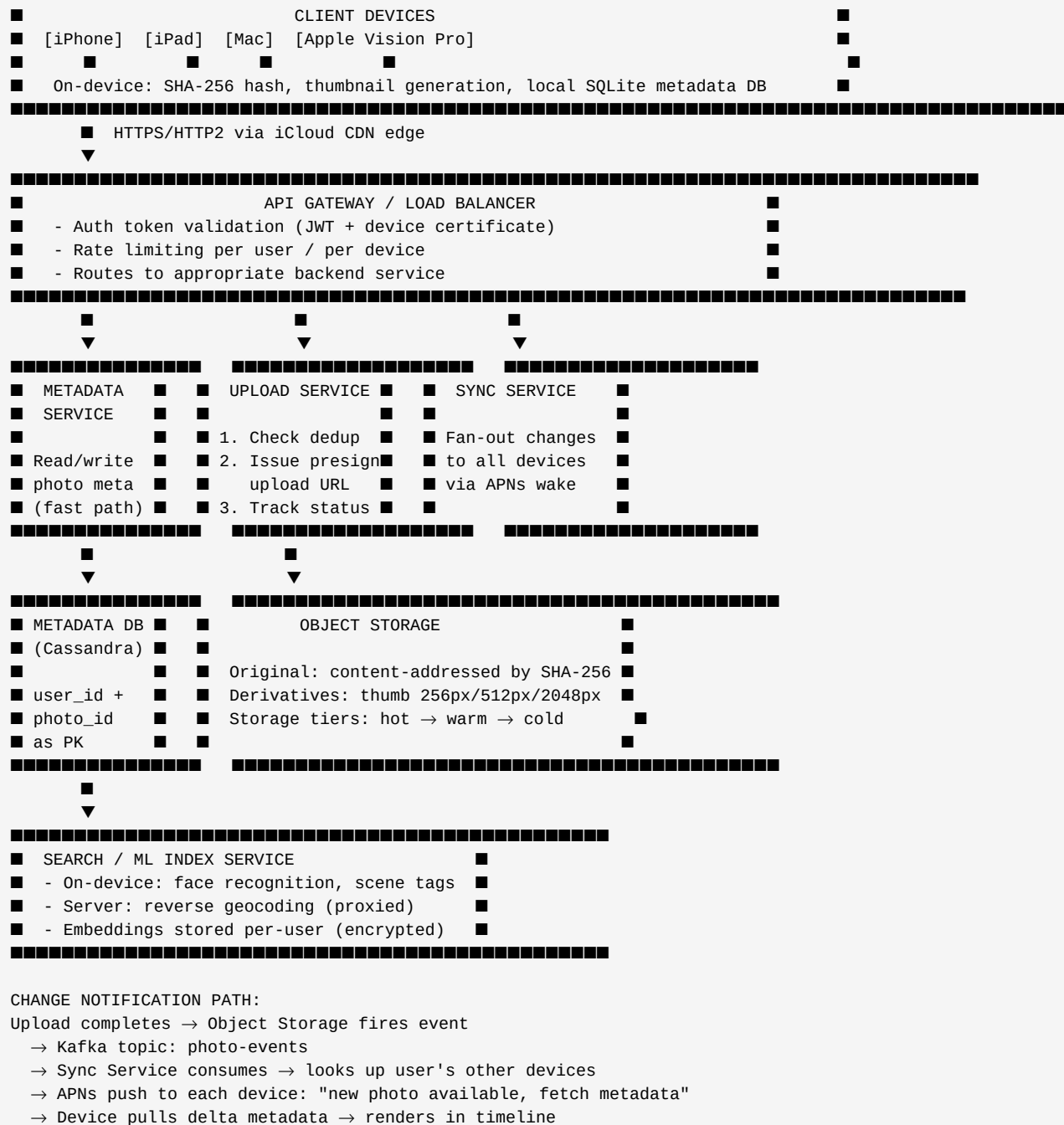
Estimation

```
500M active iCloud Photos users
Avg library: 5,000 photos × 4MB avg = 20GB per user
Total storage: 500M × 20GB = 10 exabytes

Upload rate: 500M users × 5 uploads/day ÷ 86,400s = ~29,000 uploads/sec (peak: 3x = ~87,000/sec)
Read QPS (browsing): 10x writes = 290,000 reads/sec
Metadata QPS (timeline, search): 100x writes = ~3M QPS

Network bandwidth (uploads): 29,000/sec × 4MB avg = ~116 GB/s at peak
```

High-Level Architecture Diagram



Key Design Decision 1: Metadata Storage

The problem: Metadata (filename, GPS, timestamp, album membership, face tags) needs:

- Very high read QPS (browsing, search)
- Writes on every upload

- Range queries (date range, album, location bounding box)
- User-scoped data (partition by user_id)

Options

Option	How It Works	Pros	Cons	Best When
Cassandra	Wide-column store; (userid, photoid) partition+clustering key; LSM writes	Scales horizontally, tunable consistency, excellent write throughput, designed for this pattern	Eventual consistency; no joins; secondary index queries are scatter-gather; operational complexity	High write volume, user-partitioned data, team has Cassandra expertise
PostgreSQL (sharded)	Relational; shard by user_id % N; connection pooling via PgBouncer	Full SQL, ACID, rich query support, easier development	Sharding complexity; cross-shard queries impossible; scaling requires re-sharding	Moderate scale, complex query requirements, simpler operations
DynamoDB	Managed key-value/document; partition key = userid; sort key = photoid	Fully managed, auto-scaling, single-digit ms latency, no ops overhead	Limited query patterns (no range queries across partition key); cost scales with throughput; vendor lock-in	Want zero ops, AWS ecosystem, simpler access patterns
TiDB	Distributed MySQL-compatible; automatic sharding (Raft-based regions)	MySQL compatibility, auto-sharding, HTAP (OLTP + OLAP), strong consistency	Newer, smaller community; higher latency than Cassandra for pure writes; complex to tune	Need SQL semantics + auto-sharding without manual partitioning

■ Recommended: Cassandra

Reason: iCloud Photos is a canonical Cassandra use case. Data is naturally partitioned by user_id. Write throughput of 29K–87K/sec is well within Cassandra's sweet spot. The query patterns are simple: "get all photos for user X sorted by date" — exactly what Cassandra's clustering keys handle. The trade-off (no ACID, eventual consistency) is acceptable because a 1-2 second delay before a new photo appears on a second device is fine for this use case.

```

Schema:
CREATE TABLE photos (
  user_id      UUID,
  photo_id     TIMEUUID,  -- time-sorted UUID = natural chronological order
  filename     TEXT,
  taken_at     TIMESTAMP,
  location     frozen<geo_point>,
  sha256       TEXT,      -- content hash for dedup
  size_bytes   BIGINT,
  status       TINYINT,   -- 0=uploading, 1=available, 2=deleted
  PRIMARY KEY (user_id, photo_id)
) WITH CLUSTERING ORDER BY (photo_id DESC);
-- Newest photos first = efficient timeline query

```

Key Design Decision 2: Object Storage for Photos

The problem: Store 10 exabytes of binary data. Reads are random-access (user requests specific photo). Writes are write-once (photo never changes after upload).

Options

Option	How It Works	Pros	Cons	Best When
Custom object store (like Apple uses)	Erasure coding (e.g., Reed-Solomon 12+4); data striped across nodes; content-addressed by hash	Maximum control, lowest cost at Apple scale, custom hardware optimization	Enormous engineering investment; only makes sense at >1 exabyte scale	Apple/Google/Microsoft scale
Amazon S3	Replicated blob storage; 11 nines durability; geo-replication	Near-zero ops, battle-tested, rich ecosystem, presigned URL upload bypasses servers	Cost at Apple scale (\$20B+/year), vendor dependency, egress fees	Startups to mid-scale; fast to market
HDFS + custom metadata	Distributed filesystem; 3x replication; NameNode for metadata	Large-file friendly, open-source, no egress fees	Not designed for small files or object semantics; NameNode is SPOF; complex ops	Hadoop-centric analytics workloads, not user-facing photo storage

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Ceph	Open-source distributed storage; RADOS object layer; erasure coding	Self-hosted, no vendor lock-in, object + block + file interfaces	Complex to operate; tuning required; not battle-tested at Apple's exact scale	On-prem object storage, telecom, OpenStack environments

■ Recommended: Custom object store (for Apple) / S3 (for a startup building iCloud-like)

At Apple's scale, custom object storage with erasure coding is the only cost-viable option. At 10 exabytes, even modest improvements in storage efficiency save hundreds of millions of dollars.

What is Erasure Coding? (Deep Dive)

```

Reed-Solomon Erasure Coding (12+4 configuration):
- Split data into 12 data chunks + 4 parity chunks = 16 total chunks
- Distribute across 16 different drives/nodes
- Can reconstruct original data from ANY 12 of the 16 chunks
- 4 drives can fail simultaneously with zero data loss

vs. 3x Replication:
- 3x replication: store 3 complete copies
- Storage overhead: 3x (1TB data = 3TB stored)
- Erasure coding 12+4: overhead = 16/12 = 1.33x (1TB data = 1.33TB stored)
- At 10 exabytes: saves 16.7 exabytes of storage = massive cost difference

Drawback of erasure coding:
- Reconstruction requires reading 12 shards (vs 1 shard for replication)
- Higher read latency for degraded reads
- More CPU for encode/decode
- Mitigated by: hot tier uses 3x replication; cold tier uses erasure coding

```

Key Design Decision 3: Deduplication Strategy

Problem: Many users have the same meme, stock photo, or forwarded image. Store it once.

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Client-side dedup (hash before upload)	Client computes SHA-256 → sends hash to dedup service → if exists, skip upload	Zero upload bandwidth wasted; instantly handles duplicates	Hash oracle attack: malicious client claims hash of a file they don't own; server trusts without verifying

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Server-side dedup (upload then dedup)	Client uploads full file → server hashes → dedup happens server-side	Secure: server verifies content	Wastes bandwidth — file transferred even if already stored
Convergent encryption (best of both)	Client encrypts file with key = Hash(plaintext) → uploads ciphertext → server stores by ciphertext hash → different users' same file gets same ciphertext	Privacy-preserving dedup: server can dedup without seeing plaintext	Key inference attack: if attacker knows the plaintext, they can predict the key; mitigated by adding user-specific salt

■ Recommended: Client-side dedup with signed tokens

Flow:

1. Client: hash_client = SHA-256(photo_bytes)
2. Client: POST /dedup-check { hash: hash_client, user_id: X }
3. Server: checks dedup store → if exists, returns:
{ status: "exists", upload_token: HMAC(server_secret, hash+user_id+timestamp) }
4. Client: sends upload_token → server verifies → creates metadata record pointing to existing blob
5. If not exists: server issues presigned S3-style URL for direct upload

Security: upload_token is time-limited (5 min TTL) and user-scoped
prevents hash oracle attacks (token binds hash to authenticated user)

Key Design Decision 4: Multi-Device Sync — Conflict Resolution

Problem: User edits photo caption on iPhone (offline) and Mac (offline). Both come online. Which version wins?

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Last-Write-Wins (LWW)	Server keeps version with latest wall clock timestamp	Simple to implement, O(1) storage	Clock skew: device clocks can be wrong by minutes; legitimate edits silently lost	Non-critical metadata, high-volume events where losing one edit is acceptable

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Vector Clocks	Each device tracks logical clock vector; server detects concurrent versions	Detects true concurrency; no false conflict suppression	Grows with number of devices; garbage collection needed; conflict still needs resolution	Important user data where you must detect conflicts
CRDTs (Conflict-free Replicated Data Types)	Data structure mathematically guaranteed to converge regardless of merge order	Zero conflicts by design; no coordination needed; works offline	Limited data models; not all operations are CRDT-able; tombstone accumulation	Collaborative editing, sync-heavy workloads, offline-first apps
Operational Transform (OT)	Operations transformed to account for concurrent edits (Google Docs style)	Handles arbitrary concurrent text editing	Complex algorithm; hard to implement correctly; requires total ordering of operations	Real-time collaborative text editing

■ **Recommended: CRDTs for structured metadata, LWW with vector clock conflict detection for free-form fields**

iCloud Photos specific design:

Photo set (album membership) → OR-Set CRDT:

Add photo to album: tag = (photo_id, device_id, timestamp)

Remove photo: add tombstone = (photo_id, device_id, remove_timestamp)

Merge: union of all adds; remove wins over add if tombstone timestamp > add timestamp

→ Merging from 3 devices always produces same result, no matter merge order ✓

Caption (free-form text) → LWW with vector clock:

Device A edits at VC=[3,0,0]; Device B edits at VC=[0,2,0]

Server detects: neither dominates the other = CONFLICT

Resolution: present both versions to user (like Dropbox "conflicted copy") OR auto-pick latest by wall clock with user-visible notification

Face tags → G-Set CRDT:

Once a face is identified, it's added to a grow-only set per photo

Multiple devices identifying the same face = idempotent add

Deep Dive: What is a CRDT?

A CRDT is a data structure with a mathematically-guaranteed merge function where:

```

merge(A, merge(B, C)) = merge(merge(A, B), C) ← Associativity
merge(A, B) = merge(B, A) ← Commutativity
merge(A, A) = A ← Idempotence

```

If these three properties hold, ANY merge order produces the SAME result.
This means: no coordination needed. No locking. No consensus. Just merge.

Example: G-Counter (Grow-only Counter)

```

Node A counter: {A: 3, B: 0, C: 0}
Node B counter: {A: 0, B: 5, C: 0}
Node C counter: {A: 3, B: 2, C: 1} ← C saw A's updates

```

```

Merge(A, B): take max per entry = {A:3, B:5, C:0} → value = 8
Merge(A, B, C): {A:3, B:5, C:1} → value = 9

```

Result is the same regardless of whether you merge (A then B then C)
or (C then A then B). Guaranteed convergence.

Why it's efficient:

- No round trips to server for coordination
- Works fully offline (merge when reconnected)
- Scales to any number of replicas

Drawbacks:

- Not all data structures have CRDT variants
- Tombstones (deleted items) must be retained forever or GC is needed
- Some CRDTs violate intuition: OR-Set allows re-adding deleted items
- Cannot enforce invariants like "balance > 0" (no negative balance guarantee)
- PN-Counter can go negative due to concurrent decrements

When NOT to use CRDTs:

- When you need strong consistency (financial transactions)
- When your operations require reading current state before writing
(e.g., "add item only if not already present" – that's a read-modify-write)
- When deletion semantics are strict (tombstones complicate GC)

Failure Modes

Component	Failure	Detection	Recovery
Upload Service	Pod crash mid-upload	Client timeout (30s)	Client retries with exponential backoff; dedup check prevents re-upload of complete file
Object Storage	Node failure	Heartbeat monitoring	Erasur coding: reconstruct from remaining shards; hot standby nodes

Component	Failure	Detection	Recovery
Metadata DB (Cassandra)	Node down	Gossip protocol detects in < 10s	Reads/writes route to other replicas (quorum = 2 of 3)
Sync Service	Consumer lag	Kafka consumer group lag metric	Auto-scaling: add consumer instances; Kafka retains events for 7 days
APNs delivery	Device offline	APNs stores notification (30-day TTL)	Device pulls delta on reconnect; full resync if gap > 30 days
Dedup store	Redis failure	Health check	Fall back to server-side dedup (upload and check); slightly more bandwidth wasted

Privacy Design

- Photos encrypted at rest with per-user AES-256 keys derived from user's iCloud key
 - Advanced Data Protection** (opt-in): end-to-end encrypted; Apple has zero key access
 - Face recognition runs entirely on-device — face embeddings never sent to server
 - GPS coordinates reverse-geocoded server-side via Private Relay (Apple's IP is never sent, user's IP hidden)
 - No content scanning on Apple's servers (unlike Google Photos)
-

2. Design Apple Push Notification Service (APNs)

Clarifying Questions

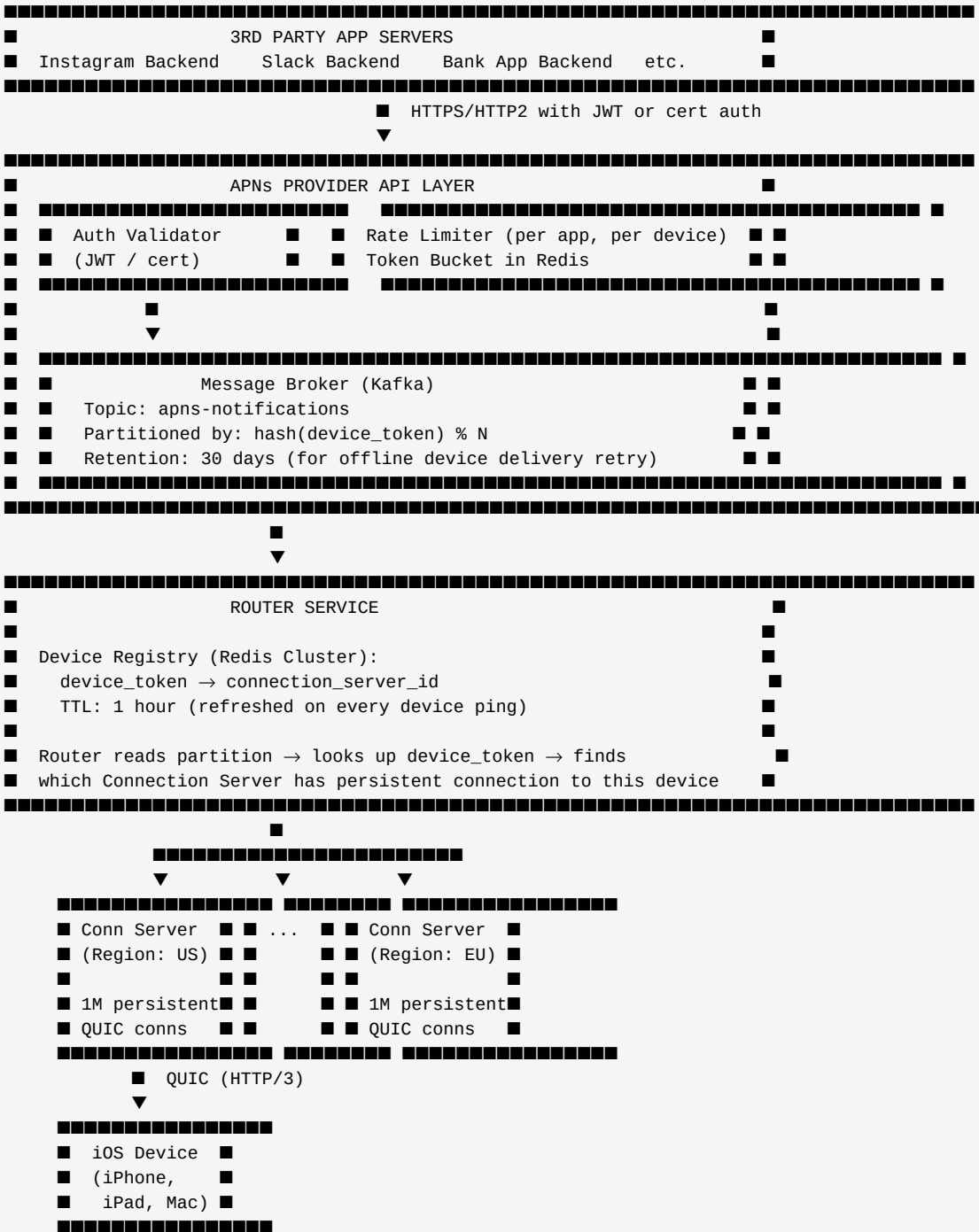
- Delivery guarantee: best-effort or at-least-once?
- What happens when device is offline? How long to retain?
- Ordered delivery required?
- Acceptable end-to-end latency? (< 1 second for priority notifications)
- Scope: visible alerts only, or also silent background wakeups?

Estimation

2B registered devices; ~1B daily active
50 notifications/device/day = 50B/day = ~578,000/sec average; 5M/sec peak (breaking news event)
Payload avg: 4KB → 578K/sec × 4KB = ~2.3 GB/sec average throughput

Persistent connections: ~500M concurrent active device connections
Connection state per device: ~2KB → 500M × 2KB = ~1TB of connection state (distributed)

Architecture Diagram



OFFLINE DEVICE PATH:

Router: device not connected → store notification in Redis

key: device_token → value: latest notification per app

TTL: 30 days

When device reconnects: fetch from Redis + deliver

Expired (>30 days): feedback API reports expired token to app server

Key Design Decision 1: Transport Protocol for Device Connections

Problem: Maintain persistent connections to 500M–1B devices. When IP changes (Wi-Fi → cellular), connection must survive or recover instantly.

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
TCP + TLS (traditional)	OS-level reliable stream; TLS on top	Universal support; well-understood	New TLS handshake on IP change (~2–3 RTT = 50–100ms); TCP HOL blocking; separate connection per stream	Legacy systems; when QUIC not available
WebSocket over TLS	HTTP upgrade to full-duplex TCP stream	Browser-compatible; bidirectional	Still TCP; IP migration = reconnect + re-handshake; no multiplexing	Browser-based push or IoT
QUIC (HTTP/3)	UDP-based; own reliability; connection ID (not IP:port); 0-RTT resumption	0-RTT reconnect on IP change; no HOL blocking; built-in TLS 1.3; connection migration	UDP sometimes blocked by enterprise firewalls; newer, less universal	Mobile clients, unstable networks, Apple's exact use case
MQTT over TLS	Pub/sub protocol designed for IoT; very lightweight (2-byte header)	Extremely low overhead; QoS levels (at-most-once, at-least-once, exactly-once); designed for constrained devices	No native HTTP; requires MQTT broker; less flexible message routing	IoT devices, sensors, battery-critical use cases

■ **Recommended: QUIC (HTTP/3)**

Why QUIC is ideal for APNs:

1. CONNECTION MIGRATION:

Traditional TCP: connection = (src_ip, src_port, dst_ip, dst_port)

If device switches from Wi-Fi (IP: 192.168.1.5) to cellular (IP: 10.0.0.42):

- TCP connection is dead. Must reconnect. TLS handshake: 2-3 RTTs.
- During handshake: notifications buffered, latency spike.

QUIC: connection = Connection ID (random 64-bit number)

If device switches networks:

- Same Connection ID continues working on new IP
- Zero reconnection overhead. Notifications flow immediately.
- Apple devices switch between Wi-Fi and cellular constantly.

2. 0-RTT RESUMPTION:

After a device wakes from sleep (common pattern):

TLS 1.2: 2 full round trips before any data

QUIC 0-RTT: client sends data IN THE FIRST PACKET using cached session ticket

- Notification delivery starts ~50ms faster per wake cycle
- Across 500M devices waking/sleeping constantly: massive aggregate benefit

3. NO HOL BLOCKING:

TCP: if one packet is lost, ALL streams wait for retransmission

QUIC: stream-level independence – lost packet in one stream doesn't block others

- APNs sends multiple notification streams per connection; independence matters

QUIC Drawback:

Enterprise firewalls may block UDP → QUIC falls back to TCP/TLS automatically

Apple mitigates: always maintain TCP fallback path

Key Design Decision 2: Message Broker Between Provider API and Connection Servers

Problem: 5M notifications/sec peak. Notifications can arrive faster than connection servers can deliver them. Need decoupling + backpressure + durability.

Options

Option	How It Works	Pros	Cons	Best When
Kafka	Durable, partitioned log; zero-copy delivery; consumer groups	High throughput (millions/sec per broker); durable (replay for offline devices); fan-out to multiple consumer groups	Operational complexity; higher latency than in-memory queues (ms-level)	High throughput, need durability, need replay

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Redis Streams	Lightweight streams in Redis; consumer groups; in-memory with AOF	Lower latency than Kafka (~sub-ms); simpler operations	Memory-bound (can't hold 30-day offline backlog in RAM); less durable than Kafka	Lower scale, latency-critical, small message retention window
RabbitMQ	AMQP protocol; sophisticated routing (fanout, topic, headers); dead-letter queues	Excellent routing primitives; low latency push delivery	Lower throughput than Kafka; not designed for high fanout; clustering is complex	Complex routing logic; moderate scale
Direct delivery (no broker)	Provider API → Router → Connection Server directly	Lowest latency; simplest path	No durability for offline devices; tight coupling; if connection server is slow, provider API backs up	Latency-critical, online-only, no offline delivery needed

■ Recommended: Kafka for main path + Redis for offline storage

Two-tier design:

Tier 1: Kafka (main delivery path)

Provider API → Kafka topic partitioned by device_token

Connection Server fleet consumes from Kafka

Partitioning by device_token ensures all messages for one device

go to the same partition → preserves per-device ordering

Tier 2: Redis (offline device storage)

Router detects device not connected

→ Store notification in Redis: key = device_token:app_id, value = latest notification

→ TTL = 30 days

→ On device reconnect: fetch from Redis, deliver, delete

Why Kafka for main path:

- Durability: if a connection server crashes, another reads from Kafka offset
- Fan-out: multiple consumer groups (delivery, analytics, rate limiting audit)
- Backpressure: Kafka absorbs bursts (breaking news → 50x normal rate)
- Zero-copy: sendfile() makes Kafka broker near-zero CPU for pass-through

Why Redis for offline, not Kafka:

- O(1) lookup: device reconnects → single Redis GET for pending notifications
- Kafka lookup for a specific device requires scanning partition
- Space: Redis stores only latest-per-app (not full history)
 - APNs semantics: only latest notification per app is delivered after offline period

Failure Modes

<i>Failure</i>	<i>Impact</i>	<i>Detection</i>	<i>Recovery</i>
Connection Server crash	~1M devices lose persistent connection	Load balancer health check	Devices detect QUIC connection drop; reconnect to different server within 1s; device registry TTL expires old entry
Kafka broker failure	Notifications buffered by producers	Kafka ISR monitoring	Kafka leader election < 30s; producers retry with backoff
Router service down	Notifications pile up in Kafka	Kafka consumer lag alert	Auto-scale router instances; notifications retried from Kafka offset
Device token expired	Notification not deliverable	Delivery tracker records 410 Gone	Feedback service notifies app server to remove token; user re-registers on next app open
Notification storm (10x normal rate)	Provider API overwhelmed	QPS monitoring	Rate limit per app (configurable per app developer account); shed low-priority notifications first

3. Design iMessage End-to-End Encrypted Messaging

Clarifying Questions

- 1:1 and group messages?
- Attachments (images, video) or text only?
- Multi-device: same message history on iPhone + Mac + iPad?
- Read receipts, typing indicators?
- What consistency model? (Can messages arrive out-of-order?)

Estimation

1B iMessage users

40 messages/user/day = 40B messages/day = ~463,000/sec average; peak ~3x = 1.4M/sec

Average encrypted payload: 2KB (1KB content + ~1KB E2E overhead)

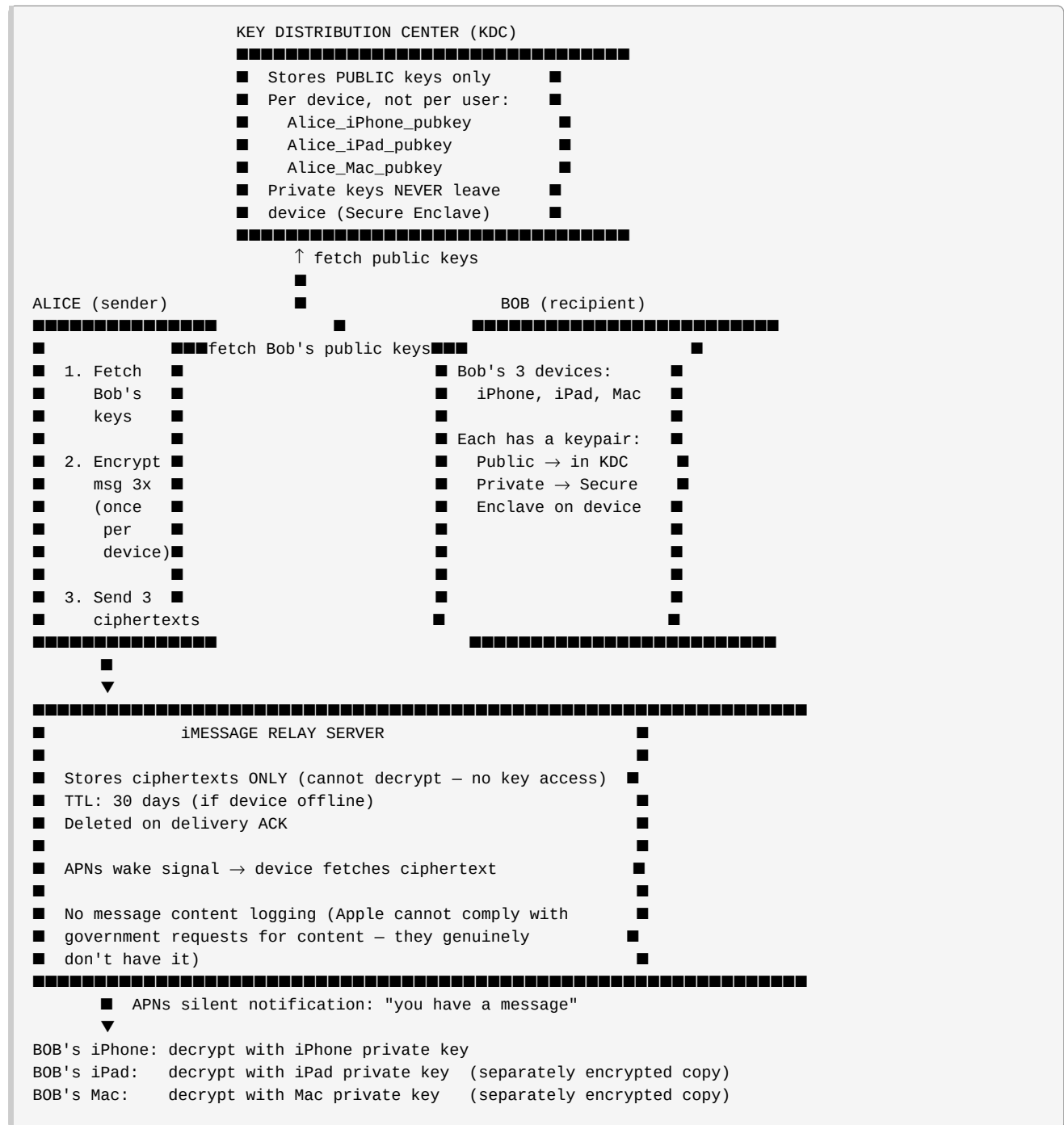
Throughput: 1.4M/sec × 2KB = ~2.8 GB/sec peak

Attachments: ~10% of messages have attachments

→ Stored separately in object storage, not in message relay

→ Message carries a reference URL, not inline content

Architecture Diagram



Key Design Decision 1: Encryption Model

Problem: Multiple devices per user. Each device needs its own encrypted copy. How do you manage keys?

Options

Option	How It Works	Pros	Cons	Best When
Per-device asymmetric keys (iMessage model)	Each device generates its own keypair; sender encrypts once per recipient device	Perfect forward secrecy per device; private key never leaves device; Apple cannot read messages	N ciphertexts per message where N = total recipient devices; KDC is a trust root (Apple controls it)	End-to-end encryption, multi-device, high security
Shared account key	One keypair per user; synced across devices	Simpler; one ciphertext per message	Key must be synced across devices — how? If sync server stores key, server can read messages	Low-security apps, single-device scenarios
Signal Protocol (Double Ratchet)	Combines DH key exchange with ratcheting for forward secrecy and post-compromise security	Gold standard for E2E messaging; perfect forward secrecy; post-compromise security (ratchet advances)	Complex implementation; key rotation must be handled; not trivially multi-device	WhatsApp, Signal — single-primary-device model
MLS (Messaging Layer Security)	IETF standard for scalable E2E encrypted group messaging; TreeKEM for group key evolution	Scales to large groups; efficient key update (log N operations); standard	Newer, less battle-tested; complex to implement	Large group chats with E2E encryption

■ **Recommended: Per-device keys (like iMessage) for 1:1, MLS for large groups**

Why per-device keys work for iMessage:

1. Alice sends to Bob who has 3 devices:
 `encrypt(plaintext, Bob_iPhone_pubkey) = ciphertext_A [2KB]`
 `encrypt(plaintext, Bob_iPad_pubkey) = ciphertext_B [2KB]`
 `encrypt(plaintext, Bob_Mac_pubkey) = ciphertext_C [2KB]`
 → Send ciphertext_A, B, C to relay server
 2. Bob's iPhone fetches ciphertext_A, decrypts with iPhone_privkey
 Bob's iPad fetches ciphertext_B, decrypts with iPad_privkey
 etc.
 3. Each device has read its own copy independently.
 Apple relay sees 3 ciphertexts but cannot decrypt any.
- Cost: 3 encryptions + 3 network transfers per message
 → At 1.4M messages/sec, this is manageable for Apple's infrastructure
 → For a 10-device user: 10 encryptions – still fine

KDC Attack Vector (Apple's Achilles heel):
 Apple controls KDC. A state actor could compel Apple to:
 → Register a rogue public key for Bob's devices
 → Alice fetches rogue key → encrypts to attacker too
 → Attacker can read messages (MITM)

Apple's mitigation (2023): Key Transparency
 → Public, append-only, auditable log of all public key registrations
 → Client can verify its registered public key matches what others see
 → Any key substitution would be detectable
 → Similar to Certificate Transparency for TLS

Key Design Decision 2: Message Store — How Long to Retain on Server

Options

Option	How It Works	Pros	Cons
Short-lived relay (iMessage model)	Server stores ciphertext for delivery only; deleted on ACK; max 30-day TTL for offline	Minimal server storage; server never has long-term message access	If device is offline > 30 days and no iCloud backup, messages are lost
Permanent server storage (encrypted)	All messages stored indefinitely on server, E2E encrypted	Message history available on any new device	Server must store more data; backup encryption key management complexity
Client-side-only	Messages only on device; no server backup	Maximum privacy	Lost if device is lost and no local backup

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
iCloud Backup (iMessage in iCloud)	Messages synced to iCloud encrypted; available on new devices	History available on new device	iCloud encryption key management; Advanced Data Protection needed for true E2E

■ **Recommended: Short-lived relay + iCloud encrypted backup (user-controlled)**

Key Design Decision 3: Group Message Key Management

Problem: 20-person group chat. Someone joins. Someone leaves. How do keys evolve?

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Fan-out to each member	Sender encrypts once per group member (like iMessage does today)	Simple; no shared group key to manage	$O(N \times D)$ encryptions where N =members, D =devices per member; expensive for large groups
Shared group key + key rotation on membership change	Encrypt once with group key; rotate key on join/leave	$O(1)$ encryption per message	Key rotation on every membership change; new joiner can't read history (good); leaver technically has old key
MLS / TreeKEM	Binary tree of DH keys; join/leave requires $O(\log N)$ key updates	Efficient for large groups; forward secrecy; post-compromise security	Complex; newer standard; implementation subtlety

■ **Recommended: Fan-out for small groups (< 50), MLS for large groups (50+)**

4. Design Apple Maps Routing Service

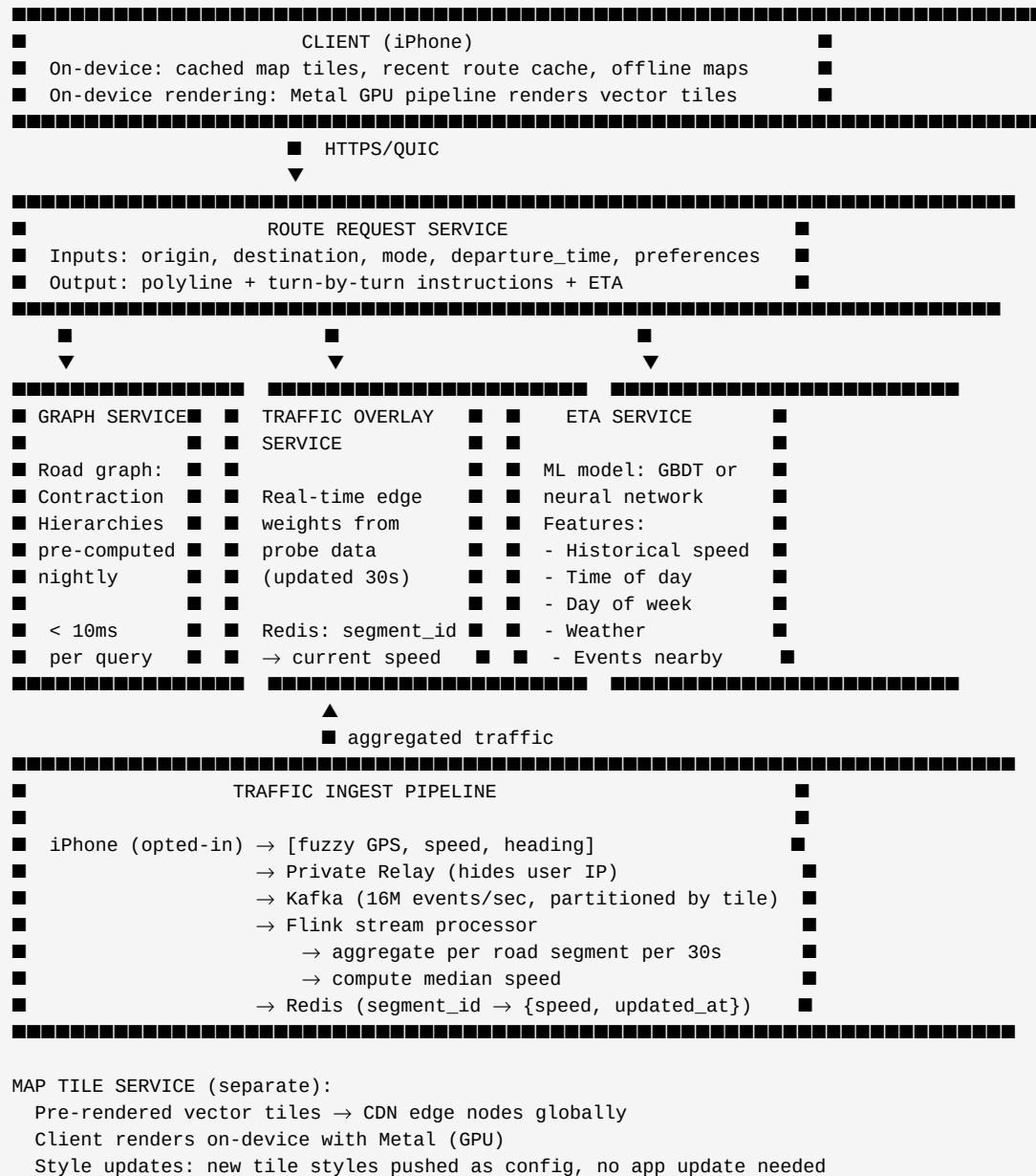
Clarifying Questions

- Turn-by-turn with real-time traffic, or static routing?
- Transport modes: driving, walking, cycling, transit?
- Global or one region?
- ETA accuracy requirement (± 2 min? ± 10 min?)
- Traffic update frequency?

Estimation

50M daily routing requests = ~ 578 requests/sec average; peak $\sim 5\times = \sim 2,900$ /sec
Probe data from opted-in iPhones: 500M devices $\times$ 1 location/30sec = ~ 16 M probe events/sec
Road graph: ~ 1 B edges globally (OpenStreetMap scale)
Map tiles: ~ 10 TB of vector tile data at all zoom levels (pre-computed)

Architecture Diagram



Key Design Decision 1: Routing Algorithm

Problem: Find the fastest route on a graph with 1B+ edges in < 10ms.

Options

<i>Option</i>	<i>How It Works</i>	<i>Latency</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Dijkstra	BFS from source with priority queue; optimal shortest path	$O((V+E) \log V)$; ~seconds on global graph	Simple; correct	Way too slow for global graph (1B edges)	Toy examples, very small graphs
A* (A-star)	Dijkstra + heuristic (straight-line distance) to prune search space	10x–100x faster than Dijkstra	Better than Dijkstra; intuitive	Still too slow for global graph without preprocessing	City-scale routing, game pathfinding
Contraction Hierarchies (CH)	Offline: contract unimportant nodes, add shortcuts. Online: bidirectional Dijkstra on contracted graph	< 10ms on global graph	Extremely fast queries; optimal paths	Expensive preprocessing (~hours); can't handle arbitrary dynamic weights easily	Production GPS navigation (Google Maps, Apple Maps, HERE)
Hub Labeling	Pre-compute for each node: set of "hub" nodes it passes through; routing = intersect hub sets	< 1ms query	Fastest known algorithm	Preprocessing even more expensive; less flexible for traffic updates	When absolute minimum query latency is needed
OSRM (Open Source Routing Machine)	CH-based; pre-computes the full CH graph offline	~1ms	Open source; production-ready	Hard to integrate real-time traffic weights	Open-source GPS routing projects

■ **Recommended: Contraction Hierarchies (CH)**

What are Contraction Hierarchies? (Deep Dive)

PROBLEM: Dijkstra on a graph with 1B edges takes 10+ seconds. Not acceptable for navigation.

KEY INSIGHT: When driving from NYC to LA, you don't care about every side street. You get on a highway (high-importance road) quickly and stay on it.

OFFLINE PREPROCESSING (runs nightly, takes hours):

1. Rank each node by "importance" (based on edge count, road type, speed)
2. Process nodes from LEAST important to MOST important
3. For each node v being contracted:
 - For each pair of neighbors (u, w) of v :
 - If the shortest path $u \rightarrow w$ goes through v : add SHORTCUT edge $u \rightarrow w$ with weight = $\text{weight}(u \rightarrow v) + \text{weight}(v \rightarrow w)$
 - Now v can be removed from the graph for query purposes
4. Result: a hierarchy where high-importance nodes (highways) are densely connected

QUERY (online, < 10ms):

Bidirectional Dijkstra: search forward from source, backward from target

Both searches only relax edges to HIGHER-importance nodes

When the two searches meet at a high-importance "hub" node $\rightarrow$ optimal path found

Diagram:

Before contraction:

NYC_SideStreet1 ■■■■

NYC_SideStreet2 ■■■■■■ $\rightarrow$ I-95_OnRamp ■■ $\rightarrow$ I-95_Highway ■■ $\rightarrow$... $\rightarrow$ LA

NYC_SideStreet3 ■■■■

After contraction:

NYC (aggregated shortcut) ■■ $\rightarrow$ I-95_Start ■■ $\rightarrow$ I-95_End ■■ $\rightarrow$ LA (aggregated)

Millions of local streets replaced by shortcuts between important nodes

TRAFFIC OVERLAY:

CH preprocessing ignores traffic (uses free-flow speeds)

At query time: multiply CH-computed road segments by current traffic factor

This is an approximation (not globally optimal with traffic) but fast enough

For 100% traffic accuracy: Customizable CH (CRP) recomputes edge weights nightly

Performance numbers (approximate):

Naive Dijkstra on global graph: ~10 seconds

A* with good heuristic: ~1 second

CH query: ~3ms on global road network

Hub Labeling: ~0.3ms

Key Design Decision 2: Real-Time Traffic Data Collection

Problem: Collect speed data from millions of iPhones without compromising user privacy.

Options

<i>Option</i>	<i>How It Works</i>	<i>Privacy</i>	<i>Accuracy</i>	<i>Cost</i>
Opt-in precise GPS probes	Devices send exact coordinates + speed	Low (precise location)	High	Low (crowdsourced)
Opt-in fuzzy GPS probes (Apple's approach)	GPS rounded to $\pm 100\text{m}$; random noise added; rotating identifiers	High	Good (noise averages out with volume)	Low
Floating Car Data (FCD) from commercial fleet	Trucking/taxi companies sell GPS data	Medium (commercial, not individual)	High (consistent sampling)	High (\$\$\$)
Sensor fusion (cameras, radar on roadside)	Physical infrastructure	High (no phone needed)	High	Very high (infrastructure cost)

■ Recommended: Opt-in fuzzy GPS probes (Apple's approach)

Privacy-Preserving Traffic Probe Design:

On-device before transmission:

1. GPS: round to nearest 100m grid cell (fuzzy location)
2. Speed: quantize to 5 mph buckets
3. Identifier: rotating per-session ephemeral ID (NOT device ID)
 - New ID every trip
 - Apple cannot link trip A to trip B to same user
4. Transmission: via iCloud Private Relay
 - Device IP $\rightarrow$ Private Relay (hides real IP from Apple)
 - Apple sees: fuzzy GPS + bucket speed + ephemeral ID
 - Apple CANNOT identify: which user, which device, what route they took

Server-side aggregation:

Kafka: 16M probe events/sec, partitioned by geographic tile

Flink stream processor:

- Aggregate probes per road segment per 30-second window
- Compute P50 speed (median, robust to outliers)
- Apply differential privacy noise to aggregated speed

Redis: segment_id $\rightarrow$ {p50_speed_mph, sample_count, updated_at}

Result: accurate traffic information with zero individual-trackable data

Key Design Decision 3: Map Tile Serving — Raster vs Vector

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	
Raster tiles	Pre-rendered PNG/JPEG images at each zoom level; server sends pixels	Simple; no client-side rendering code	Fixed style (dark mode = re-download all tiles); large download size; pixelated on zoom	Legacy mapping systems
Vector tiles (Apple's approach)	Server sends geometric data (roads as line segments, polygons); client renders with GPU	Smooth zoom; style changes client-side (no re-download); smaller data size; accessible rendering	Client must have GPU rendering engine (Metal on iOS)	Modern mapping (Apple Maps, Mapbox, Google Maps)
Hybrid	Vector for roads/buildings; raster for satellite imagery	Best of both	Two tile pipelines to maintain	Satellite + street map views

■ Recommended: Vector tiles + on-device Metal rendering

Why vector tiles are superior for Apple Maps:

File size comparison for 1 city block:

Raster PNG (256×256px): ~50KB per zoom level × 20 zoom levels = 1MB

Vector tile (roads + buildings as geometry): ~5KB per zoom level = 100KB total

Style changes:

Dark mode change with raster: re-download all tiles (huge bandwidth)

Dark mode change with vector: just update the style JSON on device (kilobytes)

Client re-renders existing geometry with new colors instantly

Zoom behavior:

Raster: pixelates between zoom levels → ugly

Vector: smooth mathematical interpolation → crisp at any zoom

On-device rendering with Metal:

Metal compute shaders render vector geometry directly on GPU

60fps scrolling/zooming with no server round trips after tiles are cached

Tile cache: LRU eviction, ~200MB on device

Privacy:

User's map viewport never sent to server (all rendering is local)

Only routing requests reveal destination (which they must)

5. Design the App Store Search & Discovery

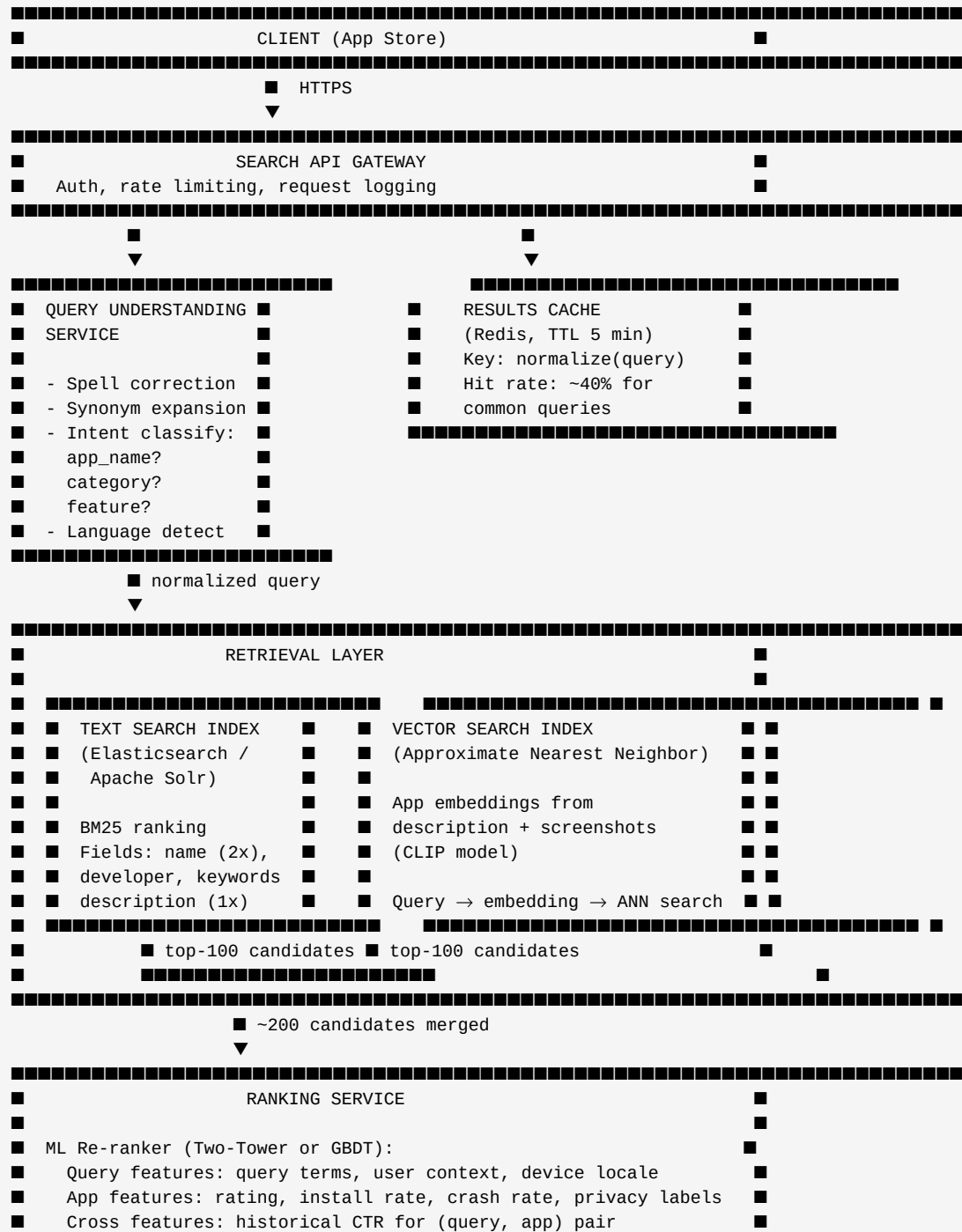
Clarifying Questions

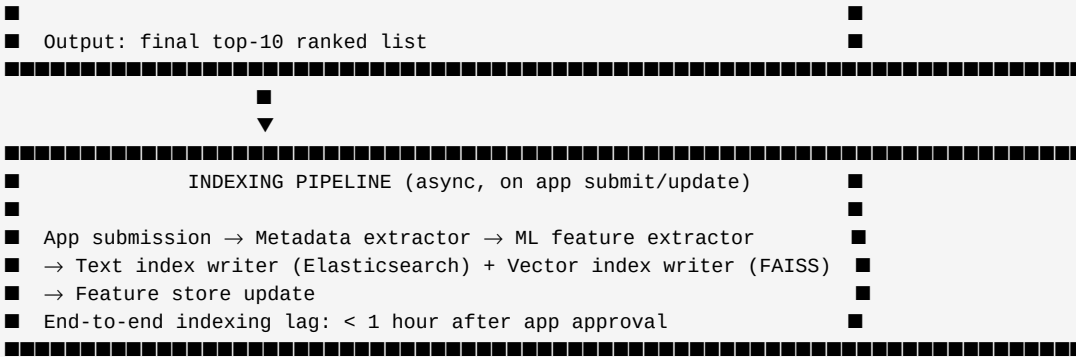
- Search by name, developer, category, keyword?
- 2M apps in scope?
- Personalization using user history?
- Ranking: relevance-only or include quality/commercial signals?
- Near-realtime indexing after app update/submission?

Estimation

```
2M apps; avg metadata 50KB = 100GB total text index (fits in memory across cluster)
500M App Store users
Search QPS: 500M × 5 searches/day ÷ 86,400 = ~29,000 QPS average; peak ~3x = ~87,000 QPS
Indexing: ~1,000 app updates/hour = low throughput
Latency SLO: P99 < 200ms end-to-end
```

Architecture Diagram





Key Design Decision 1: Search Index Technology

Problem: 2M apps; 29K–87K QPS; < 200ms P99; needs relevance ranking + faceted filtering.

Options

Option	How It Works	Pros	Cons	Best When
Elasticsearch	Distributed inverted index; BM25 scoring; horizontal scaling; REST API	Battle-tested at scale; rich query DSL; aggregations for facets; managed (AWS OpenSearch)	JVM GC pauses can affect P99; complex cluster management; not great for vector search	Full-text search, faceted navigation, well-understood text queries
Apache Solr	Similar to Elasticsearch; older; ZooKeeper-based coordination	Mature; good for high-QPS text search	Less ecosystem than ES; harder ops; declining mindshare	Legacy systems using Solr
Custom inverted index (trie-based)	Build in-house for specific requirements; 2M apps fits in memory on one machine	Maximum control; can tune for exact access patterns; zero overhead	Huge engineering investment; reinventing the wheel	Only if Elasticsearch genuinely can't meet requirements
Typesense / Meilisearch	Modern, typo-tolerant search engines written in C++/Rust	Very fast; typo tolerance built-in; simple to operate	Smaller ecosystem; less proven at 87K QPS	Small-to-medium scale, developer tools

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Hybrid: Elasticsearch + Vector DB (FAISS/Pinecone)	BM25 for keyword matching; ANN for semantic similarity; merge candidates	Best recall: catches "photo editor" query for apps tagged "image processing"	Two systems to maintain; latency of both must fit in budget	Semantic search, discovery beyond keyword matching

■ Recommended: Elasticsearch (text) + FAISS (vector) hybrid retrieval

Why Hybrid Retrieval?

BM25 (term frequency-based) alone misses:

Query: "photo editor"

App: "Lightroom" – keywords: "photography, color, Adobe"

BM25 score: near-zero (no "photo editor" in metadata)

But: Lightroom IS the right answer!

ANN Vector Search alone misses:

Query: "netflix" (exact app name)

ANN finds semantically similar apps ("video streaming", "movies") but may rank the exact Netflix app lower than close semantic neighbors

Combined (Reciprocal Rank Fusion):

BM25 finds: Netflix #1, Hulu #2, Disney+ #3 (exact keyword match wins)

ANN finds: Netflix #2, Hulu #1, Amazon Prime #3 (semantic match)

RRF merge: $\text{score} = \sum 1/(k + \text{rank}_i)$ per result

Final: Netflix #1 (ranked high in both), correct!

FAISS (Facebook AI Similarity Search) – What it is:

Library for efficient ANN (Approximate Nearest Neighbor) search

Stores app description embeddings as high-dimensional vectors (e.g., 512-dim)

Query: convert search query to embedding → find nearest 100 vectors

IVF (Inverted File Index) partitions vectors into clusters:

Build time: k-means cluster vectors into 1000 clusters

Query: probe nearest 20 clusters instead of all vectors

Result: 50x speedup vs brute-force; ~95% recall

HNSW (Hierarchical Navigable Small World) – alternative:

Graph-based; higher recall than IVF; slightly more memory

Preferred for < 100M vectors (2M apps: trivially small for HNSW)

Key Design Decision 2: Ranking Model

Problem: From 200 retrieved candidates, how do you order the final top-10?

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
BM25 score only	Pure relevance ranking based on term frequency	Simple; fast; no ML	Doesn't incorporate quality signals (ratings, install rate)	Baseline; very small scale
Weighted linear combination	score = w1×relevance + w2×rating + w3×install_rate	Simple to understand; easy to tune	Weights tuned manually; doesn't capture interaction effects	Quick iteration; explainability requirement
GBDT (Gradient Boosted Decision Trees)	Learn tree model from (query, app, label) training data; XGBoost/LightGBM	Strong baseline; fast inference; handles sparse features well; interpretable feature importance	Requires labeled training data; no sequential/contextual modeling	Ranking with good training data; classic industry choice
Two-Tower Neural Model	Query tower + app tower → embedding → dot product score; train on click/install data	Learns deep semantic features; enables ANN retrieval at scale; handles new apps via embedding	Black box; requires substantial training data; slower inference	Large-scale semantic ranking, personalization
Learning to Rank (LambdaMART)	Directly optimizes ranking metrics (NDCG, MRR) rather than classification	Directly optimizes what you care about	Complex training; harder to debug	When you have good relevance labels

■ **Recommended: GBDT re-ranker (LightGBM) with two-tower for semantic retrieval**

Two-Stage Ranking Architecture:

Stage 1 – Retrieval (two-tower model, runs during indexing):

App tower: app_description + app_name + screenshots → 512-dim embedding

Stored in FAISS index

At query time: query → 512-dim embedding → ANN search → top-100 apps

Stage 2 – Re-ranking (GBDT, runs at query time in < 10ms):

Features for each of the 100 candidates:

- BM25 relevance score
- Cosine similarity (from two-tower)
- App rating (1-5 stars)
- Install rate for this query-app pair (historical click/install data)
- Crash rate (low crash rate = boost)
- Privacy nutrition label score (fewer permissions = boost at Apple)
- App recency (recently updated = small boost)
- Sponsored indicator (top 2 slots can be sponsored)
- User's install history match (on-device signal, differential privacy)

Training data: (query, app, installed=1/skipped=0) pairs from anonymized history

Output: ranked list, top-10 surfaced to user

6. Design a Real-Time Health Data Platform (HealthKit Backend)

Clarifying Questions

- Data types: heart rate, steps, sleep, ECG, blood oxygen, nutrition?
- Write-heavy (continuous Watch sensor data)?
- Who reads: user only, or care team with consent?
- HIPAA compliance?
- Data retention: lifetime?

Estimation

50M Apple Watch users

Heart rate: 1 sample/min × 50M users = 50M writes/min = 833K writes/sec

Steps: 1 sample/min = 833K writes/sec

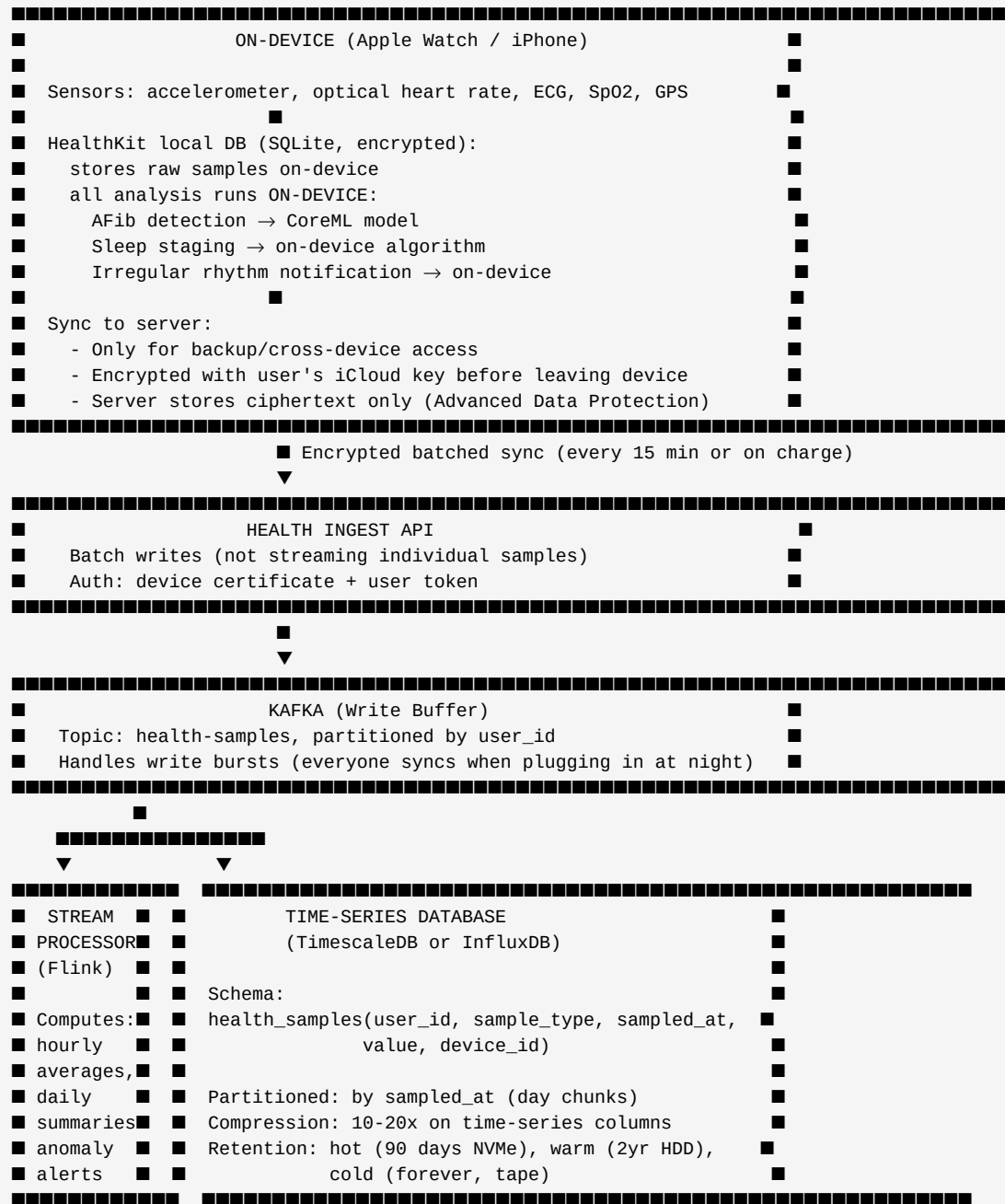
SpO2, sleep, ECG: lower frequency

Total: ~2M writes/sec across all sensor types

Read QPS: 100K/sec (users checking Health app, sharing with doctor)

Storage: ~50TB/day raw; with 10x compression: ~5TB/day retained

Architecture Diagram



Key Design Decision 1: Time-Series Database Choice

Problem: 2M writes/sec, monotonically increasing timestamps, range queries by time window.

Options

<i>Option</i>	<i>How It Works</i>	<i>Writes/sec</i>	<i>Query Pattern</i>	<i>Pros</i>	<i>Cons</i>
TimescaleDB	PostgreSQL extension; hypertable partitioned by time; column compression	~500K/sec per node (scalable)	Full SQL with time functions	SQL familiarity; continuous aggregates; mature; PostgreSQL ecosystem	PostgreSQL overhead for simple time-series; complex to scale beyond single node
InfluxDB	Purpose-built TSS; line protocol; tag-based indexing; Flux query language	~1M/sec per node	Time-range + tag filter	Excellent compression; InfluxQL intuitive; designed for this exact workload	Non-SQL query language; InfluxDB 3.0 breaking changes; less ecosystem
Apache Cassandra (time-series pattern)	Wide-column; `(user_id, bucket)` partition, `sampled_at` clustering	~1M+/sec distributed	Range query per user per time bucket	Extreme write scalability; proven at Apple scale; tunable consistency	More complex schema design; no native time-series functions; aggregations need application logic
Apache Druid	Columnar OLAP; pre-aggregated roll-ups; real-time ingestion via Kafka	Very high (OLAP)	Aggregate analytics	Sub-second analytics on billions of rows; roll-up for time bucketing	Not great for individual sample queries; approximate results with roll-ups
BigQuery / Redshift	Managed columnar data warehouse; batch loading	Low (batch)	Analytics	Serverless scaling; excellent for analytics	High latency (seconds); not suitable for real-time or individual sample lookups

■ **Recommended: TimescaleDB for user-facing queries + Apache Druid for analytics**

Why TimescaleDB:

TimescaleDB hypertable chunks by time:

```
CREATE TABLE health_samples (  
  user_id      UUID NOT NULL,  
  sample_type  SMALLINT NOT NULL,    -- 0=heart_rate, 1=steps, 2=spo2  
  sampled_at   TIMESTAMPTZ NOT NULL,  
  value        DOUBLE PRECISION,  
  device_id    UUID  
);  
SELECT create_hypertable('health_samples', 'sampled_at',  
                        partitioning_column => 'user_id',  
                        number_partitions => 16);  
  
-- Each chunk = 1 day × 1 user_id range  
-- Queries like "heart rate for user X from 2pm-3pm" hit exactly 1 chunk  
-- Old chunks compressed automatically: 10-20x ratio  
  
-- Continuous aggregate (pre-computed):  
CREATE MATERIALIZED VIEW hourly_hr WITH (timescaledb.continuous) AS  
  SELECT user_id, time_bucket('1 hour', sampled_at) AS hour,  
         avg(value), min(value), max(value)  
  FROM health_samples WHERE sample_type = 0 -- heart rate  
  GROUP BY user_id, hour;
```

What is a Continuous Aggregate?

Instead of computing `avg(heart_rate)` on every query (scanning all rows), TimescaleDB maintains a pre-computed aggregate that is incrementally updated as new data arrives. Query reads the materialized view ($O(1)$) instead of raw rows ($O(n)$).

Trade-off: slightly stale (refreshed every 1 hour) vs. always-fresh raw query.
For health analytics: hourly granularity is fine. For real-time monitoring: query raw.

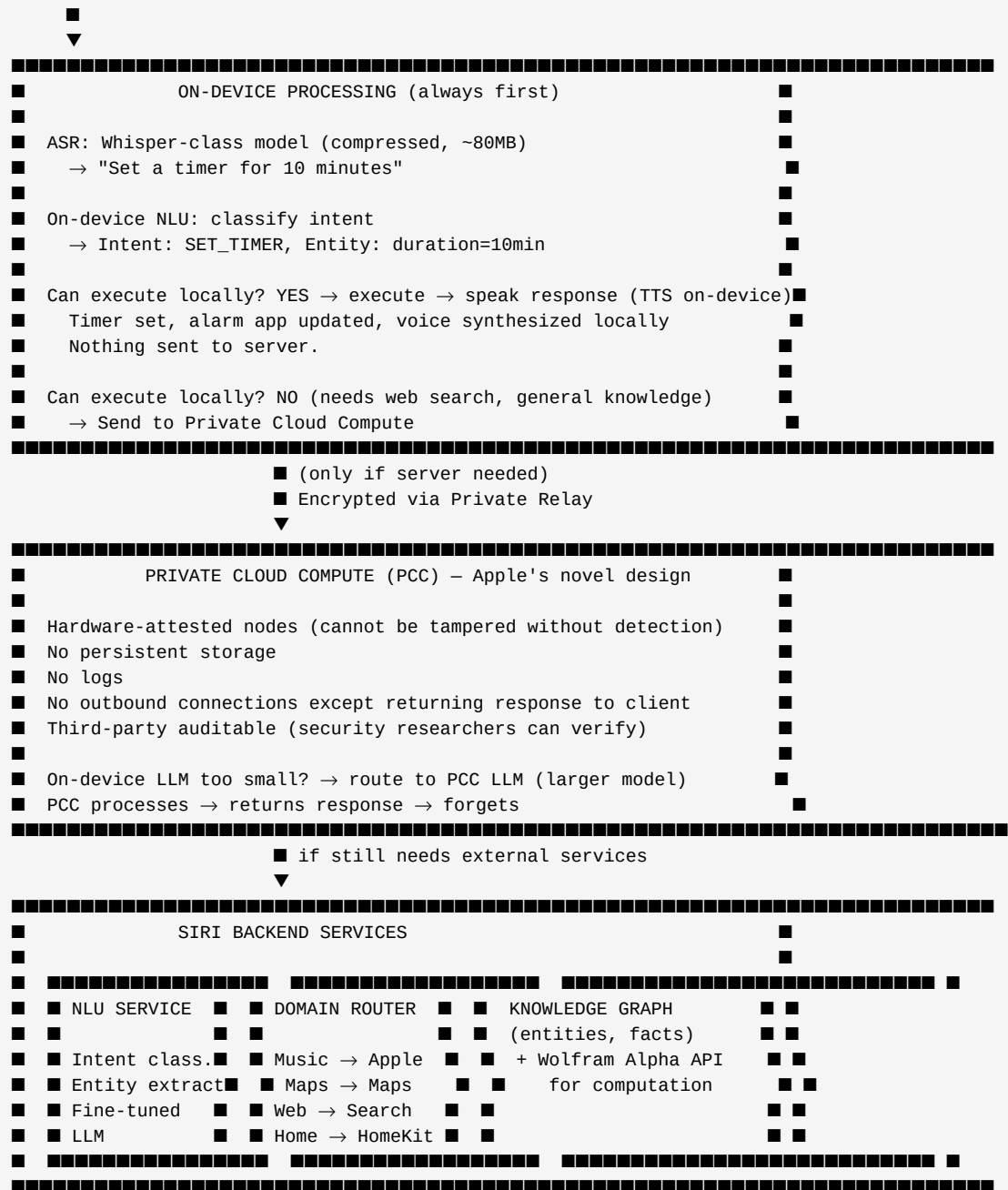
7. Design Siri — Voice Assistant Backend

Clarifying Questions

- Full pipeline (ASR → NLU → execution → TTS) or one component?
- On-device vs cloud processing boundary?
- Multi-turn conversation scope?
- Which domains: reminders, music, device control, general knowledge?

Architecture Diagram

USER SPEAKS



Key Design Decision: On-Device vs Cloud LLM Boundary

Options

<i>Option</i>	<i>Latency</i>	<i>Privacy</i>	<i>Capability</i>	<i>Battery</i>
Always on-device	Fast (< 1s)	Perfect (data never leaves)	Limited (3B param model)	Good (optimized model)
Always cloud LLM	Higher (network + inference)	Low (query sent to server)	Unlimited	Battery saved (offload compute)
On-device first, cloud fallback	Fast for simple; higher for complex	High (only complex queries leave)	Balanced	Good
PCC (Apple's approach)	Medium (network + attested inference)	Very high (hardware-verified no logging)	High (larger model than on-device)	Medium

■ Recommended: Tiered approach with PCC

Decision tree (simplified):

Query: "set a 10 minute timer"

→ On-device NLU: intent=TIMER → execute locally → done. 0 network requests.

Query: "what's the weather in Tokyo?"

→ On-device: can't answer (no live data)

→ Route to: Weather service API (no sensitive personal data needed)

→ Privacy: only "Tokyo" and approximate time leave device

Query: "summarize my emails from this week"

→ On-device Apple Intelligence (small LLM): try first

→ If on-device model insufficient: route to PCC

→ PCC: receives email content (encrypted), processes, returns summary, deletes

→ Key: PCC hardware attestation means Apple can PROVE to auditors that PCC cannot retain data, even if government demands it

Query: "plan a trip to Paris based on my calendar and budget"

→ PCC or cloud LLM with appropriate context scoping

What is Private Cloud Compute (PCC)?

Normal cloud: Apple's server processes your request → request logged →

Apple employees can access logs → government can subpoena logs

PCC: request processed on hardware with:

- Sealed firmware (signed by Apple, any tampering breaks attestation)
- No persistent disk writes for user data
- No remote shell access (not even for Apple engineers)
- Auditable: Apple publishes PCC software images; security researchers can verify the running code matches what's published
- Client verifies attestation before sending: device checks PCC node's TPM (Trusted Platform Module) certificate before trusting it

Result: Apple can CRYPTOGRAPHICALLY PROVE that it cannot retain query data, even if legally compelled. This is a new privacy architecture category.

Limitation: PCC adds ~200ms network RTT vs on-device 0ms

8. Design a Privacy-Preserving Analytics System

The Problem

Apple needs fleet-wide behavioral analytics (which features used, error rates, battery impact) **without collecting individual user data**.

Key Concept Deep Dive: Differential Privacy

INTUITION:

A statistical algorithm is ϵ -differentially private if:

"The output is nearly the same whether or not any single person's data is included"

Formal definition:

$\Pr[M(D) \in S] \leq e^\epsilon \times \Pr[M(D') \in S]$

Where D and D' differ by one person's data

ϵ (epsilon) = privacy budget (lower = stronger privacy, more noise)

Apple uses $\epsilon \approx 1-4$ in practice

MECHANISM – Laplace Noise:

True count: 1,000,000 users used feature X

Sensitivity: 1 (one user can change count by at most 1)

Noise drawn from: $\text{Laplace}(0, \text{sensitivity}/\epsilon) = \text{Laplace}(0, 1/2)$ for $\epsilon=2$

Noisy count sent to server: $1,000,000 + \text{noise} \approx 1,000,003$

Across millions of users, individual noise cancels out by central limit theorem.

Server sees: accurate population-level statistic.

But: cannot determine whether any specific user used the feature.

RANDOMIZED RESPONSE (simpler form used by Apple for binary attributes):

Question: "Do you use Feature X ?" (private answer)

Algorithm on device:

Flip coin:

Heads (prob 0.5): answer truthfully

Tails (prob 0.5): flip another coin

Second heads: answer "yes"

Second tails: answer "no"

Server receives "yes": user used feature WITH PLAUSIBLE DENIABILITY

(50% chance it was random noise, not truth)

Server aggregates millions of responses:

True frequency f estimated as: $f = (p_{\text{observed}} - 0.25) / 0.5$

Where 0.25 = noise contribution and 0.5 = signal coefficient

Privacy: individual response is $\epsilon = \ln(3) \approx 1.1$ differentially private

Accuracy: with 1M responses, error is $\pm 0.1\%$ – very accurate aggregate!

PRIVACY BUDGET:

Each query against a user's data consumes budget.

Budget is finite and tracked per user.

When budget exhausted: no more queries on this user's data.

Prevents: adversary asking N different questions to reconstruct individual data

Implementation: local counter on device; reset annually

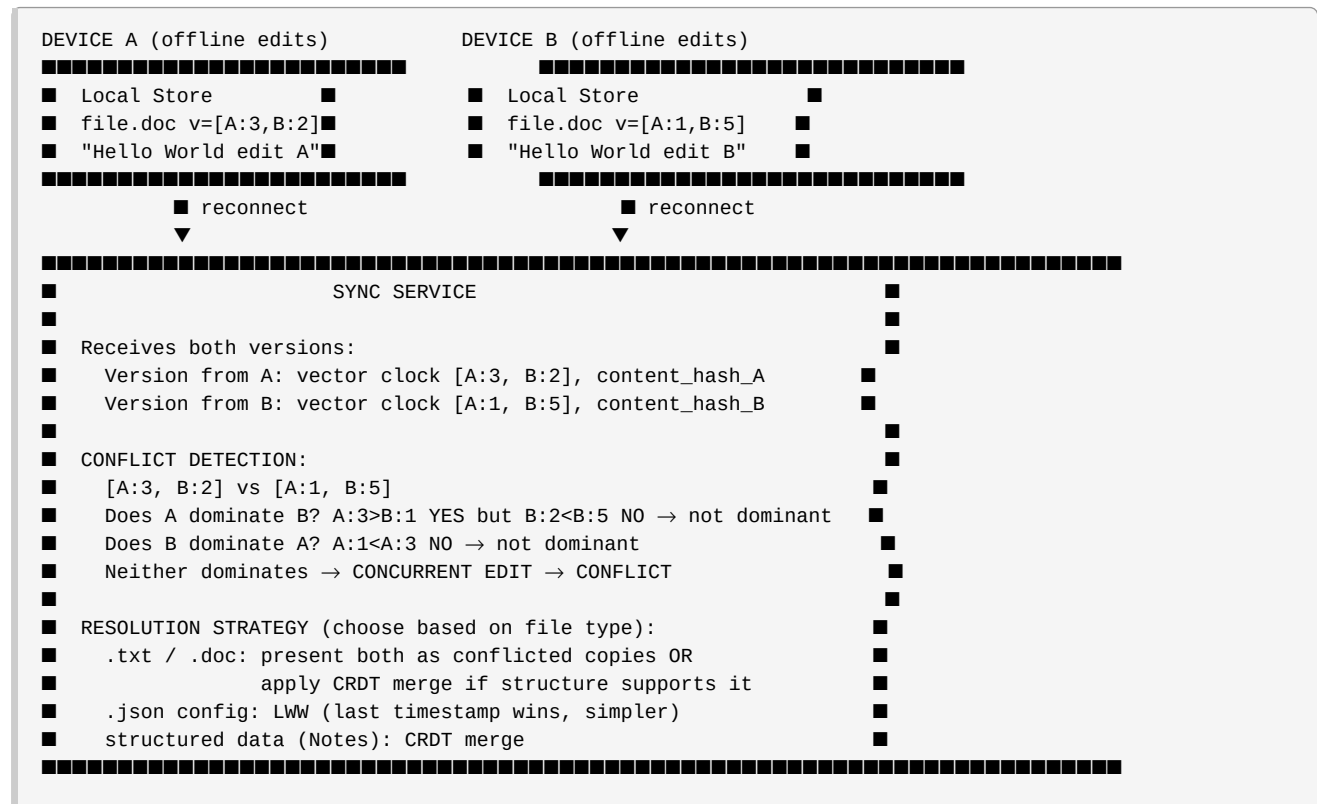
Architecture Diagram

Option	Privacy	Accuracy	Latency	Ops Complexity
Federated analytics (FL-style)	High (data stays on device; only model updates shared)	Depends on participation	Days-to-weeks	Very complex

■ **Recommended: Local differential privacy for behavioral metrics, federated learning for ML model training**

9. Design an Offline-First Sync System (iCloud Drive)

Architecture Diagram



Key Design Decision: Conflict Resolution Strategy

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best For</i>
Last Write Wins (LWW)	Keep version with highest wall-clock timestamp	Simple; zero user friction	Silent data loss: one user's edit disappears	Non-critical config, preferences, single-field updates
First Write Wins	Keep first version received; reject subsequent concurrent writes	Prevents overwrite	Writer frustration: "my edit was ignored"	Rarely appropriate
Conflicted copies (Dropbox model)	Keep both versions as separate files (file.doc + file (Alice's conflicted copy).doc)	No data loss	User must manually merge; confusing for non-technical users	Files where merging is complex (binary, spreadsheets)
CRDT merge	Mathematically guaranteed merge (see CRDTs section)	Automatic; no user friction; correct	Limited to CRDT-compatible data structures	Collaborative text, counters, sets, ordered lists
Operational Transform (OT)	Transform operations to account for concurrent edits (Google Docs)	Works for arbitrary text editing	Very complex algorithm; requires total ordering of operations	Real-time collaborative text editing
3-way merge (git-style)	Find common ancestor; merge A's changes and B's changes from base	Works for text; git-proven	Requires knowing common ancestor; merge conflicts still possible	Code, structured text, files with clear line structure

■ **Recommended: Strategy depends on file type**

File Type Decision Matrix:

Plain text / Notes:

- CRDT merge (RGA or LSEQ sequence CRDT)
- Why: no user friction; converges automatically
- Apple Notes uses a CRDT-like approach internally

Binary files (images, PDFs, Office docs):

- Conflicted copies (no auto-merge possible)
- Surface to user: "Two versions exist. Which do you want to keep?"

Structured data (Contacts, Calendars, Reminders):

- Field-level LWW with CRDT for multi-value fields
- Contact.name: LWW (only one name makes sense)
- Contact.phone_numbers: OR-Set CRDT (multiple phones, can add/delete)
- Calendar.event.start_time: LWW
- Calendar.attendees: OR-Set CRDT

Configuration files / Preferences:

- LWW (user expectation: latest change wins)

iCloud Drive documents (Keynote, Pages, Numbers):

- App-specific CRDT built into the app format
- Apple's Collaboration features use operational intent vectors

Delta Sync (bandwidth optimization):

- Don't upload full file on every change:

File changes: compute binary diff (bsdiff algorithm)
Upload: only the delta (typically 1-5% of file size)
Server applies delta to stored version

Example: editing a 10MB Pages document

- Old approach: upload 10MB on every save
- Delta sync: upload ~50KB delta = 200x bandwidth reduction

Delta sync algorithm (bsdiff):

- old_file XOR new_file → diff blocks → Bzip2 compress diff
- Server: old_file + diff → new_file
- Client: new_file → display

10. Design a Distributed Rate Limiter

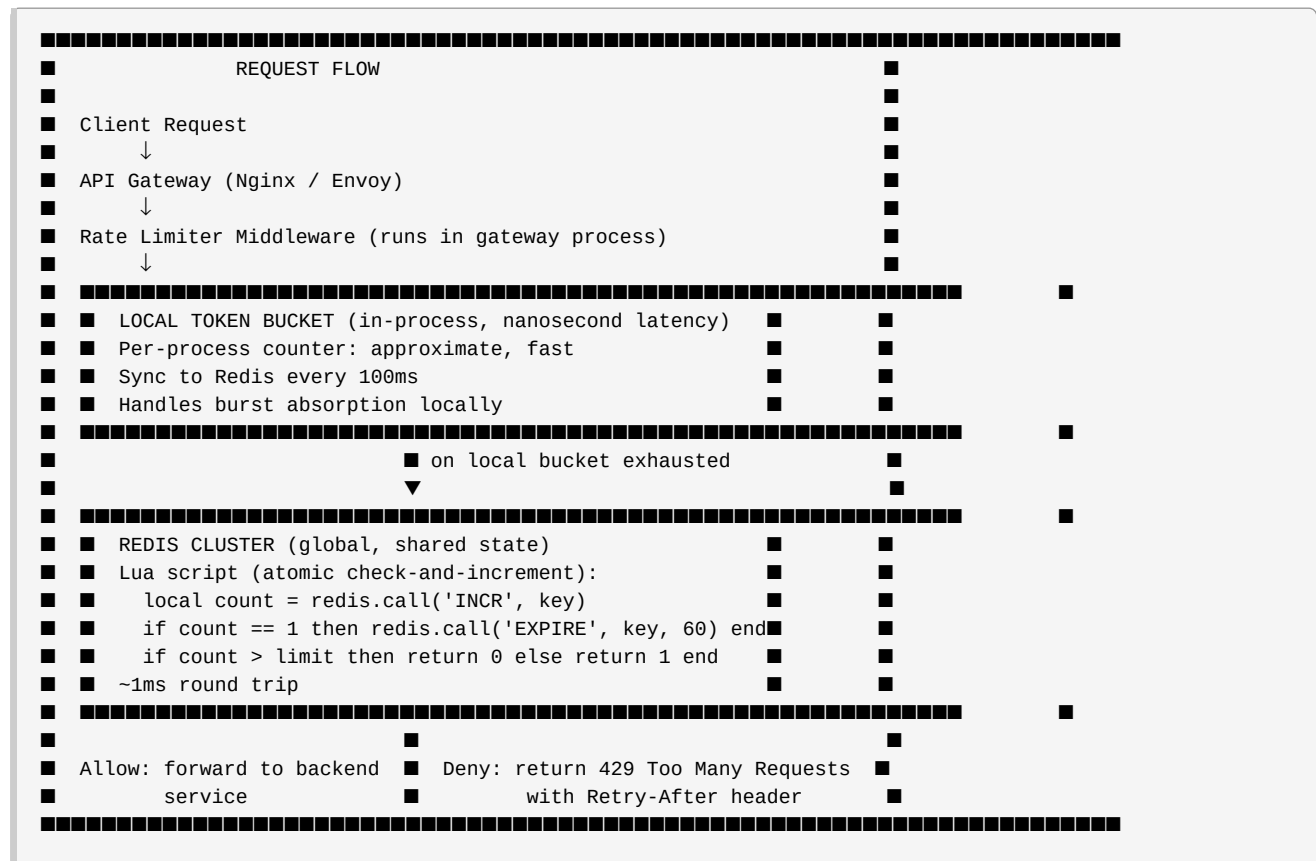
Clarifying Questions

- Per user? Per IP? Per API key? Per app?
- Hard limit (reject) or soft limit (queue)?
- Global rate limit or per-datacenter?
- What is acceptable overshoot? (Can limit be exceeded by 1-2 requests?)
- Latency budget for rate limit check?

Estimation

100M API requests/sec (Apple-scale API gateway)
Rate limit check must complete in < 1ms (adds to request latency)
100M users → 100M rate limit counters
Each counter: ~16 bytes (key + count + TTL) → $100M \times 16B = 1.6GB$ (fits in Redis)

Architecture Diagram



Key Design Decision: Rate Limiting Algorithm

Problem: Which algorithm gives the right semantics for our use case?

Options

<i>Algorithm</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best For</i>
Fixed Window Counter	Count requests in 60s buckets (00:00–01:00, 01:00–02:00)	Simple; O(1) space	Boundary burst: user sends 100 at 00:59 + 100 at 01:00 = 200 in 2 seconds (double the limit)	Low-stakes rate limiting where boundary bursts are acceptable
Sliding Window Log	Store timestamp of every request in sorted set; on new request: remove old entries, count remaining	Exact; no boundary burst	O(n) memory where n = requests per window; impractical for high-QPS	Low-volume precise rate limiting
Sliding Window Counter (hybrid)	Two buckets: current + previous minute; interpolate: count = prev×(1-elapsed/60) + curr	O(1) space; approximates sliding window well; ~1% error	Approximate (not exact); slight complexity	Best production choice for most use cases
Token Bucket	Bucket with capacity C; refills at rate R tokens/sec; each request consumes 1 token	Handles burst up to C; smooth average rate; natural semantics	Requires tracking token count + last refill time; slightly more complex	APIs where burst is explicitly allowed (e.g., 10 req/s avg but 50 burst)
Leaky Bucket	Requests enter a queue; processed at fixed rate; excess overflows (rejected)	Smooth output rate regardless of input burst	Queue adds latency; doesn't allow legitimate bursts; rarely appropriate for API rate limiting	Traffic shaping, not rate limiting

■ **Recommended: Token Bucket for per-user limits, Sliding Window Counter for per-endpoint limits**

TOKEN BUCKET (deep dive):

State per user:

```
{tokens: 8.5, last_refill: 1716000000.500}
```

On new request:

```
now = current_timestamp()
elapsed = now - last_refill
new_tokens = min(capacity, tokens + elapsed * refill_rate)
```

```
if new_tokens >= 1:
    tokens = new_tokens - 1
    last_refill = now
    ALLOW
else:
    DENY (429)
```

Example: capacity=10, rate=5 tokens/sec

t=0: user sends burst of 10 requests → tokens: 10→0, all allowed (burst!)

t=1: 5 tokens refilled → allows 5 more requests

t=2: 5 tokens → 5 more

Average: 5 req/sec (rate limit), burst up to 10 (capacity)

Redis implementation (atomic Lua script):

```
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local rate = tonumber(ARGV[2]) -- tokens per second
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4]) -- usually 1

local data = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(data[1]) or capacity
local last_refill = tonumber(data[2]) or now

local elapsed = now - last_refill
tokens = math.min(capacity, tokens + elapsed * rate)

if tokens >= requested then
    tokens = tokens - requested
    redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
    redis.call('EXPIRE', key, math.ceil(capacity / rate) + 1)
    return 1 -- allow
else
    return 0 -- deny
end
```

WHY LUA SCRIPT?

Redis executes Lua atomically (single-threaded).

Without Lua: HGET tokens → compute → HSET (two round trips, race condition)

With Lua: entire check-and-update in one atomic operation → no race condition

DISTRIBUTED CHALLENGE:

100 API gateway servers each talk to Redis.

Each Redis INCR is atomic → no double-counting.

Optimization: each server keeps a local token counter (in-process).

Sync local counter to Redis every 100ms.

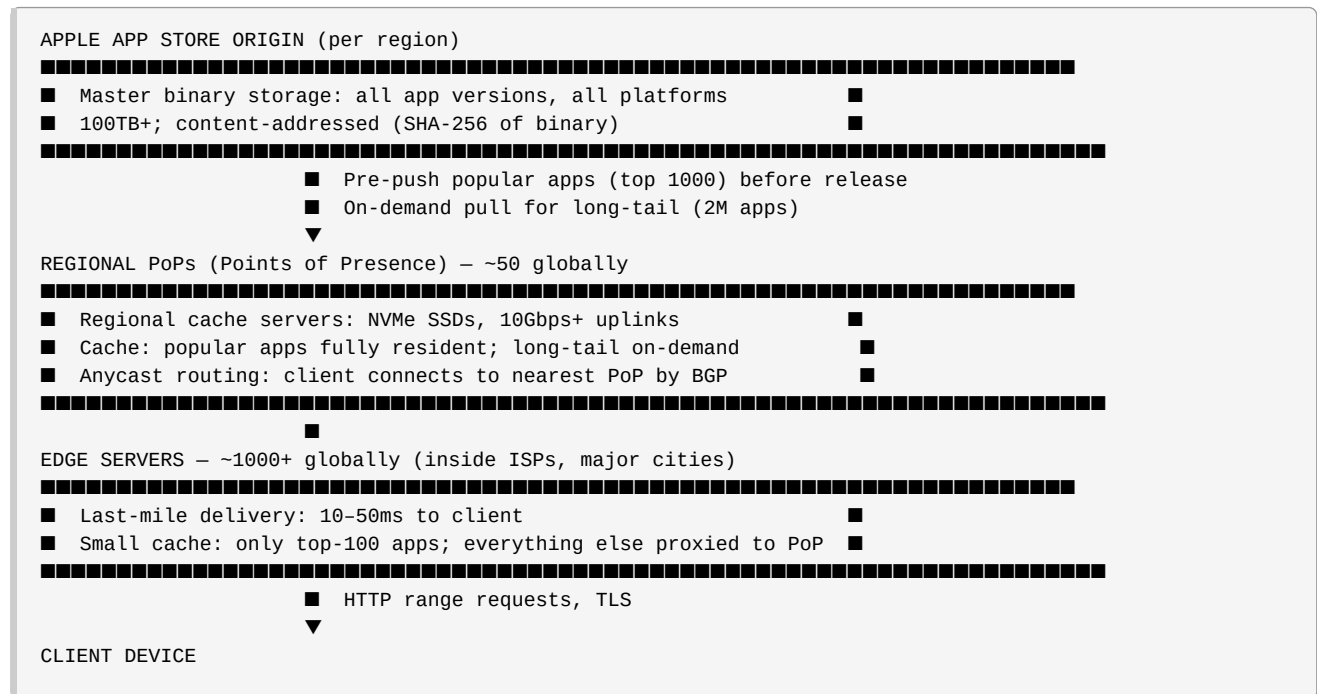
Cost: users can exceed limit by (100ms × rate) before detection.

For most APIs: this overshoot is acceptable.

For financial APIs: sync on every request (higher latency, exact).

11. Design a Global CDN for App Binary Delivery

Architecture Diagram



Key Design Decision: How to Handle Large App Updates

Options

Option	Approach	Download Size	Cons
Full binary every update	Client downloads complete app binary on each update	100MB+ for large apps	Wastes bandwidth; slow for user
Delta updates (binary diff)	Server computes diff between v1.0 and v1.1 binary; client downloads only diff	5–20MB typical (5–20% of full size)	Client must have v1.0 installed; delta computation on server is CPU-intensive

<i>Option</i>	<i>Approach</i>	<i>Download Size</i>	<i>Cons</i>
Asset-only updates	App code unchanged; only updated assets (images, strings) delivered	Very small (KB-MB)	Only works when only assets changed; can't update code
Streaming install (App Slicing)	Deliver only device-appropriate assets (no iPad assets on iPhone)	Reduces by 30–60% for universal apps	Requires thinning at origin; more cache variants
On-Demand Resources (ODR)	Core app small; additional content downloaded as user needs it	Small initial install	Network required for content; latency when content needed

■ Recommended: Delta updates + App Slicing + ODR (Apple's actual approach)

Delta Update Flow:

Server (offline, runs when new version is submitted):

```
bsdiff(app_v1.0.ipa, app_v1.1.ipa) → delta_1.0_to_1.1.bspatch
```

```
# For each version pair (v_n-1 → v_n):
```

```
# Store delta on CDN alongside full binary
```

Client update flow:

1. App Store notifies: update available for App X
2. Client sends: current_version=1.0
3. Server checks: delta_1.0_to_1.1 exists? YES
4. Client downloads delta (e.g., 15MB instead of 100MB)
5. Client applies: bspatch(app_v1.0.ipa, delta) → app_v1.1.ipa
6. Verify: SHA-256(result) == expected hash from App Store
7. If mismatch: fall back to full binary download

Security:

Both delta and full binary are codesigned by Apple.
 bspatch result is verified against Apple-signed hash before installation.
 Even if CDN edge is compromised: tampered binary won't install.

Codesigning deep dive:

Every iOS app has a signature chain:

```
Apple Root CA → Apple Developer CA → Developer Certificate → App Binary
```

On install: device verifies signature chain back to Apple Root CA
 → Binary tampering (even 1 bit change): signature mismatch → install rejected
 → No way to distribute malware via CDN compromise
 → This is why iOS has never had a supply-chain binary attack at CDN level

<i>Policy</i>	<i>Algorithm</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
ARC (Adaptive Replacement Cache)	Self-tuning: balances LRU and LFU dynamically	Adapts to workload without manual tuning	More complex; slightly higher overhead	Mixed workloads where optimal policy shifts
TinyLFU (Redis 4.0+ allkeys-lfu)	Approximate LFU using 4-bit counters + aging	Very space-efficient; handles frequency well	Still not scan-resistant	High-hit-rate caches with stable key access patterns
Random	Evict any key at random	Simplest; sometimes surprisingly effective	Non-deterministic; unpredictable	Rarely appropriate for production

■ Recommended: LRU for general cache, LFU for session/social data

```

LRU Implementation (O(1) get and put):
  Data structures: doubly-linked list + hash map

  Hash map: key → node_pointer (O(1) lookup)
  Doubly-linked list: MRU at head, LRU at tail (O(1) insert/remove with pointer)

  get(key):
    node = hash_map.get(key)      # O(1)
    if not node: return MISS
    move node to HEAD of list     # O(1) with doubly-linked
    return node.value

  put(key, value):
    if key in hash_map:
      update node.value
      move to HEAD                # O(1)
    else:
      if cache_full:
        evict = list.TAIL        # O(1)
        hash_map.remove(evict.key)
        list.remove(evict)       # O(1)
      new_node = Node(key, value)
      list.insert_head(new_node)  # O(1)
      hash_map.put(key, new_node) # O(1)

  Redis approximation:
    Redis doesn't maintain a perfect LRU list (too memory expensive for millions of keys).
    Instead: when eviction needed, sample 5 random keys and evict the least recently used.
    Result: approximates LRU with O(1) overhead vs exact LRU's O(n) memory

```

Key Design Decision: Cache Invalidation

Options

Strategy	How It Works	Cons	Best When
TTL expiry	Key expires after N seconds automatically	Stale data during TTL window; TTL must be tuned	Read-heavy; tolerate eventual consistency
Write-through	Write simultaneously to cache and DB	Higher write latency; cache and DB always in sync	Write-heavy with immediate read consistency needed
Write-behind (write-back)	Write to cache; async flush to DB	Risk of data loss if cache crashes before flush	Ultra-low write latency needed; acceptable data loss risk
Cache-aside (lazy loading)	App reads cache → miss → read DB → populate cache	Cache miss adds latency; stale data if DB updated without cache invalidation	Most common pattern; simple to implement
Event-driven invalidation	DB change → event → invalidate/update cache entry	Consistent; no TTL staleness	When DB has CDC capability (Debezium + Kafka)

■ **Recommended: Cache-aside for reads, event-driven invalidation for write consistency**

13. Design Search Autocomplete (Spotlight / App Store)

Architecture Diagram

<i>Option</i>	<i>How It Works</i>	<i>Latency</i>	<i>Memory</i>	<i>Pros</i>	<i>Cons</i>
Elasticsearch prefix query	ES handles prefix search natively; prefix query type	5–50ms	Moderate	Full-text capabilities; fuzzy matching	Too slow for < 50ms autocomplete if not cached
Distributed trie (sharded by prefix)	Shard trie by first N characters; each shard in memory	2–5ms (one network hop)	Distributed	Scalable to any vocabulary; consistent	Network hop adds latency; shard coordination
Precomputed prefix table (hybrid)	Offline: for every prefix, pre-compute top-K; store in Redis/Cassandra	1–2ms	Moderate	Predictable; easy to update	Storage explodes for long prefixes; not good for personalization

■ **Recommended: Distributed in-memory trie with top-K stored at each node + Redis prefix cache**

Trie Node Structure:

```
TrieNode {
  char: Char
  children: HashMap<Char, TrieNode>    // 26 or Unicode
  is_word: bool
  frequency: Long                     // global search frequency
  top_k: List<(String, Long)>          // pre-computed top-5 completions
  // top_k avoids full subtree traversal at query time
}
```

Updating top-K:

Naive: recalculate entire trie subtree on every new search
Better: batch update every 1 hour from aggregated search logs

1. Kafka stream: all search queries → aggregate frequency per query
2. Top-K offline job: for each prefix, compute new top-K
3. Atomic trie update: swap old trie pointer to new trie (zero downtime)

Personalization layer:

Global trie returns top-K for all users.
Re-rank on device using:

- User's recent searches (local history)
- User's installed apps (favor apps they have)
- User's locale (US vs Japan: different popular terms)

Privacy: personalization signals never sent to server.
Server returns global top-20; device re-ranks to top-5.

FUZZY MATCHING (typo tolerance):

Query "appel" (typo for "apple")

Option 1: Precompute trigrams

"appel" → trigrams: ["app", "ppe", "pel"]

Search for: all entries containing any trigram

Rank by: Jaccard similarity of trigram sets

Option 2: BK-Tree (Burkhard-Keller Tree)

Tree indexed by edit distance

Query: find all strings within edit distance 1 of "appel"

Very fast for small edit distances

Option 3: Symspell (fastest)

Pre-generate all variants within edit distance 1 (delete 1 char)

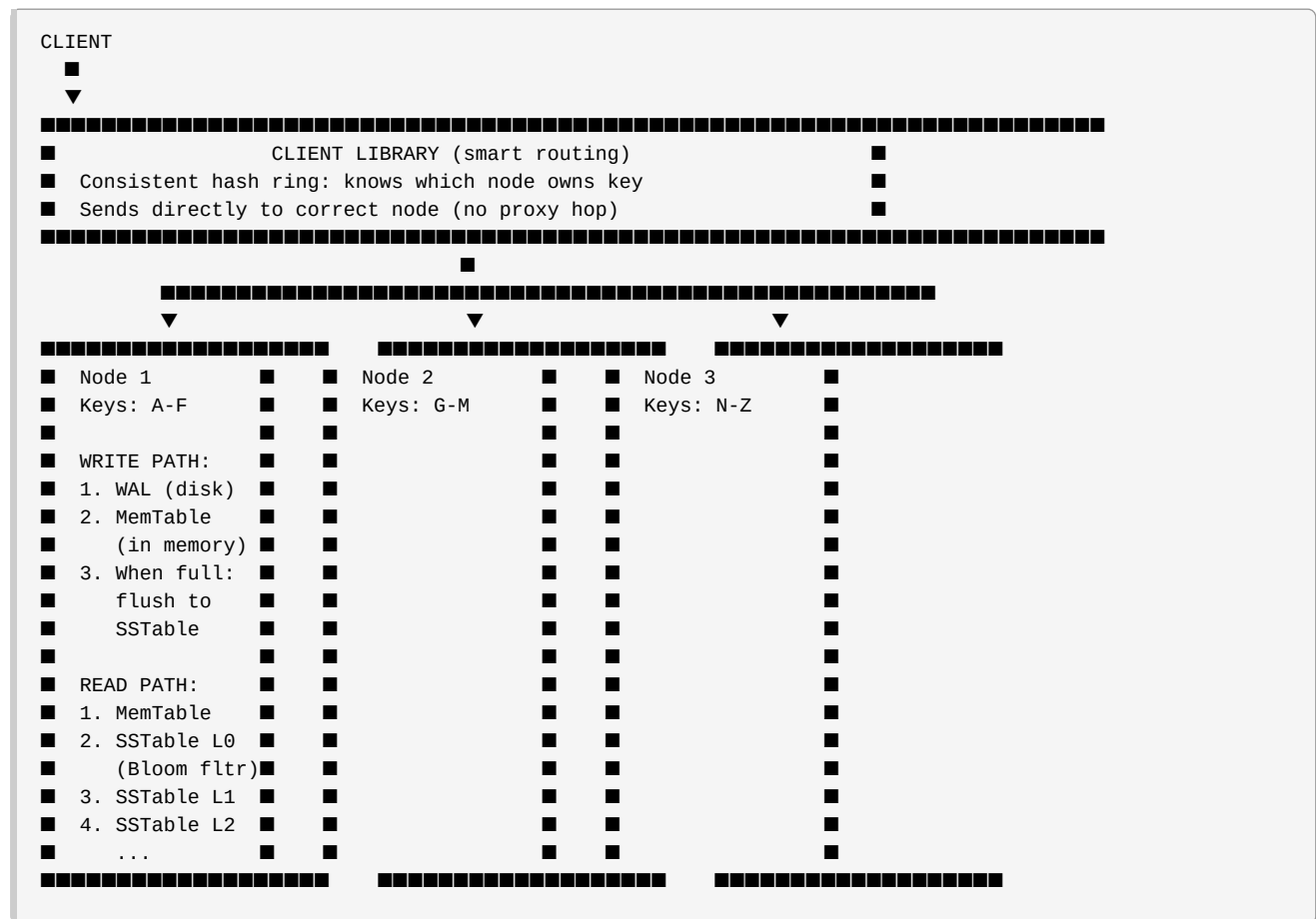
Store in hash map

Query: generate deletes of query → check hash map

O(1) lookup for edit distance 1

14. Design a Key-Value Store

Architecture Diagram



Key Design Decision: Replication Model

Options

Option	Writes to	Read from	Consistency	Availability	Latency
Single-leader	Leader only; async replicate to followers	Leader (strong) or follower (eventual)	Strong if read from leader	Leader SPOF; writes down if leader fails	Low (no coordination)
Multi-leader	Any leader; leaders sync	Any leader	Eventual (conflicts possible)	High (any DC accepts writes)	Very low (write to local DC)
Leaderless (D ynamo/Cassa ndra)	W of N nodes (quorum)	R of N nodes (quorum)	Eventual (W+R>N for strong)	Very high (any N-W nodes can fail)	Configurable

Option	Writes to	Read from	Consistency	Availability	Latency
Consensus (Raft/Paxos)	Commit when majority of nodes ACK	Leader only	Strong (linearizable)	Minority partition available; majority needed for writes	Higher (round trips for consensus)

■ Recommended: Leaderless with tunable quorum (Dynamo-style) for high-availability KV store

QUORUM READS/Writes DEEP DIVE:

N = total replicas per key (e.g., 3)
W = writes must ACK before success (e.g., 2)
R = reads must return before response (e.g., 2)

Rule: $W + R > N$ guarantees reading at least one node with latest write

Example (N=3, W=2, R=2):

Write: send to all 3, wait for 2 ACKs → return success

Read: send to all 3, wait for 2 responses → return latest (by timestamp/vector clock)

Even if Node 3 is slow/down:

Write: Node1 ACK + Node2 ACK = success (Node3 updates async)

Read: Node1 response + Node2 response = return latest (have quorum)

Tuning for different requirements:

W=3, R=1: write to all, read from any → strong consistency but slow writes

W=1, R=3: write fast, read from all → fast writes but slow reads

W=2, R=2: balanced (most common)

W=1, R=1: max performance, no consistency guarantee (fire-and-forget)

What is Hinted Handoff?

Node 2 is down during write. Quorum achieved with Node1 + Node3.

Node 1 stores "hint": "I have a pending write for Node2, key=X, value=Y"

When Node 2 recovers: Node 1 sends the hinted write to Node 2.

Node 2 now has up-to-date data.

Without hinted handoff: Node 2 recovers with stale data; next read might return stale value even though quorum seems satisfied.

What is Read Repair?

During a read (R=2): Node1 returns version 5, Node2 returns version 3.

Version 5 returned to client.

ADDITIONALLY: client library sends version 5 to Node2 (read repair).

Node2 now has the latest version.

Passive consistency mechanism: no explicit anti-entropy needed for hot keys.

Cold keys repaired by: background anti-entropy (Merkle tree comparison).

15. Design a Notification Preference & Delivery System

Architecture Diagram



Key Design Decision: Ensuring Exactly-Once Delivery

Problem: Network failures cause retries. How do you prevent duplicate notifications?

Options

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Idempotency key + dedup store	Each notification has unique ID; check Redis before sending; mark as sent after	Simple; works across retries	Redis TTL must exceed retry window; small window for duplicate if Redis fails
Transactional outbox	Write notification to DB in same transaction as business event; separate worker sends	Atomicity guaranteed; no phantom notifications	Extra DB table; polling or CDC needed
Kafka + exactly-once semantics	Kafka producer idempotency + transactions	Exactly-once at Kafka layer	Complex config; throughput cost
At-least-once + idempotent channels	Retry aggressively; APNs, email, SMS providers handle dedup	Simple; providers often handle it	Depends on provider idempotency; SMS rarely idempotent

■ **Recommended: Idempotency key + Redis dedup for most notifications, transactional outbox for financial/critical notifications**

Idempotency Key Design:

Every notification event includes:

```
idempotency_key = hash(event_id + user_id + channel + template_id)
```

Before sending:

```
result = Redis.SET idempotency_key "processing" NX EX 86400
# NX = only set if Not eXists
# EX 86400 = expire after 24 hours (longer than max retry window)
```

```
if result == nil:
    # Key already exists → duplicate → skip
    return ALREADY_SENT
```

```
# Key was set → first time → proceed to send
```

After successful send:

```
Redis.SET idempotency_key "sent" EX 86400
```

On send failure:

```
Redis.DEL idempotency_key # allow retry
```

This guarantees at-most-once delivery per idempotency_key.

Combined with at-least-once retry logic: exactly-once in practice.

FREQUENCY CAPPING:

Prevents notification spam. Implemented as rate limiting per user per app.

Data structure: Redis Sorted Set

Key: freq_cap:{user_id}:{app_id}:{hour}

Value: count (INCR)

TTL: 1 hour

Check: if count >= 5: skip notification (or downgrade to in-app only)

Why Sorted Set instead of simple INCR?

Sliding window: sorted set stores timestamps, remove old entries

More accurate than fixed 1-hour window (no boundary burst)

16. Design FaceTime / Real-Time Video Conferencing

Clarifying Questions

- Scope: 1:1 only or group calls (up to 32 participants)?
- Platforms: iOS, macOS, web browser?
- Audio only, video, or screen share too?
- What latency target? (FaceTime targets <150ms glass-to-glass)
- Should calls work behind corporate NAT/firewalls?
- Recording / transcription in scope?

Estimation

1B Apple devices; assume 50M concurrent FaceTime calls at peak
1:1 calls: ~45M calls × 2 peers = 90M streams
Group calls: ~5M calls × avg 4 = 20M streams
Total streams: ~110M simultaneous

Per stream bandwidth:

Video: H.265 720p @ 1.5 Mbps

Audio: Opus @ 32 Kbps

Total: ~1.5 Mbps per stream

Total egress: $110M \times 1.5 \text{ Mbps} = 165 \text{ Tbps}$ (served from SFU servers globally)

Signaling events: $50M \text{ calls} \times 10 \text{ events each} = 500M/\text{day} = \sim 6,000 \text{ events/sec}$

High-Level Architecture Diagram

```

[iPhone A]■■■■■HTTPS/WS■■■■■[APPLE SIGNALING SERVER]■■■■■HTTPS/WS■■■■■[iPhone B]
      ■
      SDP Offer/Answer exchange
      ICE candidate exchange
      (session negotiation)

```

[iPhone A] ■■■ DTLS-SRTP ■■■ [iPhone B]

Secure Enclave signs DTLS cert Direct UDP hole-punch

No media touches Apple servers (STUN-assisted NAT traversal)

```

[ iPhone A ] [ DTLS-SRTP ] [ TURN RELAY SERVER ] [ DTLS-SRTP ] [ iPhone B ]
              (Apple-operated relay,
                relays encrypted packets,
                cannot decrypt SRTP)

```

```

[iPhone A] ■■■DTLS-SRTP■■■
[iPhone B] ■■■DTLS-SRTP■■■
[iPad C] ■■■DTLS-SRTP■■■■■■■[SFU MEDIA SERVER]
[Mac D] ■■■DTLS-SRTP■■■ (Selective Forwarding Unit)
[AirPods E] ■■■■■■■■■■■■■■ ■
                                     ■ ■ Forwards encrypted streams
                                     ■ (no decryption at SFU)
                                     ■
Each participant receives streams
from all other participants
SFU selects which spatial layer
to forward per receiver bandwidth

```

Capture → Encode (H.265 HW) → Packetize (RTP) → Encrypt (DTLS-SRTP)
 → Congestion Control (TWCC) → UDP → Network

Network → UDP → Decrypt (DTLS-SRTP) → Jitter Buffer → Decode (H.265 HW)
 → Render (60fps)

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
P2P (mesh)	Each peer sends to every other peer	No server cost; lowest latency; E2E encrypted	Upload bandwidth = (N-1) × stream; N=5 needs 4 uploads	1:1 calls

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
SFU	Each peer sends 1 stream to SFU; SFU forwards to others	Only 1 upload per sender; scales to 32; E2E possible with key distribution	Server cost; server touches packets (even if encrypted)	Group calls up to 32
MCU	Server decodes all streams, mixes into one, re-encodes	Client only receives 1 stream regardless of N	Server decodes = CPU-intensive; breaks E2E; high latency	Legacy, WebEx-style; NOT recommended
Hybrid	1:1 → P2P; group → SFU	Best of both; no server for small calls	More complex routing logic	Apple FaceTime actual approach

■ **Recommended: Hybrid — P2P for 1:1, SFU for group**

Key Design Decision 2: NAT Traversal

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
STUN	Server tells peer its public IP:port; peers attempt direct UDP	Free; fast; works behind cone NAT	Fails on symmetric NAT (each destination gets different port)
TURN	Server relays all packets between peers	Works everywhere, even symmetric NAT	Server bandwidth cost; ~3-5% of calls need TURN
ICE	Framework: tries host → STUN → TURN in priority order	Automatic fallback; industry standard	Connection setup latency during candidate gathering

■ **Recommended: ICE with STUN + TURN fallback**

- ~85% of calls: direct P2P via STUN
- ~12% of calls: STUN-assisted hole-punch
- ~3% of calls: TURN relay (symmetric NAT, strict firewalls)

ICE Candidate Types:

1. host: 192.168.1.5:50000 (LAN address – fastest if both on same network)
2. srflx: 203.0.113.1:50000 (server-reflexive – public IP via STUN)
3. relay: 198.51.100.2:3478 (relayed via TURN – last resort)

ICE checks each pair in priority order.

First candidate pair to succeed wins → connection established.

Key Design Decision 3: Codec Selection

<i>Codec</i>	<i>Compression</i>	<i>HW Support</i>	<i>Latency</i>	<i>Best For</i>
H.264 (AVC)	Good	All Apple devices	Low	Compatibility fallback
H.265 (HEVC)	40% better than H.264	A9+ chips	Low	Primary FaceTime video
AV1	30% better than HEVC	A17+, M3+	Higher encode	Future; streaming
Opus	Excellent for voice	Software	Very low	All FaceTime audio
AAC-ELD	Apple Enhanced Low Delay	HW accelerated	<20ms	FaceTime audio specifically

■ **Recommended: H.265 (HEVC) for video + AAC-ELD for audio on Apple devices**

Deep Dive: DTLS-SRTP — Why This, Not TLS?

- Retransmission causes head-of-line blocking
- A lost packet stalls all subsequent packets
- 200ms HOL block = visible freeze in video

- DTLS: TLS handshake adapted for UDP (handles lost handshake packets)
- SRTP: Encrypts RTP payload + authenticates packets
- A lost packet = just that frame is lost, video continues
- Keys negotiated via DTLS, then used for SRTP encryption

Each RTP packet:

The diagram illustrates the structure of an RTP packet. It is represented as a horizontal bar divided into three distinct sections. The first section on the left is labeled 'RTP Header' and contains the sub-labels '(SSRC, seq, timestamp)'. The middle section is labeled 'Encrypted Payload' and contains the sub-label '(AES-128-CTR)'. The final section on the right is labeled 'Auth Tag' and contains the sub-label '(HMAC-SHA1)'. The entire bar is enclosed in a black border.

In FaceTime group calls, each participant has their own SRTP key. The SFU forwards encrypted SRTP packets without decrypting. Keys distributed via signaling server (E2E identity-verified).

Deep Dive: Adaptive Bitrate — Congestion Control

Problem: Network bandwidth fluctuates. If you send too fast $\rightarrow$ packet loss $\rightarrow$ video artifacts + audio cuts. If too slow $\rightarrow$ underutilize bandwidth.

Solution: Transport-Wide Congestion Control (TWCC)

Sender stamps every RTP packet with transport-wide sequence number.

```
Receiver sends feedback: "packets 1,2,4 arrived; 3 lost; arrival times: ..."
```

Sender runs GCC (Google Congestion Control) / Apple's variant:

Estimate available bandwidth using:

- Packet loss rate (if >2%, reduce bitrate 10%)
- Inter-packet delay gradient (increasing delay → congestion signal)
- RTT trend

Adaptive layers (Simulcast):

Sender transmits 3 spatial layers simultaneously:

Low: 180p @ 150 Kbps

Mid: 360p @ 500 Kbps

High: 720p @ 1.5 Mbps

SFU forwards the layer each receiver can handle based on their bandwidth.

Receiver can upgrade/downgrade without sender re-encoding.

Result: Smooth quality degradation under poor network vs. freezing/dropping.

Deep Dive: Jitter Buffer

Network introduces variable delay (jitter) even on packets arriving in-order.
Without jitter buffer: audio pops, video stutters.

Jitter Buffer:

Hold incoming packets briefly, release at fixed rate.

Adaptive jitter buffer:

Target delay = percentile(recent_jitter, 95th)
If jitter low → shrink buffer → reduce delay
If jitter high → grow buffer → prevent underruns

Audio: 40-120ms jitter buffer (imperceptible to user)

Video: 100-300ms jitter buffer (buffering frames for smooth playback)

Trade-off: larger buffer = more robustness but more latency.

FaceTime target: <150ms glass-to-glass → must keep jitter buffer lean.

Packet Loss Concealment (PLC):

If packet lost and not recovered by FEC:

Audio: interpolate last frame (pitch-matched synthesis)

Video: repeat last frame or use error concealment decoder

Failure Modes

Failure	Detection	Recovery
P2P path breaks mid-call	ICE consent freshness (ping every 15s)	ICE restart: gather new candidates, re-negotiate
SFU server crash	Client heartbeat timeout	Reconnect to different SFU in same region
High packet loss	RTCP RR loss fraction	Reduce bitrate, request keyframe (PLI)
CPU overload (encode lag)	Encode time > frame budget	Drop to lower resolution/framerate
Signaling server down	WebSocket disconnect	Reconnect with exponential backoff; call stays alive via DTLS

Privacy Considerations

- E2E encryption^{**}: DTLS-SRTP with per-call keys; Apple servers cannot decrypt media
- Signaling metadata^{**}: Apple sees who called whom and when; minimized, not content
- TURN relay^{**}: relays encrypted SRTP packets; cannot decrypt
- iCloud backup^{**}: FaceTime calls not backed up to iCloud
- Transparency report^{**}: Apple publishes legal demands received for call metadata

17. Design Apple Pay Backend

Clarifying Questions

- Scope: in-store NFC, in-app purchases, or both?
- Card provisioning flow or just the payment authorization flow?
- How much fraud detection depth required?
- Settlement + reconciliation in scope?
- Apple Cash (P2P) or only card-based pay?

Estimation

Apple Pay: 500M+ users; ~6B transactions/year = ~190 transactions/sec avg
Peak (Black Friday): 10x = ~1,900 TPS
Authorization latency SLA: <500ms end-to-end (tap to approval)
Fraud scoring: <50ms inline (must complete before auth response)

Transaction sizes: ~\$50 avg, range \$0.01 - \$10,000
Chargeback rate target: <0.1% (industry avg ~0.6%)

High-Level Architecture Diagram

[illegible]

- Stores Device Account Number (DAN), not real PAN
- Generates one-time payment cryptogram per transaction
-
- ▼

- Communicates with merchant NFC terminal
- Sends: DAN + cryptogram + amount + merchant ID
-
- ▼

■
▼

■
▼

- Detokenizes DAN → real PAN
- Routes to issuing bank
-
- ▼

- Verifies funds, fraud check
- Approves / Declines
-
- Response path reverses

Journal Pre-proof

■ ▼

- Requests tokenization from Card Network Token Service Provider (TSP)
- TSP issues Device Account Number (DAN) – replaces PAN
- DAN stored in Secure Element on device
- Apple never sees the real PAN
- ▼

- Verifies cardholder identity: SMS/call to bank
- Bank may require additional auth (green/yellow/red path)
-
- ▼

- Manages token status: active, suspended, deleted
- Handles: lost device → suspend token → new device → new token
- Handles: device wipe → token deleted

■ Recommended: Pre-auth lightweight ML score inline + async deep fraud review

Timeline:

T=0ms: Tap iPhone to terminal
T=20ms: Cryptogram generated in Secure Element
T=50ms: Card network receives auth request
T=60ms: Inline fraud score computed (<10ms): device trust + amount + velocity
Score: LOW → auto-approve path; HIGH → route to manual review queue
T=150ms: Issuer auth decision
T=200ms: Approval returned to terminal
T+async: Deep fraud ML runs on full feature set; flag if suspicious

Key Design Decision 2: Idempotency for Payment Authorization

Option	How It Works	Pros	Cons
Idempotency key per transaction	Unique key in request; server deduplicates retries	Industry standard; simple	Key must be stored with sufficient TTL
Cryptogram uniqueness	Each tap generates unique cryptogram; network rejects replays	Built into EMV standard; no extra work	Cryptogram expires; replay window narrow
Exactly-once via Kafka	Produce to Kafka with transaction ID; consume exactly once	Strong guarantee	Latency overhead; overkill for synchronous auth

■ Recommended: EMV cryptogram uniqueness (built-in) + idempotency key for in-app

EMV Cryptogram (in-store):

Generated by Secure Element per-transaction using:

- Application Transaction Counter (ATC): monotonically increments per tap
- Transaction amount + merchant ID + timestamp
- Signed with card's private key (in Secure Element)

Card network validates: ATC hasn't been replayed, cryptogram valid.

Physical replay attack impossible: Secure Element won't re-sign same ATC.

In-App Idempotency Key:

payment_request_id = UUID generated client-side before showing Apple Pay sheet

Sent with payment token to merchant → merchant → processor

If network retry: same UUID → processor returns cached result, no double-charge

Deep Dive: Hardware Security Module (HSM)

Problem: Private keys (for token decryption, for cryptogram validation) must be protected from software attacks, insider threats, OS vulnerabilities.

HSM: Tamper-evident, tamper-resistant hardware device.

Properties:

- Keys never leave the HSM in plaintext
- All crypto operations happen inside the device
- Physical tamper detection: zeroizes keys if opened
- FIPS 140-2 Level 3 or Level 4 certified
- 100,000+ operations/sec with <1ms latency

Apple usage:

Merchant private key (for in-app payment token decryption): stored in HSM

Card network validation keys: stored in HSM at network data centers

Provisioning service signing keys: HSM

Secure Enclave vs HSM:

Secure Enclave: Apple's on-device HSM

- Separate processor, isolated from main CPU
- Runs its own OS (sepOS)
- Stores: Face ID data, device key, card DANs
- Cannot be accessed by iOS even if compromised

Server HSM: Rack-mounted hardware (Thales, nCipher)

- Used server-side for Apple Pay provisioning and token services

Deep Dive: Payment Tokenization

Problem: Merchants shouldn't store real card numbers (PAN = Primary Account Number).
PAN breach → reissue all cards.

Tokenization:

PAN: 4532 1234 5678 9012 (Visa real card number)

Token: 4895 5678 1234 0000 (Device Account Number, looks like card number)

Token properties:

- Same format as PAN (16 digits, valid Luhn checksum) → works in existing infra
- Usable only by specific device + specific merchant (scoped)
- If token leaked: attacker can't use it on different device/merchant
- Issuer links token → PAN internally

Token lifecycle:

1. Provision: device → Apple → Card Network TSP → issues DAN → stored in Secure Element
2. Use: per-transaction cryptogram, never the raw DAN
3. Suspend: lost device → token suspended → transactions declined
4. Delete: wipe device → token deleted from Secure Element
5. Re-provision: new device → new DAN issued → old DAN invalidated

Failure Modes

Failure	Impact	Mitigation
Card network outage	Auth impossible	Fallback to offline PIN for NFC (EMV spec allows)
Apple provisioning service down	Can't add new cards	Existing cards on device unaffected; retry provisioning
HSM unavailable	In-app payment token decryption fails	HSM HA cluster; N+2 redundancy
Fraud service latency spike	Auth path delayed	Circuit breaker: skip inline fraud score, approve + flag async
Cryptogram replay attempt	Declined by card network	ATC counter prevents replay at network level

Privacy Considerations

- Apple never sees the real PAN**: provisioning flow gives DAN directly to Secure Element via TSP
 - Apple never sees what you bought**: transaction goes merchant → acquirer → card network; Apple not in this path
 - Face ID data**: stored only in Secure Enclave; never sent to Apple servers
 - Minimal metadata**: Apple sees transaction was authenticated (Face ID success), not what for
-

18. Design Real-Time Collaborative Editing (iWork / Notes)

Clarifying Questions

- Scope: text editing only or structured content (tables, spreadsheets, drawings)?
- Real-time: multiple cursors visible, or eventual consistency acceptable?
- Offline-first: must work with no internet, sync on reconnect?
- Max document size? (Notes vs Keynote have very different sizes)
- Max concurrent editors per document?
- Conflict resolution strategy: last-write-wins, user-prompted, or automatic merge?

Estimation

iCloud Notes: 500M+ users; assume 10M actively collaborative sessions
Concurrent editors per doc: avg 2-3, max ~20 (shared Notes)
Operation rate: 5 keystrokes/sec per user $\times$ 10M = 50M ops/sec
(but most operations are local; ~1% need server broadcast = 500K ops/sec)

Op size: ~50 bytes average (insert/delete + position + timestamp + author)
Bandwidth: 500K ops/sec $\times$ 50 bytes = 25 MB/s (very manageable)

Snapshot size: typical Notes doc = 10-100KB
Presence updates: cursor position every 100ms = 10/sec per user

High-Level Architecture Diagram

[illegible]

```

[Device offline] → Ops queued locally (SQLite WAL)
[Device reconnects] → Upload queued ops with vector clock
[Server] → Merge ops using CRDT semantics → broadcast to others
[Others] → Apply remote ops → converge to same state

```

Key Design Decision 1: Consistency Model — OT vs CRDT

<i>Option</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>	<i>Best When</i>
Operational Transform (OT)	Transform operations against concurrent ops to maintain intent; requires server to serialize	Proven (Google Docs); compact op representation	Requires central server for serialization; complex transform functions; hard to implement correctly	Central server always available; real-time collaborative text
CRDT (RGA / YATA)	Each insert tied to unique element ID, not position; tombstones for deletes; merge by causality	P2P capable; works offline naturally; no server serialization needed	More memory (tombstones); complex implementation	Offline-first; decentralized; iCloud sync style
Last-Write-Wins (LWW)	Timestamp wins; losing op discarded	Trivially simple	Loses data; concurrent edits → one deleted	Metadata fields only (title, color); never for text body
Three-way merge	Diff base → version A + version B; merge like git	Familiar; handles structural conflicts	Requires common ancestor; not real-time	Offline merge of whole documents

■ **Recommended: CRDT (RGA for text, LWW for metadata) — matches Apple's offline-first philosophy**

Deep Dive: RGA — Replicated Growable Array

Problem with position-based OT:

Doc: "AC"

User1 inserts "B" at pos 1 → "ABC"

User2 inserts "D" at pos 1 → "ADC"

Concurrent: depends on order → "ABDC" or "ADBC"? Intent unclear.

RGA: insert AFTER a specific element, identified by unique ID.

Each character: { id: (agent_id, seq), content: char, after: element_id }

Example:

Initial doc: [ROOT] → [A, id=(alice,1)] → [C, id=(alice,2)]

User1 inserts "B" after (alice,1):

Op: { insert "B" id=(bob,1) after=(alice,1) }

Doc: [ROOT] → [A] → [B] → [C]

User2 inserts "D" after (alice,1):

Op: { insert "D" id=(carol,1) after=(alice,1) }

Doc: [ROOT] → [A] → [D] → [C]

Merge (concurrent inserts at same position):

Both inserted after (alice,1).

Tie-break: sort by agent_id alphabetically.

Result: [ROOT] → [A] → [B] → [D] → [C] (bob before carol)

Both users converge to same order deterministically.

Deletes:

Don't actually remove element (would invalidate other ops referencing it).

Set tombstone flag: { id=(alice,1), deleted=true }

Render: skip tombstoned elements

GC: compact tombstones after all clients have applied the op.

Properties:

- Commutative: apply ops in any order → same result
- Associative: merge any subsets → same result
- Idempotent: apply same op twice → same result as once

→ Strong Eventual Consistency (SEC) guaranteed mathematically

Key Design Decision 2: Presence (Cursor Sharing)

Option	How It Works	Pros	Cons
WebSocket broadcast	Server broadcasts cursor positions to all editors	Simple; real-time	All editors get all cursors even if doc has 20 editors
Pub/Sub (Redis)	Each doc has a channel; clients subscribe	Scales to multiple server nodes	Redis availability affects presence
Ephemeral CRDT	Awareness protocol (like Yjs awareness)	Decoupled from doc CRDT	Extra protocol complexity

■ Recommended: WebSocket + Redis Pub/Sub for presence

```
Cursor update: every 100ms (if changed)
  publish to channel: "presence:{doc_id}"
  payload: { user_id, cursor_pos, selection_start, selection_end, color }

Other editors receive and render colored cursors.
TTL: if no update in 5s → remove cursor (user went idle or disconnected)
Presence data is ephemeral – never persisted.
```

Failure Modes

<i>Failure</i>	<i>Recovery</i>
Server crash mid-session	Client buffers ops locally; reconnects to new server; replays from last ack'd sequence
Network partition (device offline)	Local CRDT continues; ops queued; merge on reconnect
Op log corruption	Restore from latest snapshot + ops after snapshot
Conflicting deletes (both delete same word)	CRDT handles: tombstone applied once, idempotent
Clock skew causing wrong order	Vector clocks / Lamport timestamps; not wall clock

Privacy Considerations

- E2E encryption for Notes**: iCloud Notes supports E2E encryption (Advanced Data Protection); server stores encrypted CRDT state; cannot read content
- Presence data**: cursor positions ephemeral; not persisted; not accessible to Apple
- Collaboration invites**: iCloud sharing links; access controlled by owner
- Op log**: stored encrypted at rest per user key

19. Design an ML Feature Store & Model Serving Platform

Clarifying Questions

- Teams: who are the users? ML engineers, data scientists, or product engineers?
- Scale of models: how many models in production? (10s vs 1000s)
- Latency requirements: online inference (<10ms for ranking) vs batch (hours)?
- Feature freshness: do features need real-time updates or can they be hourly/daily?

- Scale: how many feature lookups/sec for online serving?
- On-device ML in scope? (Apple runs significant inference on-device)

Estimation

Apple ML workloads (internal platform estimate):

Models in production: ~500 (search ranking, Siri intent, Health anomaly, Fraud, etc.)

Online feature lookups: 1M QPS (App Store ranking + Siri + Spotlight combined)

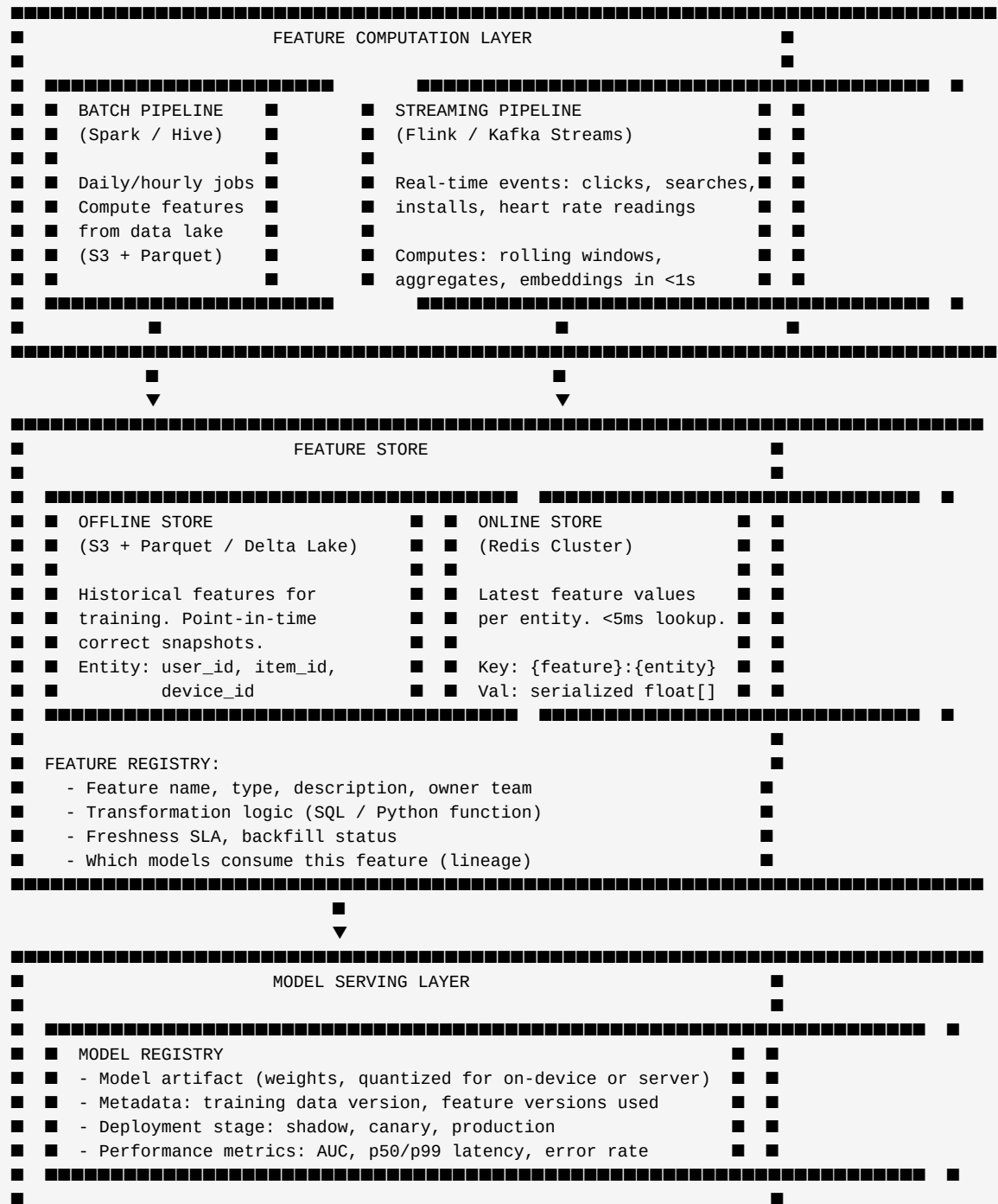
Feature computation: ~10,000 features per model × 500 models = 5M feature definitions

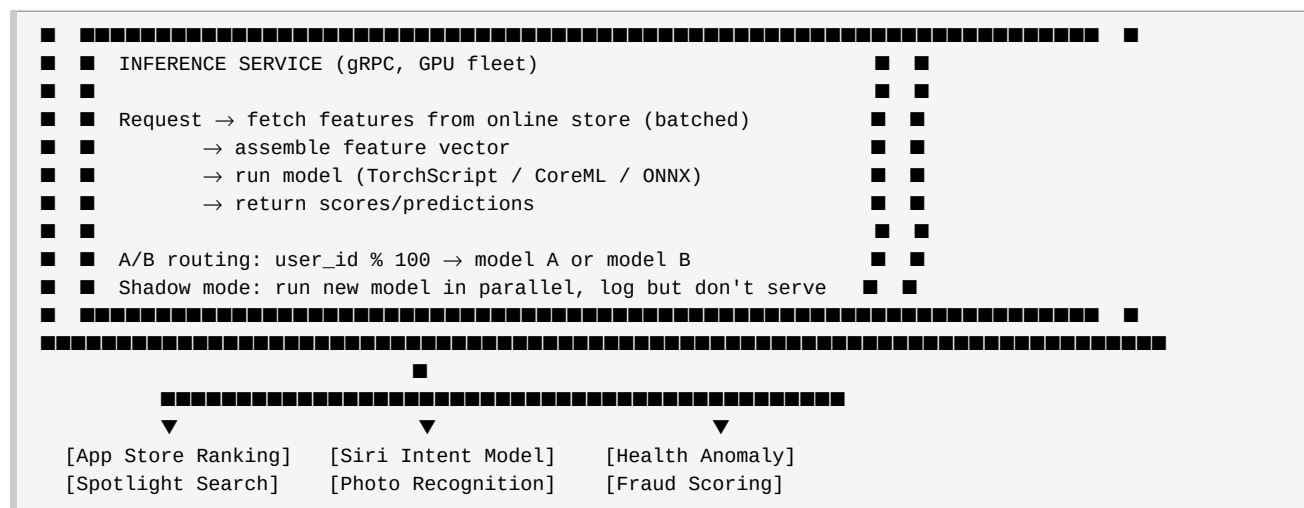
Training jobs: ~1,000/day (retraining, experiments, shadow models)

Batch feature jobs: Spark jobs over 100TB+ of behavioral data/day

Inference latency SLA: <10ms p99 for ranking; <50ms for Siri NLU; <1s for batch scoring

High-Level Architecture Diagram





Key Design Decision 1: Online Feature Store Technology

Option	Latency	Throughput	TTL Support	Best When
Redis Cluster	<1ms	Very high	Yes	Default choice; simple; fast
Cassandra	5-10ms	Very high	Yes (TTL per row)	Feature vectors too large for Redis; need persistence
DynamoDB	5-10ms	High (managed)	Yes	Managed; no ops burden; AWS-hosted teams
Aerospike	<1ms	Extremely high	Yes	Extremely high QPS; hybrid memory+SSD
TiKV	2-5ms	High	Yes	Transactional feature updates needed

■ **Recommended: Redis Cluster for hot features (<1ms latency) + Cassandra for large/cold features**

Feature tiers:

Hot (Redis): User's last 24h clicks, current session context
Key: user:{user_id}:clicks_24h → float[128]
TTL: 86400s

Warm (Cassandra): User's 30-day behavioral aggregates
Key: user_id + feature_name + date
TTL: 30 days

Cold (S3 Parquet): Historical features for training only; not served online

Deep Dive: Point-in-Time Correctness

Problem (Training-Serving Skew):

Model trained on features computed at time T.

At serving time, features are computed at time T'.

If feature definitions or data drifted: model sees different distribution.

This causes silent accuracy degradation – often hard to detect.

Bigger problem – Label Leakage:

Training example: user at time T, label = "installed app" at T+7 days.

Feature: "user's total install count" → if computed now includes post-T+7 installs

→ Feature has future information → artificially high training accuracy

→ Model fails in production (future data not available at serving time)

Point-in-Time Correct Feature Retrieval:

For each training example (entity_id, event_time):

Fetch feature value that was valid AT event_time, not at training time.

Implementation:

Offline store stores (entity_id, feature_name, computed_at, value)

Point-in-time query:

```
SELECT value FROM features
WHERE entity_id = ? AND feature_name = ?
AND computed_at <= event_time
ORDER BY computed_at DESC LIMIT 1
```

Delta Lake / Iceberg time-travel:

Table snapshots at every batch job completion.

Training job reads snapshot from the relevant past timestamp.

Guarantees no future data leaks into training features.

Key Design Decision 2: Model Serving — GPU Fleet vs CPU vs On-Device

<i>Option</i>	<i>Latency</i>	<i>Cost</i>	<i>Privacy</i>	<i>Best When</i>
Server GPU fleet	5-20ms	High	Data leaves device	Large models (LLMs, embedding models); shared across users
Server CPU fleet	10-50ms	Medium	Data leaves device	Smaller models; high QPS; embarrassingly parallel
On-device (CoreML)	<1ms	Zero server cost	Data never leaves device	Privacy-critical; latency-critical; Apple-specific advantage
Hybrid: on-device + server	Variable	Low server cost	Tiered	On-device for fast/private; server for complex/updated models

■ **Recommended: On-device for privacy-sensitive (Health, Photos face recognition) + server GPU for ranking/Siri**

Apple's On-Device ML strategy:

CoreML framework: converts PyTorch/TensorFlow → optimized .mlmodel

Neural Engine: dedicated ML accelerator in Apple Silicon

A17 Pro: 35 TOPS (trillion operations/sec)

M4: 38 TOPS

Models running on-device at Apple:

- Face recognition (Photos)
- Health anomaly detection (Watch)
- Siri wake word detection ("Hey Siri")
- Keyboard prediction (QuickType)
- Image classification (Spotlight visual search)

Benefits:

- Zero network latency
- Works offline
- User data never sent to server
- Scales with device fleet (free compute)

Challenge: model size limit (~100MB for Neural Engine optimized models)

Solution: model distillation + quantization (float32 → int8 → 4x smaller)

Failure Modes

<i>Failure</i>	<i>Impact</i>	<i>Mitigation</i>
Online store cache miss	Feature unavailable → use default/fallback value	Fall back to global avg; never fail inference; log misses
Model serving node crash	Elevated latency	Multiple replicas; load balancer routes around failed node
Feature pipeline stale	Model sees old features	Freshness SLA monitoring; alert if feature > 2× expected age
Training-serving skew	Silent accuracy degradation	Feature logging at serving time; offline comparison with training distribution
New model regression	Production accuracy drops	Shadow mode → canary (1% traffic) → gradual rollout; auto-rollback on metric violation

Privacy Considerations

- On-device inference**: preferred for sensitive data; avoids server-side data exposure
- Private Cloud Compute (PCC)**: for models needing cloud scale but privacy guarantees (server-side Siri); hardware attestation ensures Apple cannot access data
- Feature store access control**: features scoped to owning team; cross-team access requires approval
- Differential Privacy in training**: Apple uses DP-SGD for training on aggregate behavioral data; per-user contribution bounded

20. Design a Large-Scale Telemetry & Crash Reporting Pipeline

Clarifying Questions

- Scope: device crash reports only, or also app performance metrics, user events, system diagnostics?
- Real-time alerting required? (e.g., new iOS bug causing 10% crash rate spike)
- Privacy: user-consented analytics vs always-on diagnostics?
- Retention: how long to keep raw crash reports? (storage vs debugging value)
- Clients: iOS + macOS + watchOS + tvOS + visionOS?
- Symbolication: do we need human-readable stack traces or raw addresses?

Estimation

2B active Apple devices worldwide

Crash rate: ~0.1% daily crash rate → 2M crashes/day = ~23 crashes/sec avg

Bug release spike: single iOS bug can cause 5% crash rate → 100M crashes/day = 1,150/sec peak

App metrics: avg 1,000 metric points/device/day = 2T data points/day

System diagnostics: 10KB/device/day = 20TB raw data/day

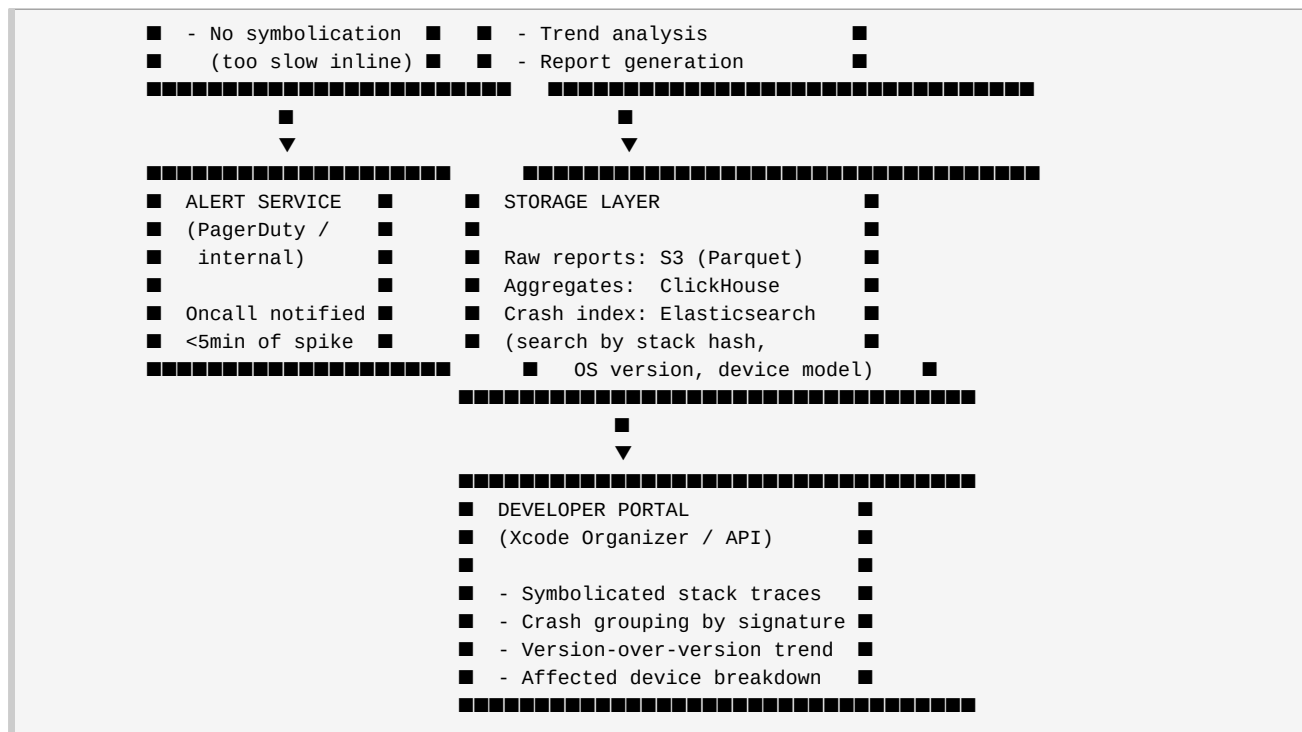
Symbolication: each crash report has ~50 stack frames; $50 \times 2M = 100M$ symbol lookups/day

Retention: raw reports 90 days; aggregates 2 years

Storage: 20TB/day $\times$ 90 days = 1.8 PB raw; compressed ~400TB

High-Level Architecture Diagram

[illegible]



Key Design Decision 1: Partitioning Strategy for Kafka

Option	Partition Key	Pros	Cons
Random	None	Even load	Correlated crashes spread across partitions; harder to aggregate
Device ID	hash(device_id)	All device events ordered	Hot partitions: one device model crashes → many same hash
App + OS version	hash(bundleid + osversion)	Bug spike → same partition → easy to detect	New iOS release can temporarily hot-partition
App ID + time bucket	bundle_id + floor(timestamp, 5min)	Time-windowed processing easy	Burst within bucket → partition overload
Consistent hash with virtual nodes	bundle_id → virtual nodes → physical partitions	Even distribution; hot app doesn't dominate	More complex routing

■ Recommended: hash(bundleid + osversion) for crash_reports topic with over-partitioning

Why over-partition?

Create 1,000 partitions even if you only have 50 consumer threads today.
Kafka reassigns partitions to consumers; easy to scale consumers later.
If partition count is too low, scaling consumers doesn't help.

Hot Partition Mitigation:

Bug in iOS 18.0 → all crash_reports for (com.apple.system, 18.0) → hot.
Solution: add random salt suffix to partition key for known-hot combos.
Key: hash(bundle_id + os_version + random_suffix(0..9))
10x parallelism automatically; consumers aggregate across 10 partitions.

Key Design Decision 2: Symbolication Strategy

Problem:

Raw crash:

Thread 0:

0: 0x00000001004a3c20 (binary offset)
1: 0x00000001004a1f40
...

Useful crash:

Thread 0:

0: -[UIApplication _handleApplicationActivationWithScene:...]
UIKit 0x00000001004a3c20 + 234
1: -[UIWindowScene _activateWithSession:...]
UIKit 0x00000001004a1f40 + 100

Symbolication requires: dSYM file (debug symbols, stripped from release binary)

Apple maintains dSYM for every build ever shipped (massive storage)

dSYM lookup: binary UUID → fetch dSYM → atos tool → resolve address to symbol

Performance decision:

Option	When	Pros	Cons
Inline (at ingestion)	At write time	Reports immediately human-readable	Adds latency to ingestion; dSYM lookup can be slow
Lazy (at query time)	When developer views report	No pre-computation waste	Slow first view; dSYM may have been GC'd
Async batch	Periodic batch job	Decoupled; parallelizable; scalable	Delay before reports are readable
Indexed stack hash	Deduplicate before symbolication	Symbolicate once per unique crash, not per occurrence	Grouping logic must be on raw addresses

■ **Recommended: Hash + deduplicate on raw addresses first → async batch symbolication on unique crash signatures**

Crash Deduplication Pipeline:

- 1. Compute crash signature: hash of top-5 raw stack frame addresses
signature = SHA-256(frame[0] + frame[1] + frame[2] + frame[3] + frame[4])
- 2. Check: have we symbolicated this signature before?
Redis: GET symbolicated:{signature}
→ Yes: return cached symbolicated trace (80%+ cache hit rate)
→ No: enqueue for symbolication job
- 3. Symbolication job (batch every 5 min):
For each unique unsymbolicated signature:
Look up dSYM by binary UUID
Run atos tool: address → symbol name + file + line
Cache result in Redis (TTL 30 days) + store in Elasticsearch

Result: 100M crash reports → ~50K unique signatures → 50K symbolications
99.5% reduction in symbolication work.

Key Design Decision 3: Real-Time Spike Detection

Option	How It Works	Pros	Cons
Threshold alert	If crash_rate > X% in 5-min window → alert	Simple to implement	Static threshold; noisy during launches; misses gradual regressions
Relative change	If crash_rate > 2× baseline → alert	Adapts to app's normal rate	Need stable baseline; short baseline window can miss seasonal patterns
Anomaly detection (ML)	Model normal behavior; alert on deviation	Catches subtle regressions; fewer false positives	More complex; model drift
Rate-of-change	Alert if delta(crash_rate) > threshold per minute	Catches sudden spikes fast	Noisy for apps with bursty traffic

■ **Recommended: Relative change vs rolling 7-day baseline + rate-of-change for sudden spike**

```

Flink streaming job:
Input: crash events from Kafka
Window: tumbling 5-minute window

For each (bundle_id, os_version):
    current_rate = crashes_in_window / active_devices_in_window
    baseline     = avg(same_5min_bucket over last 7 days)

    if current_rate > 3 × baseline AND crash_count > 100: # threshold prevents alert on tiny apps
        emit alert: { app, os_version, current_rate, baseline, timestamp }

Alert suppression:
App Store release detected → suppress alerts for 2h (new version spike expected)
iOS release detected      → suppress alerts for 4h
Cooldown: once alerted, no re-alert for same (app, version) for 1h

```

Deep Dive: Schema Evolution with Protobuf

```

Problem: 2B devices; old devices run iOS 15, new run iOS 18.
If schema changes break old clients → old devices can't report.
If new fields are required → old parsers fail on new reports.

Protobuf handles this with field numbers:
message CrashReport {
    required uint64 device_id      = 1;  // original fields
    required string os_version    = 2;
    required string bundle_id     = 3;
    repeated StackFrame frames    = 4;

    optional string architecture = 5;  // added in v2 – old clients omit; new parsers handle
    optional uint32 exception_code = 6; // added in v3
    // NEVER reuse field numbers (would cause type mismatch with old data)
    // NEVER remove required fields (old servers parsing new reports would fail)
}

Rules:
■ Add new optional fields – safe, backward and forward compatible
■ Rename fields – safe (field number is identity, not name)
■ Remove required fields – breaks old parsers
■ Change field types – breaks all versions
■ Reuse field numbers – causes corrupt data silently

Schema registry (Confluent schema registry or internal):
Each Kafka message includes schema version ID (4 bytes header).
Consumer fetches schema by ID → correct deserialization.
Enforces compatibility rules before new schema published.

```

Failure Modes

Failure	Impact	Mitigation
Kafka broker failure	Reports delayed, not lost	Replication factor 3; ISR (in-sync replicas) guarantee durability
Ingestion spike (bug in iOS release)	API gateway overwhelmed	Rate limit per device (max 100 reports/hour); queue on device and retry
dSYM missing for build	Crashes unsymbolicated	S3 versioning ensures dSYMs never deleted; alert if dSYM missing for shipped build
ClickHouse slow query	Dashboard latency	Pre-aggregate into materialized views for common queries (crash rate by version)
Device sends duplicate reports	Storage waste	Idempotency: dedup by (<i>deviceid</i> + <i>crashtimestamp</i> + <i>topframehash</i>)

Privacy Considerations

- Consent**: Diagnostics & Usage sharing is opt-in; separate from mandatory crash logs
 - Anonymization at source**: device strips user-identifiable paths before sending
 - No device fingerprinting**: *device_id* is resettable; not correlated to Apple ID on server
 - Differential Privacy**: aggregate statistics computed with DP before publishing to developers
 - Data minimization**: raw reports retained 90 days; purged after symbolication and aggregation
 - Developer access**: developers see symbolicated stack traces + aggregate stats, never raw device IDs
-

21. Design Apple TV+ Video Streaming Platform

Clarifying Questions

- VOD only or live streaming too (live sports, Beats 1 radio)?
- Global reach: all countries where Apple TV+ is available (~100+ countries)?
- Device targets: Apple TV box, iPhone, iPad, Mac, Smart TVs, web browser?
- Max resolution: 4K HDR + Dolby Vision + Dolby Atmos?
- Offline download in scope?
- Concurrent viewers during a major live event (e.g., MLS, Friday Night Baseball)?

Estimation

Apple TV+: ~25M subscribers; peak ~5M concurrent streams
Video catalog: ~500 titles (Apple original-only); growing

Per stream bandwidth:

4K HDR:	15-20 Mbps (HEVC/H.265)
1080p:	8 Mbps
720p:	4 Mbps
480p:	1.5 Mbps
Audio:	Dolby Atmos ~768 Kbps; AAC ~256 Kbps

Peak egress: 5M streams × avg 8 Mbps = 40 Tbps (served via CDN)

Transcoding: each title → ~15 renditions (4 resolutions × bitrate ladders + audio tracks)
1 hour of 4K raw footage → 2 TB input → ~400 GB output (all renditions combined)
500 titles × 400 GB = 200 TB transcoded content total (small – Apple originals only)

Segment size: 6-second HLS segments

Live event: 5M concurrent viewers; ~10x normal load

High-Level Architecture Diagram

Raw ProRes / IMF (Interoperable Master Format)
+ Dolby Vision metadata + Dolby Atmos audio



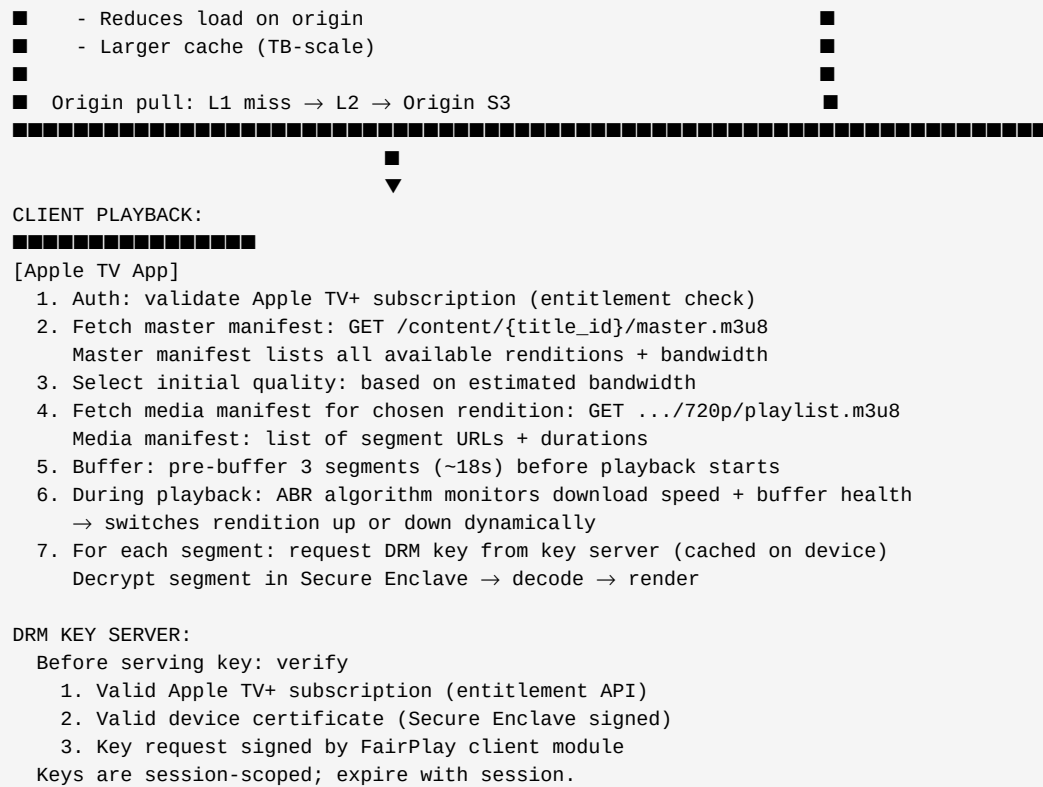
- Validate: completeness check, loudness normalization
- Store original in S3 (Glacier for long-term archive)
- Trigger transcoding pipeline





Segments: /content/{title id}/{rendition}/{segment N}.fmp4

- ▼



Key Design Decision 1: Streaming Protocol — HLS vs DASH vs Smooth Streaming

<i>Protocol</i>	<i>Creator</i>	<i>Segment Format</i>	<i>Latency</i>	<i>Best When</i>
HLS (HTTP Live Streaming)	Apple	MPEG-TS or fMP4	6-30s (standard), 2s (LL-HLS)	Apple devices, wide browser support
DASH (Dynamic Adaptive Streaming)	MPEG consortium	fMP4 (CMAF)	2-10s	Android, Smart TVs, cross-platform
Smooth Streaming	Microsoft	fMP4	~10s	Azure, legacy Xbox
CMAF (Common Media Application Format)	Apple + Microsoft	fMP4 (shared)	2s (LL)	Single segment format for both HLS+DASH

<i>Protocol</i>	<i>Creator</i>	<i>Segment Format</i>	<i>Latency</i>	<i>Best When</i>
WebRTC	Google	No segments	<500ms	Ultra-low latency; not scalable for VOD

■ Recommended: HLS (fMP4/CMAF) primary + DASH for non-Apple clients

Why fMP4 over MPEG-TS for HLS segments?

MPEG-TS:

- Original HLS format; widely compatible
- Each segment is self-contained (has full headers)
- Larger overhead per segment (~10% overhead)

fMP4 (Fragmented MP4):

- CMAF-compatible: same segments work for HLS and DASH
- Lower per-segment overhead
- Supports CMAF chunked transfer (byte-range for LL-HLS)
- Better for Dolby Vision/Atmos metadata
- Required for Low-Latency HLS

Apple TV+ uses fMP4 (CMAF) segments with HLS manifests.

Non-Apple clients (Samsung TV, etc.) served DASH manifests pointing to same fMP4 segments.

Single segment store, dual manifests. Saves 50% storage vs dual encoding.

Key Design Decision 2: Adaptive Bitrate Algorithm

<i>Algorithm</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Throughput-based	Measure download speed of last segment; pick bitrate below measured speed	Simple; responsive	Unstable: one slow segment → drops quality; oscillates
Buffer-based (BBA)	Choose bitrate based on current buffer level; ignore throughput	Stable; reduces oscillations	Slower adaptation to bandwidth changes
BOLA (Buffer Occupancy based Lyapunov Algorithm)	Utility-maximizing: maximize quality subject to buffer constraint	Near-optimal; mathematically grounded	Complex; tuning required
MPC (Model Predictive Control)	Predict future bandwidth from past N chunks; optimize over horizon	Best quality; lowest rebuffer	High compute; requires accurate predictor

<i>Algorithm</i>	<i>How It Works</i>	<i>Pros</i>	<i>Cons</i>
Neural/ML-based (Pensieve)	RL-trained policy; learns from rebuffering events	Adapts to diverse network conditions	Requires training; black-box

■ **Recommended: Hybrid BBA + throughput for Apple TV app (stable quality preferred over peak quality)**

Buffer state machine:

Buffer < 5s: Emergency → drop to lowest bitrate immediately
 Buffer 5-15s: Cautious → only upgrade if throughput clearly supports it
 Buffer > 15s: Aggressive → upgrade opportunistically
 Buffer > 30s: Pause downloading new segments

Segment download speed measurement:

Use exponential moving average (EMA) of last 3 segments:

$$\text{estimated_bw} = 0.3 \times \text{last_segment_bw} + 0.7 \times \text{previous_estimate}$$

Safety margin: only use 80% of estimated bandwidth for selection
 (accounts for estimation error + network variance)

Deep Dive: FairPlay Streaming DRM

Goal: Prevent unauthorized playback of encrypted content.
Only authenticated, subscribed devices should decrypt.

Key components:

1. Content Encryption (at origin):
Each segment encrypted with AES-128-CTR.
Encryption key (CEK) stored in Apple Key Server, NOT in segment.
Segment contains: URL of key server + key ID.
2. FairPlay Client (on device):
Built into AVFoundation framework.
Hardware-rooted trust: leverages Secure Enclave.
Cannot be bypassed by jailbroken app (Secure Enclave is isolated).
3. Key Request Flow (per playback session):
Player encounters encrypted segment in manifest.
FairPlay client generates SPC (Server Playback Context):
SPC = encrypt(device_cert + title_key_request) with Apple certificate

Player sends SPC to Key Server:

```
POST /fps/keydelivery
Body: { spc: base64(SPC), asset_id: "...", session_id: "..." }
```

Key Server:

1. Decrypt SPC using Apple FairPlay private key
2. Verify device certificate (valid Apple device, not revoked)
3. Check entitlement: is this Apple ID subscribed to Apple TV+?
4. Return CKC (Content Key Context):
CKC = encrypt(content_encryption_key) with device-specific key

FairPlay client decrypts CKC in Secure Enclave → gets CEK → decrypts segments.
CEK never appears in plaintext outside Secure Enclave.

4. Offline Download:
User downloads: request persistent key (not session key).
Key stored in on-device secure container.
Key has expiry: 30 days after first use (configurable by Apple).
On expiry: re-request key (requires network + valid subscription).
Subscription lapse: persistent keys revoked → offline content inaccessible.

Why per-segment keys are NOT used in practice:

100 segments/hour × millions of streams = billions of key requests.
Instead: one key per title or per episode, cached on device for session duration.

Key Design Decision 3: Live Streaming Architecture

<i>Option</i>	<i>Latency</i>	<i>Scale</i>	<i>Complexity</i>
Standard HLS (6s segments)	20-30s	Unlimited via CDN	Low
Low-Latency HLS (LL-HLS)	2-3s	Unlimited via CDN	Medium

<i>Option</i>	<i>Latency</i>	<i>Scale</i>	<i>Complexity</i>
WebRTC	<500ms	Hard to scale to millions	High
RTMP → HLS transcode	10-20s	Good	Medium
SRT (Secure Reliable Transport)	1-4s	Good for ingest	Medium (ingest only)

■ Recommended: LL-HLS for live sports (Apple MLS Season Pass)

LL-HLS key features:

- Partial segments: 200ms chunks within a 2s segment
- Server push (HTTP/2): server pushes next partial segment before client asks
- Blocking playlist reload: server holds response until new segment available (eliminates polling delay)
- Preload hints: manifest tells client where next segment will be

Result: 2-3s glass-to-glass latency vs 20-30s for standard HLS

Live event surge (5M concurrent viewers):

CDN handles horizontal scale.

Origin: single-origin-per-stream with shield layer absorbs CDN misses.

Ingest: broadcaster → SRT/RTMP → Apple ingest server → transcode → segment → push to CDN origin.

Segment CDN TTL: 2-3s (LL segments expire quickly; long-lived segments cached normally).

Failure Modes

<i>Failure</i>	<i>Recovery</i>
CDN PoP goes down	Client retries → CDN routes to next nearest PoP
Transcoding job fails	Job queue retry; alert; fallback to lower-quality rendition if available
Key server overload	DRM keys cached on client for session; key server outage = new sessions fail, existing continue
Live ingest connection drops	Backup ingest endpoint; broadcaster switches to backup within seconds
Origin S3 degraded	CDN serves from cache; origin misses fail gracefully (stale content)

Privacy Considerations

- Viewing history**: used for recommendation; stored encrypted; user can delete from account
 - DRM key requests**: Apple sees which titles a device decrypts → know what user is watching
 - Offline downloads**: key expiry limits offline access after subscription lapses
 - Sign-in with Apple**: Apple TV+ uses Apple ID; Apple knows subscriber identity
 - Ad-free**: Apple TV+ has no ads; no third-party ad tracking pixels
-

22. Design Find My Network

Clarifying Questions

- Scope: AirTag tracking, iPhone/iPad/Mac tracking, or both?
- Offline finding (crowd-sourced Bluetooth) in scope?
- Precision finding (UWB for last-meter accuracy) in scope?
- What privacy guarantees are required?
- Lost Mode and notifications in scope?

Estimation

2B Apple devices participating as "finders" (passive Bluetooth scanning)
500M items being tracked (AirTags + Apple devices)
Location report upload: each finder uploads at most 1 report per discovered item per ~hour
500M items × avg 10 finders see it/hour × 1 report = 5B reports/day = ~58,000 reports/sec

Report size: ~100 bytes (encrypted location + timestamp + key identifier)
Daily upload volume: 5B × 100 bytes = 500 GB/day (manageable)

Owner queries (checking location): 500M items × 1 query/day = ~5,800 queries/sec
Precision finding sessions: UWB range ~10m; ~100M UWB sessions/day

High-Level Architecture Diagram

[illegible][illegible]

- ▼

[REDACTED]

[REDACTED]

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

- ▼

```
private_key_i)
```

```
APPLE CANNOT SEE:
```

- Real location of item (reports are encrypted)
- Who owns the item (key_hash not linked to Apple ID)
- Who uploaded the report (finder anonymized)

```
Owner queries from Find My app
```

```
▼
```

```
OWNER DEVICE (Find My app):
```

```
Decrypts location reports → displays on map
```

```
Multiple reports → show most recent + history trail
```

```
"Play Sound" → APNs command → AirTag beeps (when online)
```

```
Lost Mode → mark as lost → APNs alert to all finders
```

```
PRECISION FINDING (UWB – within ~10 meters):
```

```
iPhone U1/U2 chip (Ultra-Wideband, IEEE 802.15.4a)
```

```
Time-of-Flight ranging: accurate to ~10cm
```

```
Directional arrow + distance on screen
```

```
Only works device-to-device, not through Find My server
```

```
P2P encrypted Bluetooth + UWB session
```

Key Design Decision 1: Privacy-Preserving Location — Cryptographic Design

Core challenge:

How does owner retrieve location without Apple knowing the owner or location?

Solution: Rolling Key Derivation

AirTag has: (master_private_key, master_public_key) – generated at factory

Owner registers: uploads SHA256(master_public_key) to Apple, linked to Apple ID

Key schedule (15-minute rotation):

private_key_i, public_key_i = HKDF_derive(master_key, time_bucket_i)

HKDF: HMAC-based Key Derivation Function (RFC 5869)

time_bucket_i = floor(unix_timestamp / 900) // new key every 900s = 15 min

What finders upload:

key_hash = SHA256(public_key_i) // not the key itself

encrypted_location = ECIES(location, public_key_i)

No Apple ID, no device ID, no owner info.

What owner queries with:

For last 7 days: compute all (time_bucket_i, SHA256(public_key_i)) tuples

7 days × 96 buckets/day = 672 key hashes per AirTag to query

What Apple server sees:

A list of key hashes → returns matching reports

Server cannot link key_i to master key (HKDF is one-way)

Server cannot link reports to Apple ID (query is authenticated but blind)

Why this is truly private:

- Finder doesn't know whose AirTag they saw (just a random-looking public key)
- Apple cannot decrypt location (doesn't have private keys)
- Apple cannot correlate key_i to key_j (different derivations)
- Only the owner, with master_private_key, can derive all private_key_i and decrypt

Key Design Decision 2: Anti-Stalking Protection

Problem: AirTags could be used to covertly track people.

Person A places AirTag in Person B's bag.

Person B's iPhone would report B's location to A.

Apple's multi-layer anti-stalking:

1. Unknown AirTag alert:

If an AirTag travels with someone who is NOT its owner for >8-24 hours
(configurable: was ~3 days initially, reduced after criticism)

→ That person's iPhone shows notification: "AirTag Found Moving With You"

→ They can play a sound, view owner's partial phone number, disable AirTag

2. AirTag sound:

AirTags separated from owner > 3 days → play random sound when moved

Even without iPhone, physical sound reveals presence

(Criticized: 3 days was too long; Apple reduced delay)

3. Android app (Tracker Detect):

Released to protect non-iPhone users

Scans for unknown AirTags nearby

4. NFC tap:

Anyone can tap AirTag → see serial number + last 4 digits of owner's phone

If AirTag in Lost Mode: owner's contact info shown

Engineering design:

Timer starts when AirTag is NOT co-located with registered owner.

Timer reset: if AirTag comes within Bluetooth range of owner device.

Timer state: stored on AirTag itself (low-power timer in firmware).

Alert delivery: via Find My server → APNs → victim's iPhone.

Key Design Decision 3: Scale — Report Storage and Retrieval

Option	Storage Model	Query Model	Pros	Cons
Key-value (key_hash → reports)	Hash map per key	O(1) per key lookup	Fast; simple	672 queries per AirTag per refresh
Batched multi-get	Same key-value but batched	Single request with 672 key hashes	Reduces round trips to 1	Larger request payload
Owner-keyed store	Reports indexed by owner	O(1) per owner	Single query	Breaks privacy: links reports to owner
Bloom filter pre-filter	Bloom filter of known key_hashes	Avoid full DB scan	Reduces storage reads	Bloom false positives; extra layer

■ **Recommended: Batched multi-get — owner sends all 672 key hashes in one request**

Request:

```
POST /find-my/query
Auth: Apple ID token (TLS; but server doesn't store which Apple ID queried which hashes)
Body: { key_hashes: ["abc123...", "def456...", ...] } // up to 672 hashes
```

Server:

Batch lookup: Redis pipeline → GET for each key_hash
Return matching reports only (missing keys → no reports → no results)

Partitioning:

```
key_hash[0:2] → shard ID (256 shards)
Each shard owns key_hashes with that prefix
Multi-get fans out to relevant shards → aggregate → return
```

TTL: each report has 7-day TTL
Redis EXPIREAT set to upload_timestamp + 604800
Expired reports auto-deleted

Failure Modes

<i>Failure</i>	<i>Impact</i>	<i>Mitigation</i>
Finder device offline	Report not uploaded	Device queues report; uploads when next online; 7-day window gives slack
No finders near item	Location unknown	"Last known location" shown; AirTag sound plays
Report store unavailable	Owner can't see location	Cached last location shown in Find My app
Key derivation desync	Owner queries wrong keys	Re-sync from AirTag at next Bluetooth contact
Anti-stalk false negative	Stalking victim not alerted	Apple continuously refines detection thresholds; NFC tap as fallback

Privacy Considerations

- Zero-knowledge server^{**}: Apple cannot decrypt any location report; confirmed by security researchers
- No persistent tracking^{**}: key rotation every 15 min prevents location correlation across time windows
- Finder anonymity^{**}: finder device uploads with TLS but report itself has no finder identity
- On-device decryption^{**}: all location decryption happens in Find My app on owner's device
- Anti-stalking^{**}: mandatory sound, iPhone alerts, Android app — industry-first covert tracking countermeasures

23. Design iCloud Keychain

Clarifying Questions

- Scope: password sync only, or passkeys + certificates + secure notes too?
- Cross-device sync: only Apple devices or also iCloud for Windows?
- Account recovery if all devices lost?
- AutoFill in Safari/apps in scope?
- Family Sharing of passwords in scope?

Estimation

```
iCloud Keychain users: ~500M (most iCloud users opt in)
Items per user: avg 200 passwords + 20 passkeys = 220 items
Total items: 500M × 220 = 110B items
Item size: avg 500 bytes (URL + username + encrypted password + metadata)
Total storage: 110B × 500 bytes = 55 TB (compressible, deduped)

Sync events: password save/update: 500M users × 2 changes/day = 1B events/day = ~11,600/sec
AutoFill reads: much higher – 500M users × 20 autofills/day = 10B/day = ~115,000/sec
(Most reads served from local device cache, not server)
```

High-Level Architecture Diagram

■

- XXXXXXXXXXXXXXXXXXXX**

■

■



```
Apple server stores: ciphertext blobs
Apple CANNOT decrypt: does not have iCloud Security Code
```


-

▼

■

▼

1. Fetch changed records from CloudKit
2. Unwrap: `item_key = decrypt(wrapped_key, mac_sync_key)`
3. Decrypt: `item_data = AES-GCM-decrypt(ciphertext, item_key)`

4. Store decrypted item in macOS Keychain
5. Available for AutoFill in Safari

ACCOUNT RECOVERY (if all devices lost):

Option A – iCloud Security Code Escrow:

User's iCloud Security Code (6-digit or custom) is escrowed in Apple HSM

HSM properties:

- Hardware tamper-resistant module (Apple-operated)
- Enforces: max 10 attempts; after that, data wiped
- Apple employees CANNOT override attempt limit
- Requires: Apple ID password + SMS/trusted device 2FA to attempt

Recovery flow:

User logs in on new device with Apple ID

Enters iCloud Security Code

HSM: validates code → releases sync key material

New device derives sync key $\rightarrow$ decrypts items

Option B – Recovery Contact (iOS 15+):

User designates a trusted person as recovery contact

Recovery key split: user keeps 50%, recovery contact keeps 50%

Both halves needed to recover → Apple cannot recover alone

Option C – Recovery Key (28-character):

User-generated recovery key

Stored offline by user

Overrides HSM escrow path

If lost: no recovery (Apple cannot help)

Key Design Decision 1: Zero-Knowledge Architecture

Zero-knowledge means: service provider (Apple) cannot read stored data even if compelled.

iCloud Keychain achieves this via:

1. Key derivation from secrets Apple doesn't have:

Sync Key = $KDF(\text{Device_Key}, \text{iCloud_Security_Code})$

Device Key: in Secure Enclave, never leaves device hardware

iCloud Security Code: only the user knows it; Apple stores HASHED version in HSM

Even if Apple's CloudKit servers are subpoenaed:

- They have: encrypted ciphertext blobs
- They don't have: Sync Key (need Device Key + Security Code)
- They cannot decrypt: mathematically impossible without both secrets

2. iCloud Advanced Data Protection (iOS 16.2+):

Users can opt into E2E encryption for almost all iCloud categories (Photos, Notes, iCloud Drive, etc.)

iCloud Keychain is ALWAYS E2E encrypted (no opt-in needed)

3. HSM-enforced recovery with attempt limit:

If Apple is legally compelled to attempt recovery brute force:

HSM firmware enforces 10-attempt limit

After 10 wrong codes: key material wiped from HSM

Apple cannot override HSM firmware (tamper-evident hardware)

Legal standing: Apple can honestly say "we cannot access this data"

4. CloudKit E2E encrypted zone:

CloudKit has two zones: regular (Apple can decrypt) and E2E (Apple cannot)

iCloud Keychain uses E2E zone exclusively

Zone encryption key generated per user per device cluster

Key Design Decision 2: Passkey (WebAuthn/FIDO2) Architecture

What is a Passkey?

Replaces password with a public-key credential.

User's device generates a key pair per site:

private_key: stored in iCloud Keychain (Secure Enclave preferred)

public_key: registered with the website's server

Login flow:

1. Website sends challenge (random nonce)
2. iOS Face ID / Touch ID authenticates user (local biometric)
3. Secure Enclave signs challenge with private_key
4. Website verifies signature with stored public_key
5. Login granted

Why passkeys are better than passwords:

- Phishing-resistant: private key never leaves device; can't be phished
- No server-stored secrets: website only has public key; breach doesn't expose credentials
- Biometric-bound: requires local authentication; stolen device alone not enough

iCloud Keychain passkey sync:

private_key encrypted with sync_key → stored in CloudKit E2E zone

Syncs to all user's Apple devices

User sets up passkey on iPhone → automatically available on Mac

Cross-platform (Android, Windows):

FIDO2 spec: passkey can be used via QR code + Bluetooth proximity

iPhone shows QR → Windows Chrome scans QR → iPhone authenticates

→ Sends signed response to website via Bluetooth relay

No passkey transferred; iPhone signs and returns result

Key Design Decision 3: Conflict Resolution on Sync

<i>Conflict Type</i>	<i>Strategy</i>	<i>Rationale</i>
Same password updated on two devices simultaneously	Last-Write-Wins on modification timestamp	Passwords are full replacements; no merge needed
Password deleted on one device, updated on another	Deletion wins after grace period	Avoid zombie passwords; user intent is deletion
Passkey created on two devices for same site	Both kept (different key pairs, different device bindings)	Each passkey is independent credential; both valid
Username changed on one, password changed on another	Field-level merge	Treat each field independently; rare in practice

Failure Modes

Failure	Impact	Mitigation
CloudKit unreachable	Items still available from local keychain; sync paused	Local-first: keychain fully functional offline; sync resumes on reconnect
Device lost before sync	Items from that device may not be on other devices	Encourage multi-device users; Recovery Key backup
Security code forgotten	Cannot recover items without Recovery Contact or Recovery Key	Apple warns prominently; Recovery Contact is opt-in backup
HSM exhausted attempts	Account locked out of recovery	By design; security > convenience for this threat model

Privacy Considerations

- Zero-knowledge^{**}: Apple provably cannot read keychain contents; confirmed by security audits
 - Secure Enclave binding^{**}: private keys hardware-bound; can't be exported even from jailbroken device
 - No analytics on passwords^{**}: Apple doesn't know what sites/credentials users have
 - Password monitoring^{**} (Compromised Password feature): checks against HaveIBeenPwned via Private Set Intersection (PSI) — Apple learns only "match or not", not which passwords user has
-

24. Design Apple Music

Clarifying Questions

- Scope: on-demand streaming + offline download + radio (Beats 1)?
- Recommendation engine: personalized playlists (Favorites Mix, New Music Mix)?
- Lyrics sync in scope?
- Social features (friend activity, shared playlists) in scope?
- Lossless / Dolby Atmos spatial audio?

Estimation

Apple Music: ~100M subscribers
Music catalog: 100M+ songs (licensed from labels)
Avg song: 4 min × 256 Kbps AAC = ~7.5 MB; Lossless ALAC = ~30 MB; Dolby Atmos = ~35 MB
Total catalog storage: 100M × 30 MB (lossless) = 3 PB (plus lower-quality renditions = ~5 PB total)

Stream QPS: 100M users × 30 streams/day / 86400 = ~35,000 streams/sec
Offline downloads: 100M users × avg 200 songs = 20B songs downloaded

CDN bandwidth: 35,000 streams × 256 Kbps = ~9 Gbps (+ spikes for new releases)
New album release (Taylor Swift): 10× spike → ~90 Gbps; pre-warm CDN

High-Level Architecture Diagram

[illegible]

HLS with fMP4 segments (same CDN infra as Apple TV+)

AAC 64 Kbps (low data mode)
AAC 256 Kbps (standard – default)
ALAC 16-bit 44.1 kHz (lossless ~1 Mbps)

ALAC 24-bit 192 kHz (hi-res lossless ~4 Mbps)
Dolby Atmos (EC-3) spatial audio ~768 Kbps

DRM:

FairPlay Streaming (same as Apple TV+)
One key per album (not per song) – reduces key server load

OFFLINE DOWNLOADS:

Same FairPlay DRM; persistent keys
Stored in app sandbox (not accessible to other apps)
Key expiry: as long as subscription active
Lapse → keys revoked → offline playback disabled

Key Design Decision 1: Music Recommendation Engine

<i>Approach</i>	<i>Signal</i>	<i>Pros</i>	<i>Cons</i>
Collaborative filtering	"Users who liked X also liked Y"	Discovers non-obvious connections	Cold start for new songs; doesn't use audio features
Content-based (audio ML)	Audio fingerprint similarity	Works for new songs; genre-consistent	Filter bubble; doesn't discover unexpected gems
Matrix factorization (ALS)	Factorize play matrix into latent factors	Scalable; blends implicit signals	Implicit signals noisy (partial plays vs full plays)
Two-tower neural model	User embedding + song embedding	High accuracy; handles context	Expensive to train; needs large labeled data
Editorial curation	Human curators at Apple Music	High quality; trend-aware	Doesn't scale to 100M users personally
Hybrid	Combine all signals with re-ranking	Best of all	Most complex

■ **Recommended: Hybrid — two-tower neural for personalized playlists + editorial for featured content**

User embedding:
 Input: play history (last 500 songs), likes, saves, skips, time-of-day patterns
 Output: 256-dim user vector (taste representation)

Song embedding:
 Input: audio features (tempo, key, energy, danceability from audio ML)
 + metadata (genre, release date, popularity)
 + collaborative signal (who else plays this song)
 Output: 256-dim song vector

Scoring: $\text{score}(\text{user}, \text{song}) = \text{user_vector} \cdot \text{song_vector}$ (dot product)
 Retrieve: FAISS ANN search → top-K similar songs → re-rank by diversity + freshness

Playlists generated daily:
 "Favorites Mix" → personalized by heavy collaborative filtering
 "New Music Mix" → bias toward recent releases + editorial picks
 "Friends Mix" → friends' listening patterns + your taste filter

Personalization privacy:
 User listening data processed on-device where possible (Shortcut suggestions ML)
 Server-side personalization: differential privacy applied to behavioral aggregates

Key Design Decision 2: Catalog Search

Option	Technology	Pros	Cons
Full-text search (Solr/Elasticsearch)	Inverted index on song/artist/album	Fast; relevance ranking; typo tolerance	High memory; index rebuild time
Phonetic search	Soundex / Metaphone algorithms	Handles misspellings phonetically	Imprecise; many false positives
Prefix trie	In-memory trie for autocomplete	Sub-millisecond autocomplete	Only prefix, not mid-string; memory heavy
Embedding-based search	Text embedding similarity	Semantic: "sad songs for rain"	Expensive; less precise for exact matches
Hybrid: BM25 + neural rerank	BM25 retrieval + BERT rerank	Best accuracy	Two-stage latency

■ **Recommended: Elasticsearch (BM25) for catalog search + phonetic analysis for artist names**

```

Index structure:
Document per song:
{ song_id, title, artist_name, album_name, genre,
  release_year, popularity_score, lyrics_snippet }

Custom analyzer for music search:
1. Phonetic: "Beyoncé" matches "beyonce", "bey-onsay"
2. Edge n-gram: "Swif" matches "Swift" (autocomplete)
3. ASCII folding: "Björk" matches "Bjork"
4. Synonyms: "hip hop" = "hip-hop" = "rap"

Ranking signals (BM25 base + boosting):
+ popularity_score boost (trending songs ranked higher)
+ exact title match boost
+ recent plays by this user boost (personalized)
+ editorial boost for Apple Music originals

```

Failure Modes

<i>Failure</i>	<i>Recovery</i>
CDN miss (cold song, long tail)	Origin pull from S3; cold start latency ~500ms
Playback Service key server down	DRM keys cached on device for session; new streams fail; graceful fallback to cached songs
Catalog DB unavailable	Read replica serves reads; search temporarily degraded
Recommendation service slow	Return cached daily playlist; skip real-time personalization
Subscription check fails	Fail open for 24h (allow playback); log for reconciliation

Privacy Considerations

- Listening history^{**}: used for recommendations; user can delete; opt out of personalization
- Friends activity^{**}: opt-in; shows what friends are playing; only visible to mutual followers
- On-device ML^{**}: Siri and Shortcuts leverage on-device listening patterns; not sent to Apple
- DRM and privacy tension^{**}: FairPlay key requests tell Apple what user is playing; Apple sees playback metadata (not content, but behavior)

25. Design Game Center — Leaderboards & Matchmaking

Clarifying Questions

- Scope: global leaderboards only, or friends leaderboards too?
- Game types: turn-based async, real-time multiplayer, or score submission (arcade)?
- Anti-cheat: client-side score validation or server-side?
- Real-time matchmaking: skill-based (ELO) or random?
- Scale: top N leaderboard only or arbitrary rank lookup?

Estimation

```
Game Center users: ~400M
Active games: ~1M games using Game Center SDK
Score submissions: peak during lunch/evening
    400M users × 5 game sessions/day × 1 score/session = 2B scores/day = ~23,000/sec peak

Leaderboard reads:
    "My rank" + top 100 = ~400M × 10 queries/day = 4B/day = ~46,000/sec
    Friends leaderboard: filter global to friends list

Matchmaking requests: ~5M concurrent games; ~1M new matches/hour = ~280/sec

Top leaderboard size: top 10,000 scores stored in Redis sorted set
Full leaderboard: 400M entries per game × 1M games = 400T entries (cannot store all in Redis)
```

High-Level Architecture Diagram

[illegible]

- Game ends: score = 1,000,000
- Sign score: score_payload = { player_id, game_id, score, timestamp }
- signature = HMAC-SHA256(score_payload, game_secret_key)
- (prevents tampering; game_secret_key embedded in app + verified server-side)

▼

- Validate signature
- Anti-cheat: is score within plausible range?
- is submission rate reasonable?
- is score progression suspicious?

▼

- Write to: Cassandra (all scores, durable long-term storage)
- Write to: Redis sorted set (top 10K scores for fast leaderboard reads)
- ZADD leaderboard:{game_id} score player_id

▼

```
Async: update friend leaderboards, trigger achievements, update stats
```

ZREVRANGE leaderboard:{game_id} 0 99 WITHSCORES
 $O(\log N + 100)$ – extremely fast

ZREVRANK leaderboard:{game_id} {player_id}
 $O(\log N)$ – fast for top-10K players

Approximate rank from Cassandra: count players with higher score
 SELECT COUNT(*) FROM scores WHERE game_id=? AND score > my_score
 → Returns approximate rank; not exact (acceptable for low-rank players)

```

Friends list: fetch player's friend IDs (social graph service)
For each friend: ZSCORE leaderboard:{game_id} {friend_id}
Sort client-side (friend list typically < 200)
Friends not in top-10K: fetch score from Cassandra

```

XX

- { game_id, player_id, skill_rating, region, device_latency }

▼


```
Matchmaking pool per (game_id, region, skill_bucket):  
    skill_bucket = floor(skill_rating / 100) // bucket by 100-point ranges
```

Match attempt:

1. Look for waiting player in same skill_bucket
2. If no match in 5s: expand to ± 1 bucket
3. If no match in 15s: expand to ± 3 buckets
4. If no match in 30s: match with anyone (avoid infinite wait)

Match found:

```
    Assign game session  
    Notify both players via APNs  
    Players connect to game relay server
```



[GAME RELAY SERVICE]

```
    UDP relay for game state packets  
    Low-latency: edge servers per region  
    Encrypted: DTLS  
    Timeout: session cleanup after 30s inactivity
```

Deep Dive: Leaderboard Scaling with Sorted Sets + Approximate Ranking

Redis Sorted Set:

Data structure: skip list + hash map

Operations:

ZADD: $O(\log N)$ – insert/update score
ZRANK: $O(\log N)$ – get rank of element
ZRANGE: $O(\log N + M)$ – get top M elements
ZSCORE: $O(1)$ – get score of element

Problem: 400M players per game $\times$ 1M games = 400 trillion entries.
Redis sorted set practical limit: ~1M entries for fast operations.

Solution: Tiered leaderboard

Tier 1 (Redis): top 10,000 players

Exact rank for anyone in top 10K.

Updated synchronously on score submission.

Memory: $10,000 \times (\text{player_id: 16 bytes} + \text{score: 8 bytes}) \times 1\text{M games} = 240\text{ GB Redis}$
(acceptable; Redis Cluster with sharding per game_id)

Tier 2 (Cassandra): all scores

Schema: (game_id, period, score DESC, player_id) $\rightarrow$ PRIMARY KEY

Exact count for arbitrary rank: $O(N)$ full scan $\rightarrow$ too slow.

Approximate rank for low-rank players:

Pre-compute score distribution histogram:

Buckets: [0-100], [100-1000], [1000-10000], ...

Store bucket counts: { game_id $\rightarrow$ { bucket $\rightarrow$ count } }

Update async: Kafka consumer updates histogram every 5 min.

Approximate rank:

rank $\approx$ SUM of players with score $>$ my_score

$\approx$ SUM of bucket counts above my_score bucket

Error: within $\pm$ bucket_size (e.g., ± 100 for bucket size 100)

Acceptable: "You are approximately rank 2.4 million" is fine.

Exact rank for top 10K (where it matters for prizes/recognition).

HyperLogLog for unique player counts:

Problem: how many unique players have played today?

Exact count: store all player_ids $\rightarrow$ too much memory.

HyperLogLog: probabilistic data structure, 12KB $\rightarrow$ estimates cardinality with 0.8% error

PFADD unique_players:{game_id}:{date} {player_id}

PFCOUNT unique_players:{game_id}:{date} // 0.8% error, constant 12KB memory

Deep Dive: Anti-Cheat Score Validation

Client-side score submission is inherently untrusted.

Three defense layers:

1. HMAC signature:

Score payload signed with game-specific secret.

Prevents casual tampering.

Weakness: secret embedded in app binary → can be extracted by determined cheater.

2. Server-side plausibility checks:

Max score per session: if game's max theoretical score is 100,000 → reject 999,999,999

Submission rate: can't submit score without a game session; session token validated

Score progression: if player went 100 → 500 → 1,000,000 in 1 day → flag

3. Server-side game simulation (for important games):

Client sends game replay data (input sequence)

Server replays and verifies final score

Expensive: only for tournaments, top-N prize games

4. Social graph anomaly:

New account, no friends, immediately #1 global → flag

Cluster of accounts all submitting same suspicious score → ban cluster

Machine learning:

Train on labeled cheat/legit sessions.

Features: score, session duration, device model, account age, social graph depth.

Real-time scoring: flag suspicious submissions; human review queue for top-100 players.

Failure Modes

Failure	Recovery
Redis sorted set unavailable	Serve leaderboard from Cassandra (slower, approximate)
Score submission spike (major game launch)	Kafka queue absorbs burst; async writes to Redis
Matchmaking pool empty	Expand skill range; bot match for new game; notify user
Anti-cheat false positive	Appeal process; human review; restore falsely removed scores

Privacy Considerations

- Score data**: public by default (leaderboard is public); friends leaderboard is social-scoped
- Game session data**: not retained by Apple after matchmaking; game developer's responsibility
- Social graph**: friend list accessed by Game Center API; friend connections are Apple ID linked
- Anti-cheat analytics**: behavior patterns analyzed server-side; results are ban decisions, not profiling

26. Design HomeKit / Matter Smart Home Hub

Clarifying Questions

- Scope: local automation only, or remote access when away from home?
- Protocols: Matter only, or legacy (Z-Wave, Zigbee, proprietary bridges)?
- Hub device: HomePod, Apple TV, iPad as home hub?
- Scale: typical home (10-50 devices) vs enterprise/hotel (1,000+ devices)?
- Voice control via Siri integration in scope?
- Family sharing of home access?

Estimation

HomeKit homes: ~50M homes with at least one HomeKit device
Avg devices per home: 15 (lights, locks, sensors, thermostats, cameras)
Total devices: $50M \times 15 = 750M$ HomeKit devices

Local event rate: per home, ~1 event/sec avg (motion sensor, state change, automation trigger)
 $50M \text{ homes} \times 1 \text{ event/sec} = 50M \text{ events/sec system-wide}$
BUT: 99% handled locally on hub – only remote access hits Apple servers

Remote access events: $10\% \text{ of homes} \times 10 \text{ events/day} = 50M \text{ remote events/day} = \sim 580/\text{sec}$
(Very manageable – local-first design dramatically reduces server load)

Automation latency SLA: <100ms local (turn on light when motion detected)
Remote access latency: <500ms acceptable

High-Level Architecture Diagram

XXXXXXXXXXXXXXXXXXXX

[REDACTED]

- XXXXXXXXXXXXXXXXXXXXXXXXXXXX**

[illegible]

■

■

■

■

Page 122 of 137

Key Design Decision 1: Automation Engine — Local vs Cloud

<i>Option</i>	<i>Latency</i>	<i>Reliability</i>	<i>Privacy</i>
Fully local (hub)	<50ms	Works during internet outage	Excellent: no data leaves home
Fully cloud	200-500ms	Requires internet	Poor: all events go to cloud
Local + cloud sync	<50ms local, 200ms remote	Local works offline; cloud for remote access	Good: only remote access uses cloud
Edge + cloud hybrid	<50ms local, <200ms cloud	Best redundancy	Tiered

■ Recommended: Local-first with CloudKit relay for remote access — Apple's actual design

```
Local automation execution:
  Trigger: motion sensor fires (Thread mesh, <10ms to hub)
  Hub automation engine:
    IF sensor.motion == detected
    AND time BETWEEN 8pm AND 11pm
    AND home_state == "Someone home"
    THEN light.brightness = 80%
        light.color_temp = 2700K (warm evening light)
  Execute: Hub sends command to bulb via Matter
  Total: motion to light-on < 100ms
  Internet NOT required.

Backup hub:
  Multiple HomePods / Apple TVs designated as home hubs
  Primary hub: lowest ping to most devices
  If primary offline: secondary takes over via CloudKit coordination
  User doesn't notice; failover < 30s

Remote access security:
  User authenticates with Face ID / Touch ID locally on iPhone
  Session key derived for this session; short-lived (1 hour)
  Command encrypted before reaching Apple relay
  Even Apple employees cannot eavesdrop on home commands
```

Key Design Decision 2: Thread Mesh Networking

Problem: Wi-Fi devices drain battery (continuous connection).
Motion sensors, door sensors run on coin cells for years.
Cannot be on Wi-Fi (too power hungry).

Thread: Low-power mesh network (IEEE 802.15.4)

Mesh topology:

Router nodes: powered devices (smart plugs, thermostats) forward packets
End devices: battery-powered (sensors) sleep most of the time
Border router: Thread ↔ IP bridge (HomePod or Apple TV)

Power profile:

End device: 2µA sleep current; wakes for 1ms to send reading
AA battery: ~2 years of operation

Self-healing:

If one router goes down → mesh automatically re-routes
Multiple paths → high reliability for critical devices (smoke detector)

IP-native:

Every Thread device has an IPv6 address
Matter runs directly over Thread
No proprietary hub protocol needed (unlike Zigbee or Z-Wave)

Apple's role:

HomePod and Apple TV contain Thread Border Router functionality
Manage Thread network partition/merge
Expose Thread devices to local IP network and HomeKit

Failure Modes

<i>Failure</i>	<i>Impact</i>	<i>Mitigation</i>
Hub (HomePod) offline	No remote access; local automations via secondary hub	Multiple hub support; automations stored on all hubs
Thread border router failure	Thread devices unreachable	HomePod + Apple TV both act as border routers; redundant
Internet outage	Remote access fails; local automations work fine	By design: local-first means core functionality offline-capable
Device firmware update breaks compatibility	Device stops responding	Matter specification enforces backward compatibility; rollback via hub
CloudKit relay outage	Remote access unavailable	No impact to local; degrade gracefully; cached state for read-only

Privacy Considerations

- Local processing^{**}: automations execute on-hub; no home behavior sent to Apple
 - Camera footage^{**}: HomeKit Secure Video (HKSU) — video analyzed on-device for events (people, vehicles); only clips stored in iCloud E2E encrypted
 - No behavioral analytics^{**}: Apple does not mine automation patterns for advertising
 - Third-party devices^{**}: Matter-certified devices have minimum security requirements; Apple audits HomeKit certification
 - Guest access^{**}: HomeKit home sharing with access control; guest access can be time-limited
-

27. Design Apple ID & Authentication Platform

Clarifying Questions

- Scope: Apple ID login, Sign in with Apple (3rd party), or both?
- Passkey support (FIDO2/WebAuthn)?
- Multi-factor: SMS, trusted device, security key?
- Account recovery flows in scope?
- Enterprise (Apple Business Manager) or consumer?
- Scale: peak during device activation events (new iPhone launch day)?

Estimation

```
Apple ID accounts: ~1.5B
Active sessions: ~500M concurrent (devices always signed in)

Authentication events:
  New sign-in: 50M/day = ~580/sec
  Token refresh: ~1B/day = ~11,600/sec (silent background refresh)
  2FA requests: 20% of new sign-ins = 10M/day = ~115/sec

Peak (new iPhone launch day): 5-10x normal new sign-ins = ~5,000 new auth/sec
Sign in with Apple (3rd party): 1B+ authentications/day across millions of apps

Session token lifetime:
  Access token: 1 hour (short-lived, JWT)
  Refresh token: 6 months (long-lived, opaque)

Account recovery: ~1M requests/month (complex, high-stakes operation)
```

High-Level Architecture Diagram

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX


```
{ apple_id: "user@icloud.com", password: SRP_client_proof }
```


▼

-

▼

■

■

■

A [illegible]

▼

i0:

```
User confirms: share name/email with app?
Option: hide real email (use relay address)
Face ID / Touch ID authenticates
■
▼
[APPLE OAUTH SERVER]
Issue authorization code (one-time, short-lived)
■
▼
[Third-party app backend]
Exchange code for:
  id_token: JWT with { sub (stable user ID), email, name }
  access_token: for userinfo endpoint
■
Validate id_token: verify RS256 signature with Apple's public keys
sub is stable, opaque (not Apple ID email) – user's real email not exposed
■
▼
[Third-party app] creates/signs in user account using sub as identity

PRIVATE EMAIL RELAY:
If user chose "Hide My Email":
  Apple creates random relay address: xyz123@privaterelay.appleid.com
  Third-party app sends email to relay address
  Apple forwards to user's real email
  App never learns real email
  User can disable relay → stop receiving from that app
```

Key Design Decision 1: Password Protocol — SRP vs Standard Auth

Standard password auth (what most services use):

Client → HTTPS POST { email, password } → Server

Server: hashes password, compares with stored hash

Problems:

- Password transmitted to server (even over TLS)
- If TLS intercepted or server compromised: password exposed
- Server stores hash: if DB leaked, offline attack on hash possible

SRP (Secure Remote Password, RFC 2945):

Mathematical zero-knowledge proof.

Client proves it knows the password WITHOUT sending it.

Setup (registration):

salt = random()

$x = H(\text{salt} || H(\text{email} || ":" || \text{password}))$ // private, not stored

$v = g^x \bmod N$ // verifier, stored server-side

Server stores: { salt, v } // v reveals nothing about password

Auth protocol (simplified):

1. Client → Server: username

2. Server → Client: salt, $B = (kv + g^b) \bmod N$ (server ephemeral)

3. Client computes:

$x = H(\text{salt} || H(\text{email} || ":" || \text{password}))$

$u = H(A || B)$ ($A = g^a \bmod N$, client ephemeral)

$S = (B - kg^x)^{(a + ux)} \bmod N$ // shared secret

$M1 = H(A || B || S)$ // proof that client knows S

4. Client → Server: A, M1

5. Server verifies M1 using stored v

If valid: server sends $M2 = H(A || M1 || S)$ // server proof

6. Client verifies M2

Properties:

- Password never in transit (only proofs)
- Server never sees password (only verifier v)
- MITM cannot extract password from captured traffic
- DB breach: attacker gets v but can't recover password from v efficiently

Cost: 2 round trips (vs 1 for basic auth)

Apple uses SRP for Apple ID authentication (documented in security whitepaper)

Key Design Decision 2: Token Architecture

Token Type	Format	Lifetime	Stored Where	Use
Access token	JWT (RS256 signed)	1 hour	Client memory only	API authorization (short-lived, stateless)
Refresh token	Opaque random	6 months	Client keychain + server DB	Get new access token without re-login

<i>Token Type</i>	<i>Format</i>	<i>Lifetime</i>	<i>Stored Where</i>	<i>Use</i>
ID token	JWT (OIDC)	Single use	Client (verify and discard)	Sign in with Apple identity assertion
Device certificate	X.509	Years	Secure Enclave	iCloud E2E encryption, 2FA trusted device

JWT Access Token structure:

Header: { alg: RS256, kid: key_id }

Payload:

```
{ sub: apple_id_hash,          // stable user identifier
  iss: "appleid.apple.com",
  aud: "com.apple.icloud",    // intended service
  iat: 1716000000,
  exp: 1716003600,           // 1 hour
  scope: ["icloud", "music", "tv"],
  device_id: hash(device_cert),
  risk_level: "low" }
```

Signature: RS256(header.payload, apple_private_key)

Stateless validation:

Services validate JWT locally using Apple's public key (from JWKS endpoint)

No need to call Auth Service for every API request

Revocation: wait for expiry (1hr max); or maintain small revocation list for compromised tokens

Refresh token rotation:

Each use of refresh token → issues new refresh token + new access token

Old refresh token immediately invalidated

Protects against refresh token theft: attacker's use triggers rotation → victim can't use old token → gets err...

Deep Dive: Passkeys (FIDO2/WebAuthn) at Apple Scale

Passkey registration:

1. Service sends registration challenge
2. iOS prompts Face ID / Touch ID
3. Secure Enclave generates key pair:
 - private_key: stays in Secure Enclave (hardware-bound)
 - public_key: sent to service
4. Service stores: { user_id, credential_id, public_key }

Passkey authentication:

1. Service sends auth challenge
2. iOS prompts Face ID / Touch ID
3. Secure Enclave signs challenge with private_key
4. Service verifies signature with stored public_key
5. Login granted

Why this eliminates phishing:

private_key is bound to exact origin (e.g., appleid.apple.com)
If phishing site (apple.com) requests signature:
WebAuthn checks origin → different domain → refuses to sign
User cannot accidentally sign a phishing challenge

iCloud Keychain sync:

private_key encrypted with iCloud Keychain sync_key
Stored in CloudKit E2E zone
Available on all user's Apple devices

Question: if private_key leaves Secure Enclave for sync, is it still secure?

Answer: the key is encrypted with sync_key before leaving Secure Enclave
sync_key is also hardware-derived
On destination device: decrypted into destination Secure Enclave
Key travels encrypted, lives in Secure Enclaves
Server never sees plaintext key

Failure Modes

Failure	Recovery
Auth service region down	Active-active across 3+ regions; requests routed to healthy region
Session store (Redis) down	Refresh tokens validated against SQL fallback; higher latency
2FA SMS delivery failure	Retry SMS; fallback to trusted device push
Passkey lost (all devices wiped)	Recover via iCloud Keychain recovery (HSM-guarded); register new passkey
Token signing key compromise	Rotate key; invalidate all access tokens; users re-authenticate

Privacy Considerations

- Sign in with Apple privacy^{**}: stable sub (user ID) is different per app — app A cannot correlate with app B
 - Private email relay^{**}: user's real email hidden from third-party apps
 - No tracking across apps^{**}: opaque sub prevents Apple ID from becoming a cross-app tracking identifier
 - Differential Privacy in 2FA analytics^{**}: Apple uses DP to analyze 2FA usage patterns without linking to individuals
 - SRP^{**}: Apple's servers never see plaintext passwords; breached DB doesn't expose passwords
-

28. Design iCloud Calendar & Contacts Sync

Clarifying Questions

- Protocols: CalDAV/CardDAV compliance or proprietary?
- Scope: personal calendars only, or shared family/team calendars?
- Meeting invites (iTIP/iMIP) and RSVPs in scope?
- Recurring events with exceptions in scope?
- Scale: iCloud users only, or also Google Calendar / Exchange bridging?
- Conflict resolution strategy when same event edited on two devices?

Estimation

```
iCloud Calendar users: ~500M
Avg calendars per user: 3 (personal, work, shared family)
Avg events per user: 500 active events + 5,000 past = 5,500
Total events: 500M × 5,500 = 2.75T events in system

Event size: ~500 bytes (title, time, recurrence, attendees, notes)
Total storage: 2.75T × 500 bytes = 1.4 PB

Sync events: user creates/edits/deletes ~3 events/day
500M × 3 / 86400 = ~17,400 changes/sec
Sync reads (pull on device wake): 500M × 10 syncs/day = ~58,000/sec

Meeting invites: 10% of events have attendees
500M × 500 events × 0.1 = 25B active invite-bearing events
Invite delivery: ~1M invites/day = ~11.6/sec
```

High-Level Architecture Diagram

XX

■■■ Event (VEVENT)

```

#### UID: globally unique (e.g., uuid@icloud.com)
#### DTSTART: 2026-06-15T09:00:00Z
#### DTEND: 2026-06-15T10:00:00Z
#### SUMMARY: "Team Standup"
#### RRULE: FREQ=WEEKLY;BYDAY=MO,TU,WE,TH,FR (recurring rule)
#### EXDATE: 2026-06-22T09:00:00Z (exception: this instance cancelled)
#### ORGANIZER: mailto:alice@icloud.com
#### ATTENDEE: mailto:bob@icloud.com;PARTSTAT=ACCEPTED
#### ATTENDEE: mailto:carol@corp.com;PARTSTAT=NEEDS-ACTION
#### SEQUENCE: 3 (version counter; increments on each edit)

```

██████████

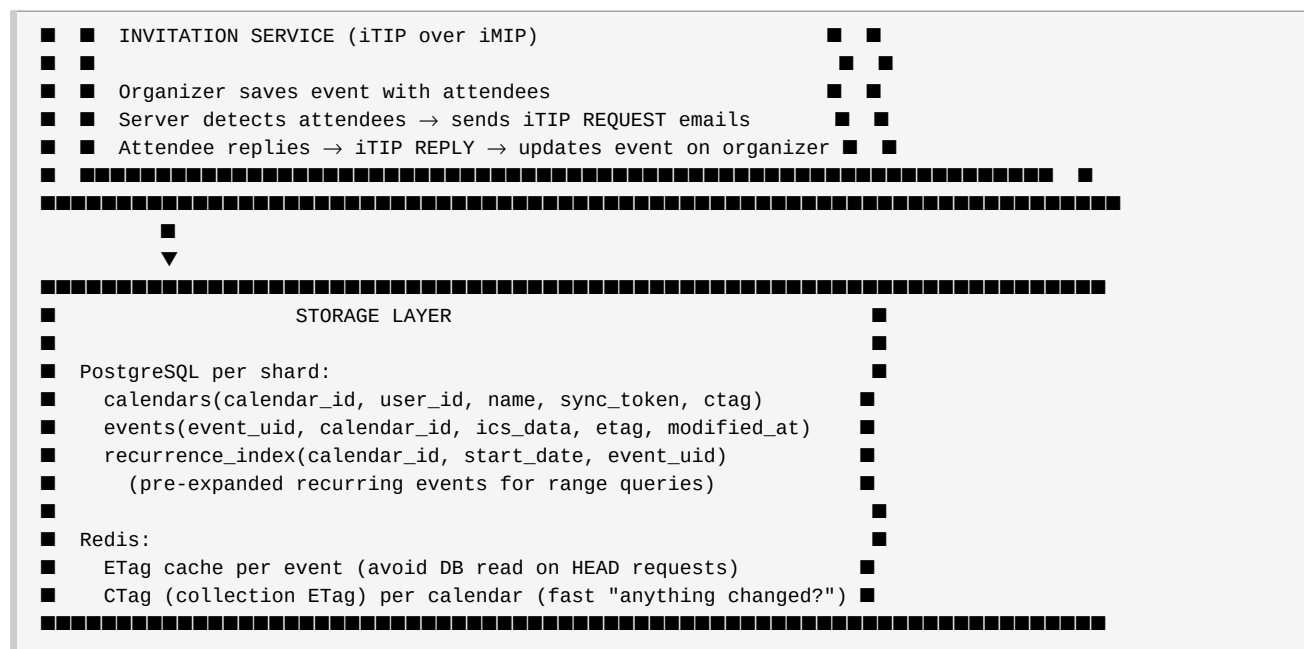
- On wake / every 15 min / push notification:
- PROPFIND /caldav/user/{uid}/calendar/ (list all calendars + ETags)
- Compare ETags with local cache
- For changed ETags: GET /caldav/user/{uid}/calendar/{event_uid}.ics
- For deleted events: 404 response = deleted
- For local changes: PUT /caldav/user/{uid}/calendar/{event_uid}.ics
- If-Match: {old_ETAG} (optimistic concurrency)

[REDACTED]

```

■          CALDAV SERVER
■
■
■
■ ■ SYNC COLLECTION SERVICE
■ ■
■ ■ CalDAV sync-collection report:
■ ■ Returns only changes since sync-token N (not full re-fetch)
■ ■ sync-token: monotonic counter per calendar
■ ■ Client sends: REPORT (sync-token: 12345)
■ ■ Server returns: changed/deleted since token 12345
■
■
■
■ ■ CONFLICT RESOLUTION
■ ■
■ ■ Optimistic concurrency via ETag:
■ ■   PUT If-Match: {ETag}
■ ■   If server version changed → 412 Precondition Failed
■ ■   Client must: fetch latest → merge → retry PUT
■
■
■

```



Key Design Decision 1: Recurring Event Storage — Expand vs Rule

Option	Storage Model	Query	Pros	Cons
Store rule only (RRULE)	One record per event with RRULE string	Compute instances at query time	Minimal storage; handles infinite recurrences	Query "all events in June" requires expanding rules in app/server
Pre-expand recurring events	One record per instance	Direct range query	Fast range queries; simple	Infinite recurrences impossible; storage explosion for daily events over years
Hybrid: rule + expanded index	Store RRULE + index for next N instances	Range query on index	Fast queries + handles infinite	Must maintain index; consistency between rule and index
Materialized view	Rule in main table; expanded view refreshed	Query view	DB handles consistency	View refresh lag

■ **Recommended: Hybrid** — store RRULE in events table + separate recurrence_index for range queries

```
recurrence_index schema:
  (calendar_id, occurrence_date, event_uid, is_exception)

Pre-expand: up to 2 years ahead (covers most use cases)
Expand more on demand (user scrolls calendar further out)

For "all events June 2026":
  SELECT event_uid FROM recurrence_index
  WHERE calendar_id = ?
  AND occurrence_date BETWEEN '2026-06-01' AND '2026-06-30'

Exceptions (EXDATE, modified occurrences stored as VEVENT with RECURRENCE-ID):
  is_exception = true → override row for that specific instance
  Deleted exception: is_exception = true, deleted = true

Index maintenance:
  New recurring event created → background job expands + inserts index rows
  RRULE modified → delete future index rows + re-expand
  Job runs async; slight lag OK (calendar is not real-time critical)
```

Deep Dive: CalDAV ETag-Based Sync

ETag (Entity Tag): opaque string that changes when the resource changes.

Event created: ETag = hash(event content + timestamp) = "abc123"

Event updated: ETag changes = "def456"

Sync flow (optimistic, efficient):

Initial sync:

Client: PROPFIND /calendar/ (depth: 1)

Server: returns all event UIDs + their ETags

Client: download any event not in local cache

Incremental sync (CalDAV sync-collection):

Client stores sync-token (monotonic counter from server)

Client: REPORT /calendar/ with { sync-token: 12345 }

Server: returns only changes since token 12345:

{ changed: [event1, event5], deleted: [event3] }

Client fetches changed events, removes deleted

Why sync-token > polling:

Without sync-token: must compare all ETags → N requests for N events

With sync-token: one request → O(changed_items) response

For user with 5,000 events, sync in <100ms instead of 5,000 HEAD requests

Conflict detection (two devices edit same event):

Device A reads event (ETag: "abc123")

Device B reads event (ETag: "abc123")

Device A saves: PUT If-Match: "abc123" → success → new ETag: "def456"

Device B saves: PUT If-Match: "abc123" → 412 Precondition Failed

Device B must:

GET latest event (ETag: "def456") = Device A's version

Merge: field-level merge if possible

(Device B changed title, Device A changed time → both changes apply)

(Both changed title → Device B's version wins with user notification)

PUT with new ETag

Field-level merge logic:

Fields changed by only one side: take that side's value

Fields changed by both sides: take higher SEQUENCE number's value

SEQUENCE: organizer increments on each meaningful edit

Last SEQUENCE wins for conflicted fields

Key Design Decision 2: Invitation System (iTIP/iMIP)

iTIP (iCalendar Transport-Independent Interoperability Protocol, RFC 5546):

Defines METHOD values for calendar coordination:

REQUEST: invite attendees to event
REPLY: attendee responds (ACCEPTED/DECLINED/TENTATIVE)
CANCEL: organizer cancels event
COUNTER: attendee proposes time change
DECLINECOUNTER: organizer declines counter-proposal
REFRESH: attendee requests latest event details

iMIP (iCalendar Message-Based Interoperability Protocol, RFC 6047):

Delivers iTIP messages via email (MIME attachment: text/calendar)

Allows cross-platform invites: Apple Calendar ↔ Google Calendar ↔ Outlook

Invite flow (cross-platform):

Apple user (alice@icloud.com) invites Google user (bob@gmail.com):

1. Alice creates event, adds Bob as attendee
2. CalDAV server: detects external attendee (not iCloud)
3. Server sends email to bob@gmail.com:
MIME-Type: text/calendar; method=REQUEST
Attachment: event.ics with METHOD:REQUEST
4. Google Calendar parses .ics attachment → adds event to Bob's calendar
5. Bob accepts → Google sends reply email to alice@icloud.com:
Attachment: event.ics with METHOD:REPLY; PARTSTAT=ACCEPTED
6. Apple CalDAV server processes reply → updates Alice's event attendee status

Internal Apple invite (both users on iCloud):

Skip email entirely.
Server-side: push invite directly to Bob's CalDAV account
Much faster, no email server dependency

Failure Modes

Failure	Recovery
Sync conflict (412)	Client fetch-merge-retry; show user conflict notification for unresolvable conflicts
CalDAV server unavailable	Device caches last-known state; shows stale data clearly; queues changes for retry
Recurring event expansion job behind	Range queries return partial results; background job catches up; no data loss
Invite email delivery failure	Retry with exponential backoff; in-app notification as backup for Apple-to-Apple
Time zone database out of date	Floating time events (no timezone) safe; zoned events may show wrong time; OS tzdata update required

Privacy Considerations

- Calendar data sensitivity**: high — reveals location, meetings, personal events
 - iCloud Keychain / Advanced Data Protection**: Calendar included in E2E encryption when user enables Advanced Data Protection (iOS 16.2+)
 - Without ADP**: iCloud servers can read calendar data (encrypted in transit + at rest, but Apple has keys)
 - Invites and email relay**: organizer sends to attendee email; Apple does not inspect invite content for ads
 - Siri suggestions**: "Leave for meeting" uses on-device calendar access; location not sent to Apple
 - Shared calendars**: access controlled by owner; audit log of who accessed shared calendar
-

This guide covers all 28 Apple-relevant system design questions with:

- Multiple technology options for each key decision*
- Trade-off tables for every choice*
- Best recommendation with rationale*
- ASCII architecture diagrams*
- Deep dives on critical keywords (HLS/ABR, FairPlay DRM, DTLS-SRTP, RGA CRDTs, point-in-time correctness, HSM, payment tokenization, Protobuf schema evolution, ECIES crowd-sourced location privacy, SRP zero-knowledge auth, WebAuthn/passkeys, CalDAV ETags, iTIP/iMIP, Thread mesh networking, Matter protocol, etc.)*